



SPRIN-D Validierungsauftrag

Abschlussbericht

Dieser neuartige mathematische Coprozessor zählt auch ein auf eine wichtige Thematik von SPRIN-D:

„In einer digitalen Welt der Zukunft wird **der Alltag und die Arbeitswelt um uns herum permanent durch Künstliche Intelligenz unterstützt**. Daten aus unzähligen Sensoren und Aktoren werden durch kleinste Rechner ausgelesen, verarbeitet, über ein Netz von Datenstrecken zusammengeführt in größere Knotenpunkte mit mehr Rechenleistung, interpretiert, zurückgeführt. Sie verändern, steuern, unterstützen, bewegen unser Leben. Dazwischen sitzen riesige Rechenzentren, die die großen Aufgaben übernehmen, Daten verwalten und lenken, künstliche Gehirne trainieren und komplexe wissenschaftliche Probleme lösen.

Doch diese Vision einer vernetzten Welt hat derzeit einen hohen Preis. Wenn der weltweite Energieverbrauch für Computing und Kommunikation mit der gleichen Geschwindigkeit zunimmt wie bisher, wird er **bereits im Jahr 2040 die gesamte, weltweite Kapazität zur Energieproduktion einnehmen**. Lange ist der Energieverbrauch gesunken – weil die Chipstrukturen immer kleiner geworden sind. Doch hier stoßen die Entwicklungen nun an ihre physikalischen Grenzen.

Um unsere Zukunftsvisionen – vom autonomen Fahren über eine computergestützte Medikamentenentwicklung bis zur intelligenten Steuerung unzähliger erneuerbarer Energiequellen – umsetzen zu können, **müssen daher grundlegend neue Rechenkonzepte gefunden werden.**“¹

Vertragsnummer der SPRIN-D:

Projektnummer der SPRIN-D:

¹ Quelle : <https://www.sprind.org/de/challenges/newcomputing>

Inhaltsverzeichnis - Validierungsauftrag Abschlussbericht

1. Ziel des Validierungsauftrags	5
1.1 Unteraufgaben/-fragen.....	5
1.2 Meilensteine	6
2. Zusammenfassung der Ergebnisse.....	7
3. Die eingesetzte Technologie	10
3.1 System-on-a-Chip (SoC) mit ARM-Prozessoren und FPGA.....	11
3.2 PetaLinux und die gewählten Softwarekomponenten auf dem SoC	13
3.3 Berechnungen mit der mathematischen Engine as a Service auf dem SoC	16
3.4 Arbeitsweise des Zielsystems.....	21
4. Begleitende Technologien der paceval . Entwicklung	23
4.1 Die Bibliothek von paceval . und ihre Funktionen.....	23
4.2 paceval -service und Docker Container	27
4.3 Hyperscaler und Skalierung mit Kubernetes.....	28
4.4 Python Interface	30
4.5 Eclipse Diafanis Projekt.....	31
5. Abschnitt 1: Voraussetzung zur Durchführung der Validierung	34
6. Abschnitt 2: Ergebnisse.....	34
6.1 Erzielte Ergebnisse bis zum Zwischenbericht	34
6.2 Arbeiten seit dem Zwischenbericht	36
6.3 Arbeitsbeschreibung AP0 „Kauf und Inbetriebnahme der für die Validierung benötigten Hardware“	38
6.4 Arbeitsbeschreibung AP1 „Lauffähigkeit von paceval. auf den ARM-Prozessoren auf dem FPGA“	39
6.5 Arbeitsbeschreibung AP2: Grundlegende mathematische Funktionen beschleunigen	40
6.6 Arbeitsbeschreibung AP3: Optimierung der Funktionalität	41
6.7 Arbeitsbeschreibung AP4: Dokumentation und Beantwortung aller obenstehenden Fragen zum Validierungsauftrag.....	43
6.8 Voraussichtlicher Nutzen insbesondere Verwertbarkeit des Ergebnisses	46
6.9 Während der Durchführung des Forschungs- und Entwicklungsauftrags der Auftragnehmerin bekannt gewordener Fortschritt auf dem Gebiet des Auftrags bei anderen Stellen	49
6.10 Problematik der regulatorischen Zulassung eines Produktes mit KI und Verfahren zur Lösung	51
6.11 Erfolgte oder geplante Veröffentlichungen des Forschungs- und Entwicklungsergebnisses...	55
7. Abschnitt 3: Erfolgskontrollbericht	55

Anhang A: paceval-Übergabe Quellcode an OFFIS	58
Arbeitsweise des neuen Quellcodes	58
Die Aufrufsequenz der C-Funktionen für die FPGA-Berechnungen und die wichtigsten Werte-/Variablenparameter	60
Die Erzeugung und Verwaltung der benötigten Datenstrukturen.....	62
Verwendung im Validierungsprojekt	63
Erzeugen der ARM32 Beispiele zum Testen und der <i>paceval</i> . shared library	64
Alle Beispiele als Netzwerkversionen zum Testen gegen das Referenzdesign.....	64
Das ZC706 <i>paceval</i> . Petalinux Image	64
Anhang B: Expertenpapier-Umwandlung von neuronalen Netzen in geschlossene Ausdrücke	65
1 Einleitung und Ziele dieses Whitepapers.....	66
1.1 Tensoren	67
1.2 Automatische Differenzierung	68
1.3 Architekturen Neuronaler Netze	68
2. Methoden	70
2.1 Dynamische Verfolgung mit der Programmiersprache Julia	70
2.2 Einzelheiten zur Implementierung des Programmcodes.....	70
2.3 Überblick über den gesamten Arbeitsablauf	73
3. Verbesserungen und Lösungen.....	74
3.1 Exponentielles Wachstum der exportierten Formeln.....	74
3.2 Wachstum der Formeln mit zunehmender Größe der Eingaben	74
Anhang C: OFFIS-Machbarkeitsstudie für die Umsetzung der paceval-Software in FPGA-Hardware..	76
1. Einleitung	78
2. Untersuchungsziele.....	78
2.1 Rechenleistung.....	79
2.2 Kosteneffizienz.....	80
2.3 Energieeffizienz.....	80
3. Evaluationsmethode	81
3.1 Systemarchitektur.....	81
3.2 Implementation des Prototyps	82
3.3 Hardware-Plattform.....	83
3.4 Messverfahren	83
4. Ergebnisse	86
4.1 Rechenleistung und Ressourcenbedarf	86
4.2 Kosteneffizienz.....	87
4.3 Energieeffizienz.....	87

5. Bewertung.....	88
5.1 Rechenleistung.....	88
5.2 Kosteneffizienz.....	89
5.3 Energieeffizienz.....	89
6. Zusammenfassung	90
7. Quellenangaben	91
Anhang D: SAP-Empfehlungsschreiben	92
Abbildungsverzeichnis - Validierungsauftrag Abschlussbericht	93

1. Ziel des Validierungsauftrags

Im Rahmen eines Validierungsauftrags soll evaluiert werden, inwiefern die mathematische Engine **paceval** in Form eines Field Programmable Gate Array (FPGA)² in Hardware umgesetzt werden kann. Dies wäre die erste Prüfung der Idee, einen neuartigen mathematischen Coprozessor in Hardware als integrierte Schaltung (ASIC)³ in einem Chip mit besonderer Effizienz der Technologie bereitzustellen.

1.1 Unteraufgaben/-fragen

Als Bestandteil dieses Ziels der Validierung sollen die folgenden Unteraufgaben/-fragen beantwortet werden:

1. Welche Teile der **paceval** Engine können in Hardware umgesetzt werden? Können beispielsweise Elementaroperationen wie $+/-/*/:exp$ und parallele Ausführung von (Teil-) Funktionen umgesetzt werden?
2. Wie ist das Zusammenspiel zwischen der Hardware Implementierung und Komponenten, die weiterhin in Software vorhanden sind (z.B. mit Compiler)? Wie kann dies im Hinblick auf eine optionale Performance am besten gestaltet werden?
3. Wie kann hinsichtlich der Ausführungsgeschwindigkeit die Schnittstelle zwischen Speicher (z.B. RAM, Caches) und der HW Implementierung implementiert werden?
4. Wie verhält sich eine HW Implementierung in einem Multi-User-Context in dem viele unterschiedliche Funktionen gleichzeitig ausgeführt werden müssen? Sind hier ähnliche Mechanismen wie sie heute z.B. in Standardprozessoren und Betriebssystemen benutzt werden notwendig?
5. Welche Energieeinsparungen sind im Vergleich zu einer reinen Software Implementierung zu erwarten?
6. Welche Probleme konnten im Rahmen des Auftrags nicht gelöst werden und was sind mögliche Folgeaktivitäten für die weitere Entwicklung von **paceval**?

² Ein Field Programmable Gate Array (FPGA) ist ein integrierter Schaltkreis (IC) der Digitaltechnik, in welchen eine logische Schaltung geladen werden kann. FPGAs werden in allen Bereichen der Digitaltechnik eingesetzt, vor allem aber dort, wo es auf schnelle Signalverarbeitung und flexible Änderung der Schaltung ankommt, um beispielsweise nachträgliche Verbesserungen an den implementierten Funktionen vornehmen zu können, ohne dabei direkt die Hardware ändern zu müssen. Ein großes Anwendungsgebiet ist die Erstellung von Prototypen in der ASIC-Entwicklung zum vorherigen Test.

³ Eine anwendungsspezifische integrierte Schaltung (englisch Application-Specific Integrated Circuit, ASIC oder auch Custom Chip) ist eine elektronische Schaltung, die als integrierter Schaltkreis realisiert wurde. Die Funktion eines ASICs ist damit nicht mehr veränderbar, die Herstellungskosten sind dafür geringer bei hohen Einmalkosten.

1.2 Meilensteine

Dieser Zwischenbericht umfasst die folgenden Meilensteine:

Meilenstein 0 – Projektstart

Arbeitspaket 0 (AP0): Kauf und Inbetriebnahme der für die Validierung benötigten Hardware (FPGAs + Test Server)

Notwendig ist hierfür auch eine Vor-Ort Unterstützung von OFFIS (Institut für Informatik) in Oldenburg.

Meilenstein 1 – Portierung von *paceval*. auf FPGA

Arbeitspaket 1 (AP1): Lauffähigkeit von *paceval*. auf den ARM Prozessoren auf dem FPGA
Parallelisierung sicherstellen und erste Messungen der Geschwindigkeit und Stromaufnahme bei Durchführung des MNIST Benchmark.

Meilenstein 2 – Grundlegende mathematische Funktionen beschleunigen

Arbeitspaket 2 (AP2): Klärung Fragen 1. und 2., s.o. „Unteraufgaben/-fragen“
Umsetzung von mathematischen Operationen und Teilfunktionen in Hardware auf dem FPGA (+/- /*/:/exp).
Parallelisierung sicherstellen und zweite Messungen der Geschwindigkeit und Stromaufnahme bei Durchführung des MNIST Benchmark.

Meilenstein 3 – Optimierung der Funktionalität

Arbeitspaket 3 (AP3): Klärung Fragen 3. und 4., s.o. „Unteraufgaben/-fragen“
Parallelisierung verbessern, Schnittstellen zum Hostsystem festlegen. System möglichst verallgemeinern, um die Benutzerfreundlichkeit für andere herzustellen.
Abschätzung wie andere neuronale Netze mit der Lösung arbeiten.

Meilenstein 4 – Dokumentation und Beantwortung aller obenstehenden Fragen zum Validierungsauftrag

Arbeitspaket 4 (AP4): Klärung Fragen 5. und 6., s.o. „Unteraufgaben/-fragen“
Finale Messungen der Geschwindigkeit und Stromaufnahme bei Durchführung des MNIST Benchmark.

2. Zusammenfassung der Ergebnisse

Mit neuen und vorhandenen Technologien wurde ein erstes Referenzdesign entwickelt, das **paceval** in Hardware von den geforderten Funktionalitäten abbildet. Dieses Referenzdesign basiert in Hardware auf einem programmierbaren System-on-a-Chip (SoC) der Zynq-7000 Plattform von AMD Xilinx mit ARM32-Prozessoren. Die mögliche Zertifizierung und die daran sich anschließende regulatorische Zulassung eines finalen Produktes auf dieser Hardware sollte unproblematisch sein, da der SoC bereits die harten Anforderungen der Automobilindustrie AEC-Q100 erfüllt.

Das grundlegende Betriebssystem auf dem SoC ist PetaLinux als sogenannte „embedded Linux-Distribution“, wie sie von Xilinx unterstützt wird. Als Anwendung innerhalb des PetaLinux ist der Dienst **paceval**-service eingerichtet, der auf HTTP-Anfragen⁴ an Port 8080 reagiert. Die REST-API⁵, die dieser Dienst anbietet, ermöglicht es, geschlossene mathematische Funktionen⁶ beliebiger Länge und mit beliebiger Anzahl an Variablen effektiv auszuwerten. Damit lassen sich alle finanztechnischen, stochastischen, ingenieur- und naturwissenschaftlichen Funktionen sowie insbesondere alle Modelle für maschinelles Lernen darstellen. Einfacherweise kann darüber hinaus die übliche mathematische Standardschreibweise verwendet werden. Da die mathematischen Funktionen und insbesondere die bei der künstlichen Intelligenz (KI) zur Ausführung verwendeten neuronalen Netze komplett in textueller Form vorliegen, existiert somit ein Pfad zur Zertifizierung und anschließenden regulatorischen Zulassung der Software. Dies in Verbindung mit der Zulassung der Hardware ergibt ein zugelassenes Gesamtsystem, was so per se stand heute nicht möglich ist bei Produkten, die auf neuronalen Netzen basieren⁷.

Das gewählte Interface für die Kommunikation mit dem Referenzdesign ist Ethernet. Das SoC bietet in seiner Gesamtheit für die Kommunikation PCI Express, USB, Ethernet, SPI, SD/SDIO, I2C, CAN, UART und GPIO an. Die gewählte Softwarelösung ließe sich auch auf ein weitaus leistungsstärkeres Multiprocessor-System-on-a-Chip (MPSoC) mit ARM64 leicht übertragen, die es in geeigneten Formfaktoren zu guten Anschaffungspreisen⁸ gibt (Abbildung 1). Dadurch ermöglichen sich nicht nur kleine, energiesparende IoT- und IIoT-Varianten, sondern auch große



Abbildung 1: MPSoC Modul mit AMD Zynq UltraScale+ ZU4EG-1E (5.2 x 7.6 cm)

⁴ Das Hypertext Transfer Protocol (HTTP) ist ein 1991 eingeführtes zustandsloses Protokoll zur Übertragung von Daten auf der Anwendungsschicht über ein Rechnernetz. Es wird hauptsächlich eingesetzt, um Webseiten (Hypertext-Dokumente) aus dem World Wide Web (WWW) in einen Webbrowser zu laden. Es ist jedoch nicht prinzipiell darauf beschränkt und auch als allgemeines Dateiübertragungsprotokoll sehr verbreitet.

⁵ Eine REST-API ist eine Programmierschnittstelle, die HTTP-Anfragen nutzt, um mittels PUT, GET, POST und DELETE auf Daten zuzugreifen. REST ist eine Programmierparadigma des Internets.

⁶ In der Mathematik ist ein geschlossener Ausdruck ein mathematischer Ausdruck, der eine endliche Anzahl von Standardoperationen verwendet.

⁷ Das Patent US20210279567 METHOD AND SYSTEM FOR ADJUSTING A MACHINE LEARNING OUTPUT von Koenning et al. beschreibt ein Verfahren, das als Grundlage für die Zertifizierung von neuronalen Netzen gegeben als textuelle mathematische Ausdrücke beschreibt, s.

https://patentscope.wipo.int/search/de/detail.jsf?docId=US335379602&_cid=P12-LLWDVV-21587-1

⁸ MPSoC Modul mit AMD Zynq UltraScale+ ZU4EG-1E (5.2 x 7.6 cm), s. <https://shop.trenz-electronic.de/en/TE0817-01-4BE21-A-MPSoC-Module-with-AMD-Zynq-UltraScale-ZU4EG-1E-4-GBByte-DDR4-5.2-x-7.6-cm?c=490>

Varianten für Rechenzentren und insbesondere die bekannten Hyperscaler wie die Amazon Web Services (AWS), Microsoft Azure und die Google Cloud Platform.

Zudem ist es möglich, die Bibliothek von **paceval**. in einer Variante für RISC-V Prozessoren mit unlimitierter Unterstützung von Multicores zu erstellen. Die RISC-V Architektur bietet Firmen und natürlich auch Staaten den Vorteil, ohne Lizenzkosten und externe Abhängigkeiten eigenständig Hardwaresysteme zu entwickeln. Diese Hardwaresysteme würden skalieren von einem Prozessorsystem auf einem FPGA oder ASIC bis hin zu den Rechenchips Rhea⁹ oder dem Rechenbeschleuniger Occamy¹⁰ mit jeweils mehreren Hundert RISC-V Rechenkernen.

Das OFFIS als Institut für Informatik in Oldenburg wurde beauftragt, die Konfiguration des FPGA eines geeigneten AMD Xilinx SoC mit einer Hardwarebeschreibungssprache umzusetzen. Die Implementierung des Coprozessors als ASIC kann die Geschwindigkeit für die Ausführung von KI nochmals um ein Vielfaches erhöhen. Die zu erreichende Effizienz eines ASICs gegenüber einer GPU wäre demnach enorm gross.

Insgesamt zeigt sich nach den Untersuchungen von OFFIS, dass dieses erste Referenzdesign ein hohes Energieeinsparpotential bei der geforderten Ausführung von KI aufweist. Bei der Ausführung eines neuronalen Netzwerks ergibt sich folgender Vergleich:

	Standard 3.0 GHz CPU + NVIDIA GeForce GTX TITAN X	paceval . 4.24 auf CPU APPLE MAC STUDIO M1 20 Cores	paceval . 4.24 auf FPGA PS Zynq Ultrascale+ XCZU7EV 4 Cores	paceval . in Hardware/ FPGA PL Zynq Ultrascale+ XCZU7EV 96 FPU*s	paceval . in Hardware/ASIC 256 FPU*s (Werte abgeschätzt)
Stromaufnahme	400 Watt (CPU+GPU) ¹¹	68 Watt	3,4 Watt	3,4 Watt	< 3,4 Watt
Bearbeitungszeit MNIST LeNet	< 2 ms	5 ms	114 ms	< 75 ms	< 15 ms
Anschaffungskosten	< \$ 2.000	\$ 3.300	\$ 850	\$ 850	\$ 700 ¹² + Gewinnmarge
Laufende Energiekosten ¹³	\$ 700/Jahr	\$ 120/Jahr	\$5/Jahr	\$5/Jahr	< \$5/Jahr
CO2 Emission	560 kg/Jahr	95 kg/Jahr	0,9 kg/Jahr	0,9 kg/Jahr	< 0,9 kg/Jahr
Kühlung	aktiv	aktiv	passiv	passiv	passiv
Gehäusegröße	> 8.000 cm ³	3.700 cm ³	< 50 cm ³	< 50 cm ³	< 35 cm ³

*Werte für FPGA PL interpoliert aus Anhang C: „OFFIS-Machbarkeitsstudie für die Umsetzung der paceval-Software in FPGA-Hardware“

Mit maximal 3,4 Watt Leistungsaufnahme sind bei der durchgehenden Ausführung von KI keinerlei zusätzliche Maßnahmen gegen eine Hitzeentwicklung notwendig.

⁹ Allzweckprozessoren Rhea (mindestens 73 Cores) der European Processor Initiative (EPI), <https://www.european-processor-initiative.eu/>

¹⁰ Europäischer Open-Source-Beschleuniger Occamy (432 Cores) mit RISC-V, <https://pulp-platform.org/occamy/>

¹¹ Quelle: <https://www.pcgameshardware.de/Titan-X-Grafikkarte-265165/Tests/Test-Benchmark-Tuning-Overclocking-vs-1080-1206879/>

¹² Die Herstellungskosten für den ASIC sind nach bestem Wissen geschätzt.

¹³ Die dargestellten Energiekosten und CO₂ Emissionen sind abhängig vom Ort, dem Anbieter und dem gewählten Energiemix.

Es wurden alle Meilensteine erreicht und der hier vorliegende Abschlussbericht zusätzlich mit den Anhängen

- Anhang A „paceval-Übergabe Quellcode an OFFIS“
- Anhang B „Expertenpapier-Umwandlung von neuronalen Netzen in geschlossene Ausdrücke“
- Anhang C „OFFIS-Machbarkeitsstudie für die Umsetzung der paceval-Software in FPGA-Hardware“

erstattet detailliert Bericht über die Arbeiten und das Potential des Referenzdesigns.

3. Die eingesetzte Technologie

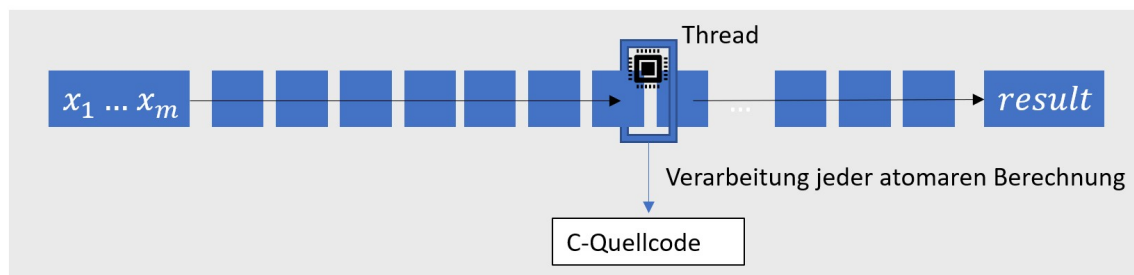
Damit möglichst effektiv, d.h. auf diesen Validierungsauftrag bezogen, vor allem energiesparend und mit einer hohen Geschwindigkeit komplexe mathematische Berechnungen durchgeführt werden, war die Auswahl der richtigen Basistechnologie notwendig.

Wir gehen in diesem Validierungsauftrag von der mathematischen Engine¹⁴ *paceval*. in der Version 4.04 aus.¹⁵ Diese liegt als systemunabhängige Softwarebibliothek vor, hat bereits einen kleinen binären Fußabdruck, eine beeindruckende Leistung und gleichzeitig einen geringen Stromverbrauch. Die Softwarebibliothek ist in C und C++ geschrieben, und das ist schon jetzt eine der nachhaltigsten Kombinationen von Programmiersprachen (Abbildung 2). Diesen Status möchten wir weiter durch Hardware verbessern.

Wie in der Patentanmeldung¹⁶ zu *paceval*. beschrieben, generiert und verarbeitet diese mathematische Engine intern eine verkettete Liste atomarer Berechnungen, welche die mathematische Funktion des Benutzers darstellen. Die Verarbeitung der verketteten Liste bei einer konkreten Durchführung einer Berechnung mit Werten für die Variablen erfolgt in einer einzigen C-Funktion, die von jedem Thread des zugrundeliegenden Systems aufgerufen wird.

Total					
Energy		Time		Mb	
(c) C	1.00	(c) C	1.00	(c) Pascal	1.00
(c) Rust	1.03	(c) Rust	1.04	(c) Go	1.05
(c) C++	1.34	(c) C++	1.56	(c) C	1.17
(c) Ada	1.70	(c) Ada	1.85	(c) Fortran	1.24
(v) Java	1.98	(v) Java	1.89	(c) C++	1.34
(c) Pascal	2.14	(c) Chapel	2.14	(c) Ada	1.47
(c) Chapel	2.18	(c) Go	2.83	(c) Rust	1.54
(v) Lisp	2.27	(c) Pascal	3.02	(v) Lisp	1.92
(c) Ocaml	2.40	(c) Ocaml	3.09	(c) Haskell	2.45
(c) Fortran	2.52	(c) C*	3.14	(c) PHP	2.57
(c) Swift	2.79	(v) Lisp	3.40	(c) Swift	2.71
(c) Haskell	3.10	(c) Haskell	3.55	(c) Python	2.80
(v) C#	3.14	(c) Swift	4.20	(c) Ocaml	2.82
(c) Go	3.23	(c) Fortran	4.20	(v) C*	2.85
(c) Dart	3.83	(c) F#	6.30	(c) Hack	3.34
(v) F#	4.13	(c) JavaScript	6.52	(v) Racket	3.52
(c) JavaScript	4.45	(c) Dart	6.67	(c) Ruby	3.97
(v) Racket	7.91	(v) Racket	11.27	(c) Chapel	4.00
(c) TypeScript	21.50	(c) Hack	26.99	(v) F#	4.25
(c) Hack	24.02	(c) PHP	27.64	(c) JavaScript	4.59
(c) PHP	29.30	(v) Erlang	36.71	(c) TypeScript	4.69
(v) Erlang	42.23	(c) Ruby	43.44	(v) Java	6.01
(c) Lua	45.98	(c) TypeScript	46.20	(c) Perl	6.62
(c) Ruby	46.54	(c) Ruby	59.34	(c) Lua	6.72
(c) Ruby	69.91	(c) Perl	65.79	(v) Erlang	7.20
(c) Python	75.88	(c) Python	71.90	(c) Dart	8.64
(c) Perl	79.58	(c) Lua	82.91	(c) Ruby	19.84

Abbildung 2: Energieeffizienz der Programmiersprachen



Der C-Quellcode führt diese Verarbeitung durch:

1. **FETCH**
Holen des Operators und der Operanden (z. B. „Addition von 2 und 3“)
2. **DECODE** und **DECIDE**
Verwende das zwischengespeicherte Ergebnis aus dem Cache oder rufe den nächsten Schritt EXECUTE auf
3. **EXECUTE**
Führe den Operator mit den Operanden aus (z. B. rufe die C-Funktion auf, um 2 und 3 zu addieren und als Ergebnis 5 zu erhalten)
4. **WRITE BACK**
Speichere das Ergebnis der atomaren Berechnung in den Cache zwischen (dies schließt untere und obere Intervallgrenzen oder Fehler ein)

¹⁴ Eine mathematische Engine ist ein Teil eines Computerprogramms oder ein Teil der Computerhardware, die als Engine bezeichnet wird, verantwortlich für die effiziente Verarbeitung von mathematischen Modellen.

¹⁵ *paceval*. Homepage : <https://paceval.com>

¹⁶ EP3385834 HARDWARE DRIVER FOR EFFICIENT ARITHMETIC von Koenning et al., s. https://patentscope.wipo.int/search/de/detail.jsf?docId=EP231575389&_cid=P12-LLWE94-27922-1

Dies entspricht dem Standardzyklus, der von allen Arten von Prozessoren verwendet wird. Diesen Standardzyklus wollen wir in Hardware mit einem Field Programmable Gate Array (FPGA) umsetzen, um die Effizienz weiter zu erhöhen.

Systeme auf Basis von FPGAs bieten viele Vorteile gegenüber herkömmlichen Implementierungen. Die Anwendungslogik in einem FPGA wird in Hardware Schaltkreisen implementiert, anstatt auf einem Betriebssystem, Treibern und anderer Anwendungssoftware zu laufen. Ein FPGA kann autonom ohne Störung durch andere Logikblöcke funktionieren (Abbildung 3). FPGAs bieten die Möglichkeit, Systeme zu entwickeln, die exakt auf die vorgesehene Aufgabe zugeschnitten sind und daher äußerst effizient arbeiten. Durch die Implementierung eines Algorithmus als FPGA kann der Stromverbrauch deutlich reduziert werden.

Komplexe Aufgaben werden oft durch Softwareimplementierungen mit schnellen Prozessoren gelöst. FPGAs bieten hier eine hervorragende Alternative, die durch Parallelisierung und Anpassung an die Anwendung einen deutlichen Geschwindigkeitsvorteil gegenüber prozessorbasierten Lösungen bietet.

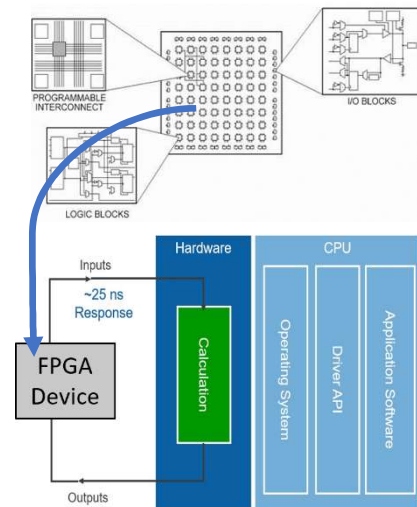


Abbildung 3: Funktionsweise des FPGA

3.1 System-on-a-Chip (SoC) mit ARM-Prozessoren und FPGA

Die gewählte FPGA Technologie ZC706 gehört zur Familie der programmierbaren System-on-a-Chip (SoC) Zynq-7000¹⁷ von AMD Xilinx (Abbildung 4). Diese kombiniert die Programmierbarkeit eines ARM-basierten Prozessors in Software mit der Programmierbarkeit eines FPGA in Hardware. Dadurch wird eine signifikante Hardwarebeschleunigung ermöglicht. Die für diesen Validierungsauftrag gewählten Entwicklungsboards¹⁸ sind mit Dual-Core-ARM-Cortex-A9-Prozessoren¹⁹ ausgestattet, die mit 28-nm-Artix-7- oder Kintex-7-basierter programmierbarer Logik für hervorragende Leistung pro Watt und maximale Designflexibilität ausgestattet sind. Mit bis zu 6,6 Millionen Logikzellen und angeboten mit Transceivern von 6,25 Gb/s bis 12,5 Gb/s ermöglichen die Zynq-7000-Bausteine hochgradig differenzierte Lösungen für eine

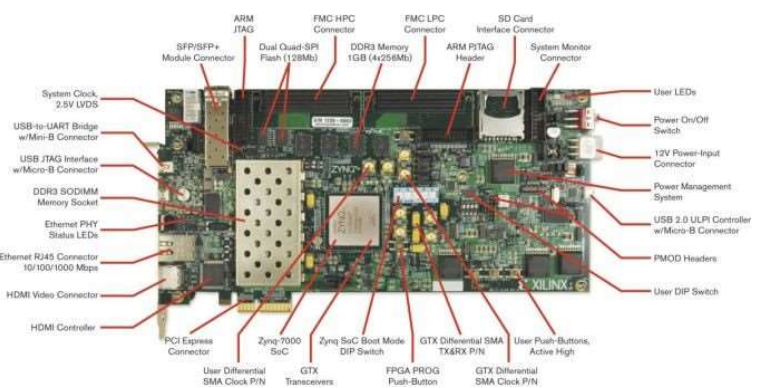


Abbildung 4: Das ZC706 Evaluation Board

¹⁷ Zynq-7000 Product: <https://www.xilinx.com/content/dam/xilinx/support/documents/product-briefs/zynq-7000-product-brief.pdf>

¹⁸ ZC706 Evaluation Board User Guide, s. <https://docs.xilinx.com/v/u/en-US/ug954-zc706-eval-board-xc7z045-ap-soc>

¹⁹ ARM-Cortex-A9, s. https://de.wikipedia.org/wiki/Arm_Cortex-A#Arm_Cortex-A9

breite Palette von eingebetteten Anwendungen, darunter Fahrerassistenzsysteme mit mehreren Kameras.

Das Evaluierungsboard ZC706 ist mit dem Zynq-7000 XC7Z045 SoC bestückt. Dieses programmierbare System-on-a-Chip besteht aus einem Verarbeitungssystem (PS) und einer Programmierbaren Logik (PL) auf einem einzigen integrierten Schaltkreis (Die). Das Verarbeitungssystem integriert zwei ARM Cortex-A9 MPCore-Anwendungsprozessoren, AMBA-Verbindung²⁰, interne Speicher, externe Speicherschnittstellen und Peripheriegeräte, einschließlich USB, Ethernet, SPI²¹, I2C²², CAN²³, UART²⁴, GPIO²⁵ und SD-Karten Interface. Das Verarbeitungssystem läuft unabhängig von der programmierbaren Logik und bootet automatisch beim Einschalten oder Zurücksetzen.

Der SoC und insbesondere das Evaluierungsboard ZC706 bieten alle Eigenschaften, die wir benötigten, um die bestmögliche benutzerfreundliche Produktlösung zu entwickeln. Für den hier beschriebenen Validierungsauftrag waren insbesondere diese Funktionalitäten wichtig (Abbildung 5):

- 1 GB Speicher DDR3 (SO-DIMM²⁶) für Programmcode und -Daten im Verarbeitungssystem (PS)
- 1 GB Speicher für Programmierbare logische Schaltung²⁷ (PL)
- Ethernet Physical-Layer-Transceiver für Kommunikation mit bis zu 1000 Mb/s
- Unterstützung von Echtzeitbetriebssystemen²⁸ insbesondere Linux
- PCI Express-Schnittstelle für direkten Einbau in Server (als Alternative zum Ethernet)
- andere Schnittstellen wie CAN für In-Car Lösungen bei Automotive (als Alternative zum Ethernet)
- SD-Karte (für schnelle Prototypen und Tests)

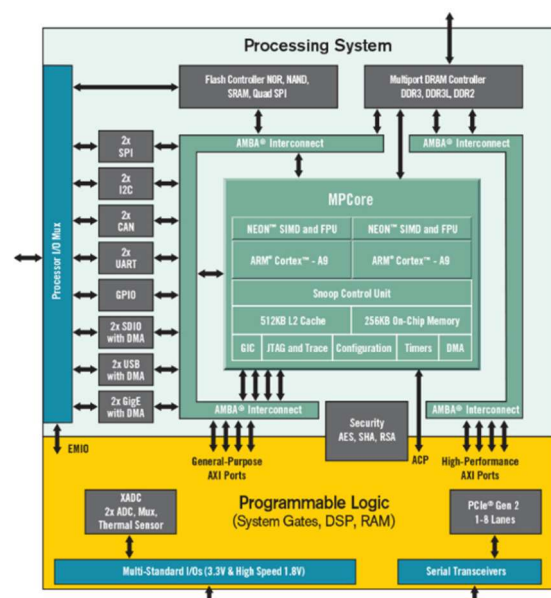


Abbildung 5: High-level Zynq-7000 Blockdiagramm

²⁰ Advanced Microcontroller Bus Architecture (AMBA) ist ein frei verfügbarer, offener Standard für die Verbindung und Verwaltung von Funktionsblöcken in einem System-on-Chip (SoC). Die Architektur erleichtert die Entwicklung von Multiprozessor-Designs mit einer großen Anzahl von Controllern und Peripheriegeräten.

²¹ Serial Peripheral Interface (SPI) ist ein synchroner, serieller Bus mit dem Daten kontinuierlich und ohne Unterbrechung gestreamt werden können.

²² I2C (Inter-Integrated Circuit) ist ein On-Board-Kommunikationsprotokoll, das ideal für kurze Distanzen und geringe Bandbreite ist.

²³ Ein Controller Area Network (CAN-Bus) ist ein robuster Busstandard für Fahrzeuge, der entwickelt wurde, um es Mikrocontrollern und Geräten zu ermöglichen, ohne einen Host-Computer mit den Anwendungen der anderen in einem Fahrzeug zu kommunizieren.

²⁴ Universal Asynchronous Receiver Transmitter (UART) ist die gängige serielle Schnittstelle bei Mikrocontrollern.

²⁵ Ein General Purpose Input/Output (GPIO) ist ein allgemeiner digitaler Kontaktstift an einem integrierten Schaltkreis, dessen Verhalten unabhängig ob als Eingabe- oder Ausgabekontakt durch logische Programmierung frei bestimmbar ist.

²⁶ Small Outline Dual Inline Memory Module (SO-DIMM), s.

https://de.wikipedia.org/wiki/Small_Outline_Dual_Inline_Memory_Module

²⁷ Programmable Logic Device (PLD), s. https://de.wikipedia.org/wiki/Programmierbare_logische_Schaltung

²⁸ Real-time operating system (RTOS), s. <https://de.wikipedia.org/wiki/Echtzeitbetriebssystem>

Somit bildet das ZC706 in Gänze ein hervorragendes Referenzdesign für diesen Validierungsauftrag. Von diesem Referenzdesign ausgehend können vielfältige Spezialisierungen des darauf basierenden ersten Produktes erfolgen. Von performanten, aber energieeffizienten, skalierbaren Lösungen für Hyperscaler²⁹ oder Endanwender bis zu besonders stromsparenden, eingebetteten Lösungen für die Automobil- und Sensorik Industrie ist alles möglich (Abbildung 6).

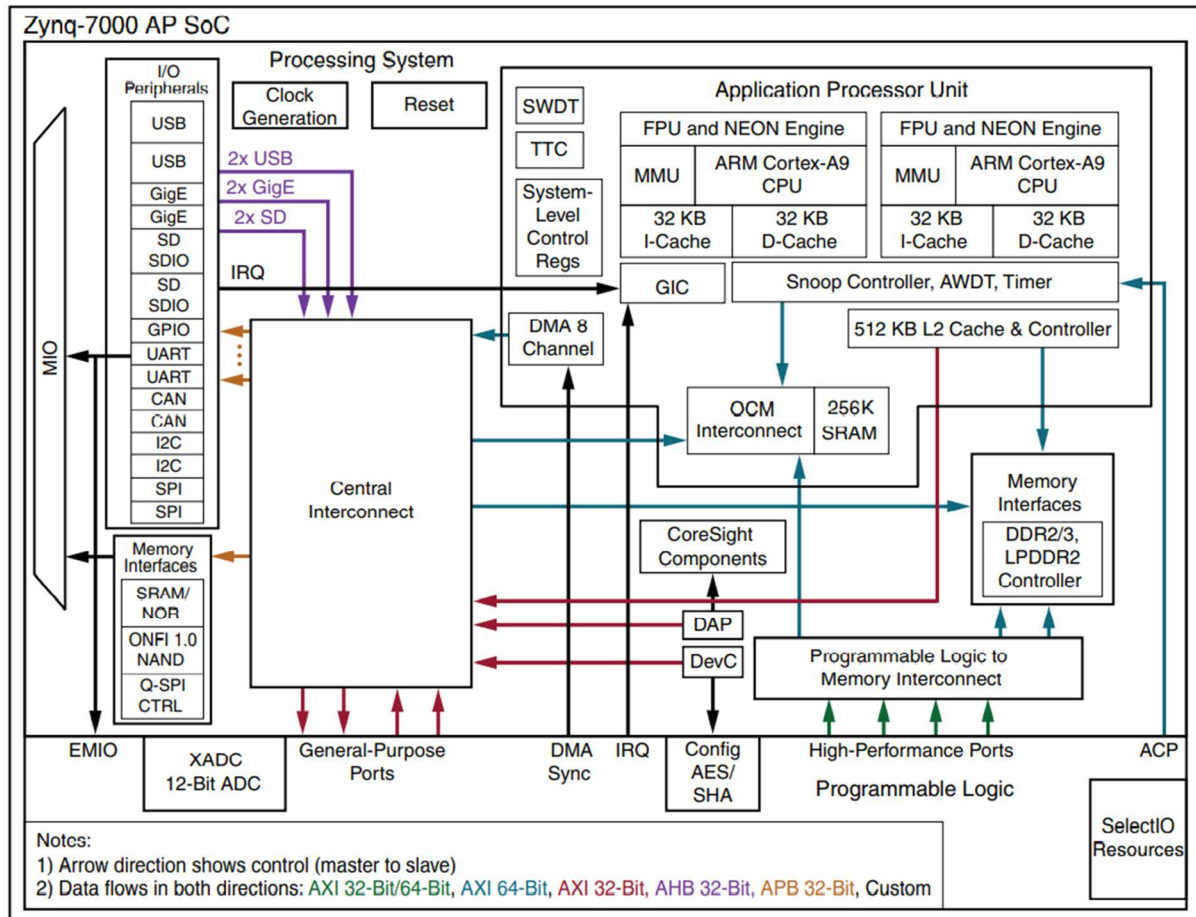


Abbildung 6: Zynq-7000 XC7Z045 SoC Blockdiagramm

3.2 PetaLinux und die gewählten Softwarekomponenten auf dem SoC

Als Betriebssystem für das Verarbeitungssystem (PS) haben wir PetaLinux gewählt. PetaLinux als sogenannte „embedded Linux-Distribution“, wie sie von Xilinx verwendet wird, basiert auf Yocto³⁰ und einem gepflegten Kernel. PetaLinux verfügt über das Hardware Software Interface (HSI) von Vivado³¹ und umfasst Werkzeuge für die Erstellung eines Boot-Images. Dies erlaubt

²⁹ Hyperscaler sind Computing-Netzwerke zur Erzielung massiver Skalierungen im Bereich Cloud Computing und Big Data.

³⁰ Das Yocto-Projekt ist ein kollaboratives Open-Source-Projekt der Linux Foundation, dessen Ziel es ist, Tools und Prozesse zu entwickeln, die die Erstellung von Linux-Distributionen für eingebettete und IoT-Software ermöglichen, die von der zugrunde liegenden Architektur der eingebetteten Hardware unabhängig sind.

³¹ Vivado ist eine von Xilinx produzierte Softwareumgebung für die Synthese und Analyse von Hardware-Designs mit entsprechenden Beschreibungssprachen (Hardware description languages).

ressourcensparende Konfigurationen, die optimiert sind für Systeme, die auf AMD Xilinx SoCs basieren.

Das von uns mit den PetaLinux Werkzeugen erzeugte Image wird auf die SD-Karte kopiert. Das Betriebssystem wird beim ersten Starten des ZC706 Entwicklungsbords konfiguriert. Es ist hierfür Zugang zum Ethernet bzw. Internet am Entwicklungsbord per einfachem LAN-Kabel nötig. Über ein Bootscript wird zuerst eine 1 GB große Auslagerungsdatei erstellt (s.u. „swap“). Damit wächst der für das PetaLinux verfügbare Speicher auf 2 GB, was nochmals größere mathematische Funktionen und somit größere neuronale Netze oder allgemein umfangreichere KI³² ermöglicht.

Dann wird die Systemzeit des PetaLinux mit der Internetzeit abgeglichen (s.u. ntpd), was vor allem für das Logging nötig ist. Schließlich wird ein Verzeichnis für den externen Dateisystemzugriff eingerichtet (s.u. „Samba“). In das Verzeichnis /home/paceval/sambashare der SD-Karte des PetaLinux auf dem ZC706 Entwicklerboards wird der **paceval**-service mit der 32-bit **paceval**-Bibliothek für ARM-Prozessoren kopiert. Samba dient hier für den unkomplizierten Austausch von Scripten, Bibliotheken, Dateien und Daten zum Test. Der **paceval**-service basiert auf Node.js und ist ein HTTP-Service, der an Port 8080 auf Eingaben wartet.

Weitere benötigte Komponenten sind npm und PM2. Mit npm wird die korrekte PM2 Version beim ersten Starten installiert. Der **paceval**-service wird dann über PM2 ebenfalls in dem Bootscript gestartet und überwacht.

Als zusätzliche Softwarekomponenten wird mit Lighthttpd und PHP ein Webinterface erzeugt, das an Port 80 auf Eingaben wartet. Dieses kann dann ganz einfach vom Benutzer aufgerufen werden, bspw. über die folgende URL:

<http://paceval-engine>

„paceval-engine“ ist der Hostname des PetaLinux auf dem ZC706 Entwicklungsbord. Aktuell werden hier grundlegende Funktionen des Systems mittels phpinfo()³³ angezeigt (Abbildung 7). Dies kann natürlich einfach verändert werden, um eine umfassende Benutzeroberfläche zu erzeugen. Eine Benutzeroberfläche ist heutzutage häufig der Standard für Systemkonfigurationen und auch um Loggingausgaben anzuzeigen. Ein gutes Beispiel für eine solche Benutzerschnittstelle wäre die Fritzbox, die vielen aus dem eigenen Haushalt bekannt sein dürfte (Abbildung 8).

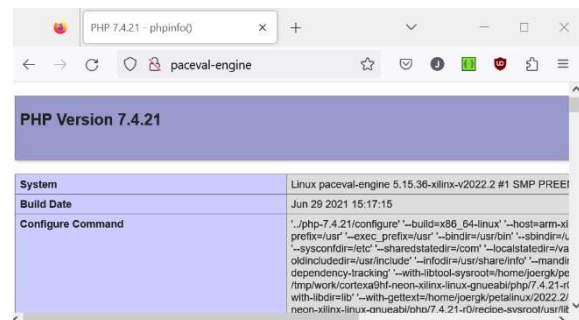


Abbildung 7: Ausgabe von phpinfo über das ZC706 Entwicklungsbord mittels Lighthttpd und PHP

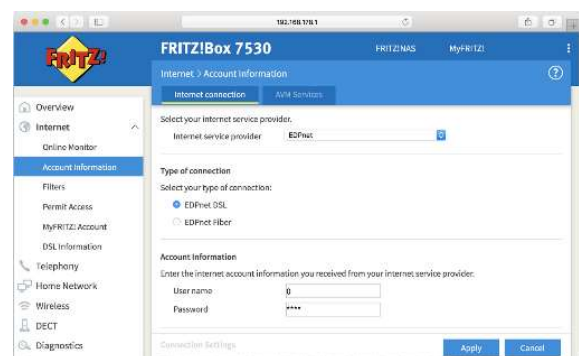


Abbildung 8: Öffnen der Fritz!Box Benutzeroberfläche einfach im Internetbrowser über die Adresse <http://fritz.box>

Nach diesem ersten Starten ist das System konfiguriert. Nachfolgende Neustarts synchronisieren lediglich die Systemzeit des PetaLinux mit der Internetzeit. Das so vorkonfigurierte System kann nach

³² Anmerkung: Wir gehen davon aus, dass sehr große neuronale Netze mehr als 1 GB an Speicher für das Processing System benötigen.

³³ Der Befehl phpinfo() von PHP zeigt eine große Anzahl von Informationen über den aktuellen Zustand von PHP an, s. <https://www.php.net/manual/en/function.phpinfo.php>

Belieben den Ansprüchen des späteren Produktherstellers angepasst werden.

Softwarekomponente / Version	Beschreibung	Zweck
paceval-service / 4.04	paceval -service ist eine mathematische Engine as a Service und bietet einen leistungsstarken und schnellen Remote-Mathematik-Coprozessor als HTTP-Service für IoT-Lösungen, der an Port 8080 auf Eingaben wartet.	Dies ist der eigentliche Service, der von dem PetaLinux auf dem ZC706 Entwicklerboard zur Verfügung gestellt wird.
node / 14.17.1	Node.js ist eine plattformübergreifende Open-Source-JavaScript-Laufzeitumgebung, die JavaScript-Code außerhalb eines Webbrowsers ausführen kann.	Node.js wird für die Laufzeit des paceval -service benötigt.
npm / 6.14.13	npm ist ein Paketmanager für die JavaScript-Laufzeitumgebung Node.js.	npm wird für die Installation von PM2 und potenziell anderen Paketen benötigt.
PM2 / 5.2.2	PM2 ist ein Prozessmanager für die JavaScript-Laufzeit Node.js.	PM2 überwacht die Laufzeit des paceval -service.
Lighttpd / 1.4.59	Lighttpd ist ein freier Webserver, der durch Module erweitert werden kann.	Lighttpd bietet im Zusammenspiel mit PHP die Möglichkeit, eine dynamische Benutzeroberfläche zu gestalten.
PHP / 7.4.21	PHP ist eine Skriptsprache mit einer an C und Perl angelehnten Syntax, die hauptsächlich zur Erstellung dynamischer Webseiten oder Webanwendungen verwendet wird.	s. Lighttpd
Samba / 4.14.8	Samba bietet Dateidienste für verschiedene Microsoft Windows-Clients und kann in eine Microsoft Windows Server-Domäne integriert werden.	Ermöglicht den Dateisystemzugriff auf /home/paceval/smbshare der SD Karte des PetaLinux auf dem ZC706 Entwicklerboard.
ntpd / 4.2.8	ntpd mit ntpd stellt das lokale Datum und die Uhrzeit ein, indem es einen Network Time Protocol (NTP)-Server abfragt.	Synchronisiert das Datum und die Zeit des ZC706 Entwicklerboards mit pool.ntp.org.
swap / util-linux 2.37.2	Swap Space ist ein Speicher-virtualisierungskonzept. Es ist eine Strategie, um Speichermangel zu mindern, wenn einem System der Speicher ausgeht. Um es einzusetzen, verschiebt oder lagert Linux Blöcke nicht kritischen Speichers auf die Festplatte aus und lagert sie bei Bedarf wieder ein.	Erweitert den Speicher des Verarbeitungssystems (PS) auf insgesamt 2 GB (bei 1GB Auslagerungsdatei) für größere Funktionen

3.3 Berechnungen mit der mathematischen Engine as a Service auf dem SoC

Im Wesentlichen sind nur zwei Schritte notwendig, um Berechnungen durchzuführen. Es wird zuerst ein Berechnungsobjekt erzeugt (Schritt „Create“), indem der Benutzer eine Funktion und die Menge der Variablenbezeichner übergibt. Dann können mit dem einmal erzeugten Objekt immer wieder konkrete Berechnungen durchgeführt werden (Schritt „Calculate“), indem der Benutzer die Werte für die Variablen übergibt (Abbildung 9).

Im Schritt „Create“ wird mit dem Berechnungsobjekt eine Liste als Berechnungsvorschrift erzeugt und auf maximale Parallelisierbarkeit ausgerichtet. Der Benutzer erhält eine eindeutige ID für das erzeugte Objekt zurück. Es kann eine beliebige Anzahl an Berechnungsobjekten erzeugt werden. Zudem gibt es keine Begrenzung der Funktionslänge und der Anzahl der Variablen.

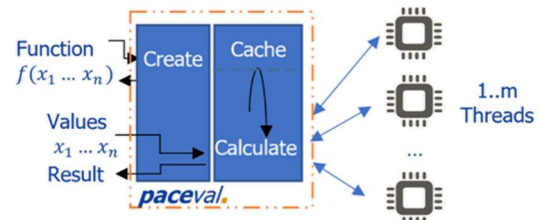


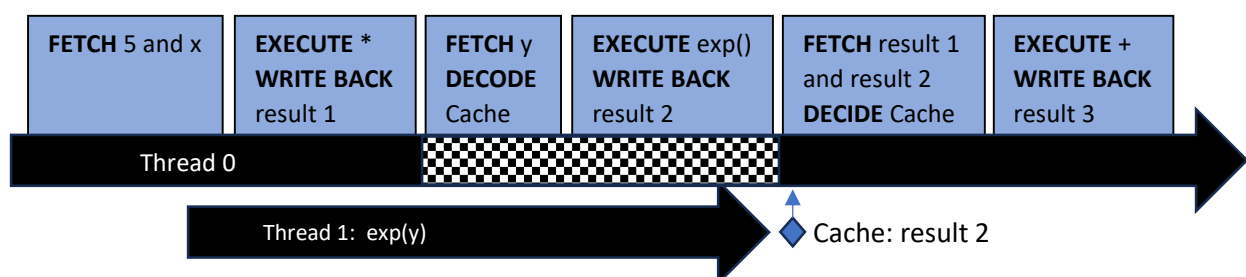
Abbildung 9: Verarbeitung der Berechnungen mit der mathematischen Engine

Im Schritt „Calculate“ wird die Berechnung mit den Werten für die Variablen durchgeführt. Dabei wird die Liste der Berechnungsvorschrift als Teilsequenzen auf alle verfügbaren Prozessoren (hier „Threads“) des Systems verteilt, um eine maximale Geschwindigkeit zu erzielen. Zusätzlich werden einmal durchgeführte und aufwendige Berechnungen von Teilsequenzen zwischengespeichert (hier „Cache“), damit diese gegebenenfalls nicht erneut berechnet werden müssen.

Nehmen wir als Beispiel die folgende Funktion und die anschließende Berechnung mit den angegebenen Variablenwerten:

$$f(x, y) = 5 * x + \exp(y) \quad \text{mit } x=2.2 \text{ und } y=3.3$$

Der Schritt „Create“, bzw. der API Aufruf /CreateComputation wird dann (vereinfacht) diese verkettete Liste mit der Bibliothek von **paceval**. im Speicher erzeugen und auch Marker für mögliche Parallelisierung mit Threads setzen:



Der Schritt „Calculate“ wird dann die Berechnung mit insgesamt 2 Threads durchführen. Zuerst wird Thread 0 gestartet, der die Aufgabe hat, die gesamte Berechnung durchzuführen, d.h. Thread 0 durchläuft die gesamte verkettete Liste. Dann wird Thread 1 gestartet, der $\exp(y)$ für $y=3.3$ parallel zu Thread 0 durchführt. Sollte Thread 1 schneller sein, so wird das Ergebnis der parallelen Berechnung $\exp(3.3)$ im Thread 0 für die finale Addition übernommen und die gesamte Berechnung, die Thread 1 abgedeckt hat wird übersprungen.

Dies ist nur ein einfaches Beispiel. Bei längeren Funktionen wird die **paceval**. Bibliothek die parallelen Berechnungen so auf alle verfügbaren Prozessoren aufteilen, dass die Threads möglichst einen idealen längeren Bereich der Nebenberechnungen abdecken.

Um nun als Beispiel eine einfache Berechnung mit dem Referenzdesign zu erstellen, ruft man einfach die folgende URL auf:

[http://paceval-engine:8080/Demo/?functionString=-sin\(x*cos\(x\)\)^\(1/y\)&numberOfVariables=2&variables=x;y&values=0.5;2&interval=yes](http://paceval-engine:8080/Demo/?functionString=-sin(x*cos(x))^(1/y)&numberOfVariables=2&variables=x;y&values=0.5;2&interval=yes)

Im Schritt „Create“ wird ein Berechnungsobjekt für die Funktion „-sin(x*cos(x))^(1/y)“ erstellt und die Berechnung mit den 2 Variablen x und y für die Werte x=0,5 und y=2 sofort durchgeführt. Variablen und Werte werden immer durch ein ; getrennt. Mit interval=yes wird angegeben, dass neben der rechnergenauen Berechnung auch das obere und untere Intervall der Berechnung ausgegeben wird. Der genaue Wert der Berechnung liegt dann in diesem Intervall (Abbildung 10).

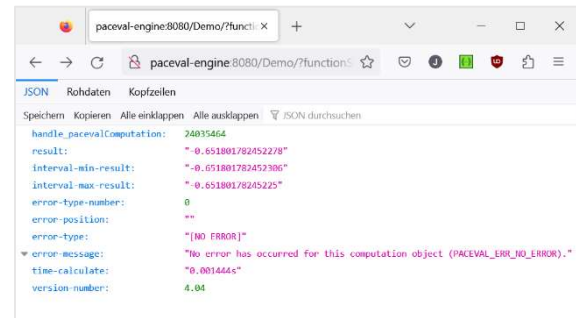


Abbildung 10: Ausgabe des Beispiels im Browser

Natürlich kann man jede beliebige mathematische Funktion und auch andere und längere Variablenamen angeben. Es gibt auch keine Beschränkungen für die Länge einer Funktion und die Anzahl der Variablen. Es können alle geschlossenen mathematischen Funktionen berechnet und kombiniert mit logischen Operatoren zusammen verwendet werden. Damit lassen sich alle geschlossenen finanztechnischen, stochastischen, ingenieur- und naturwissenschaftlichen Funktionen und darüber hinaus alle Modelle für maschinelles Lernen darstellen. Einfacherweise kann darüber hinaus die übliche mathematische Standardschreibweise verwendet werden.

Unterstützt werden aktuell folgende Operatoren, Teilfunktionen und Symbole³⁴:

- Grundrechenarten
+, -, *, /
- Logische Operatoren
NOT, AND, OR, XOR, NAND, XNOR
- Vergleichsoperatoren
<, >, =, >=, <=, <>
- Fakultät
!, fac()
- Konstanten
pi, e
- Klammern
()
- Potenz-/Wurzelfunktionen
^, sqr(), sqrt(), exp()

³⁴ Vollständige Beschreibung der unterstützten Terme in geschlossenen Ausdrücken und Intervallarithmetik, s. <https://paceval.com/product-brief/>

- Logarithmusfunktionen und Sigmoidfunktion
lg(), ln(), sig()
- Trigonometrische Funktionen und zugehörige Inverse
sin(), cos(), tan(), cot(), asin(), acos(), atan(), acot()
- Hyperbolische Funktionen und zugehörige Inverse
sinh(), cosh(), tanh(), coth(), arsinh(), arcosh(), artanh(), arcoth()
- Numerische Manipulationen
sgn(), abs(), round(), ceil(), floor()
- weitere **paceval**. spezifische numerische Manipulationen
ispos(), isposq(), isneg(), isnegq(), isnull()

Zusätzlich werden in der nächsten Version diese Operatoren unterstützt:

- Minimum und Maximum
min, max
- Modulo³⁵ symmetrische und mathematische Variante
%, mod

Bei dem Schritt „Create“ erhält man mit der obigen Beispielberechnung auch einen Verweis (auch „Token“ genannt) auf das generierte Berechnungsobjekt für die Funktion. Ab sofort kann diese Referenz einfach verwendet werden, um Berechnungen für weitere Werte zu erhalten. Referenzen sind 1 Stunde gültig, wobei sich diese Zeitspanne bei jedem Zugriff auf eine Referenz wieder auf 1 Stunde ab dem Zeitpunkt des Zugriffs verlängert. Wenn Berechnungsobjekte für 1 Stunde nicht verwendet werden, so wird automatisch das Berechnungsobjekt aus dem Speicher gelöscht.

Dies hat viele Vorteile. Wird nur die Referenz auf ein Berechnungsobjekt verwendet, muss die unter Umständen sehr lange Funktion nicht jedes Mal übergeben und ausgewertet werden. Das spart Zeit bei der Übertragung, da nur die eigentlichen Variablenwerte für die Berechnung gesendet werden müssen und darüber hinaus Rechenleistung und somit Strom.

Wenn man beispielsweise eine Referenz „handle_Computation: 24035464“ erhalten hat, ruft man einfach die folgende URL für eine weitere Berechnung mit den Werten 0,46577 für x und 2,61 für y auf (Abbildung 11):

http://paceval-engine:8080/GetComputationResult/?handle_pacevalComputation=24035464&values=0.46577;2.61

Es können auch cUrl-Anweisungen aus der Konsole heraus oder von einem IoT-Gerät direkt ausgeführt werden. cURL (Client for URLs) ist eine Programm-bibliothek und ein Kommandozeilen-Programm zum Übertragen von Dateien in Rechnernetzen. Die Programm-bibliothek steht für nahezu alle denkbaren Betriebssysteme und Prozessoren zur Verfügung³⁶. cUrl bietet die http Methoden GET und POST. Nachfolgend sind die



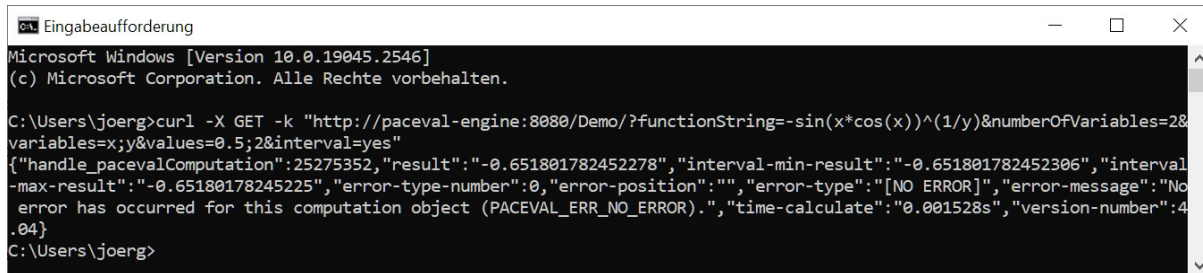
Abbildung 11: Eine neue Berechnung über die Referenz des vorherigen Beispiels

³⁵ Modulo oder „Division mit Rest“, s. https://de.wikipedia.org/wiki/Division_mit_Rest

³⁶ Die cURL Download-Seite listet mehr als 30 Betriebssysteme, s. <https://curl.se/download.html>

Aufrufe analog zu dem Beispiel oben und die jeweiligen Ausgaben auf der Windows Konsole ausgegeben:

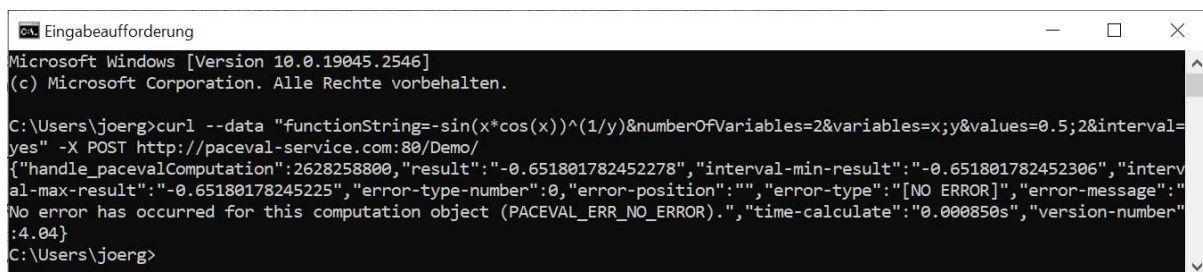
```
curl -X GET -k "http://paceval-engine:8080/Demo/?functionString=-sin(x*cos(x))^(1/y)&numberOfVariables=2&variables=x;y&values=0.5;2&interval=yes"
```



```
Microsoft Windows [Version 10.0.19045.2546]
(c) Microsoft Corporation. Alle Rechte vorbehalten.

C:\Users\joerg>curl -X GET -k "http://paceval-engine:8080/Demo/?functionString=-sin(x*cos(x))^(1/y)&numberOfVariables=2&variables=x;y&values=0.5;2&interval=yes"
{"handle_pacevalComputation":25275352,"result":"-0.651801782452278","interval-min-result":"-0.651801782452306","interval-max-result":"-0.65180178245225","error-type-number":0,"error-position":"","error-type":"[NO ERROR]","error-message":"No error has occurred for this computation object (PACEVAL_ERR_NO_ERROR).","time-calculate":"0.001528s","version-number":4.04}
C:\Users\joerg>
```

```
curl --data "functionString=-sin(x*cos(x))^(1/y)&numberOfVariables=2&variables=x;y&values=0.5;2&interval=yes" -X POST http://paceval-service.com:80/Demo/
```



```
Microsoft Windows [Version 10.0.19045.2546]
(c) Microsoft Corporation. Alle Rechte vorbehalten.

C:\Users\joerg>curl --data "functionString=-sin(x*cos(x))^(1/y)&numberOfVariables=2&variables=x;y&values=0.5;2&interval=yes" -X POST http://paceval-service.com:80/Demo/
{"handle_pacevalComputation":2628258800,"result":"-0.651801782452278","interval-min-result":"-0.651801782452306","interval-max-result":"-0.65180178245225","error-type-number":0,"error-position":"","error-type":"[NO ERROR]","error-message":"No error has occurred for this computation object (PACEVAL_ERR_NO_ERROR).","time-calculate":"0.000850s","version-number":4.04}
C:\Users\joerg>
```

Die angebotene API ist wie folgt (jeweils für GET und POST)³⁷:

- /Demo
Dieser Funktionsaufruf dient nur zu Demozwecken. Es erstellt eine Remote-Berechnung, und wenn die Erstellung erfolgreich war, gibt diese Funktion das Berechnungsergebnis zurück. Das bedeutet, intern ruft die Funktion /CreateComputation und /GetComputationResult in einem Schritt auf (s. auch das Beispiel oben).
- /CreateComputation
Erstellt ein Remote-Berechnungsobjekt für die mathematische Funktion f sowie den Variablen $x_1, x_2, \dots, x_n$, d.h.

$$f(x_1, x_2, \dots, x_n)$$

Zusätzlich kann noch gesetzt werden, ob mit oder ohne Intervallarithmetik gerechnet werden soll.

Die beim Aufruf von /CreateComputation zurückgegebene Referenz ist ein Token, das eindeutig das Remote-Berechnungsobjekt darstellt, welches vom entfernten Dienst verwaltet wird. Mit diesem Token können dann Berechnungen und Abfragen auf dem

³⁷ Als Referenz gilt SwaggerHub für "paceval-service - the Mathematical Engine as a Service (e.g. for multi-party computations)", s. <https://app.swaggerhub.com/apis-docs/paceval/paceval-service/4.04>

Remote-Berechnungsobjekt durchgeführt werden, ohne dass erneut die gegebenenfalls sehr lange Funktion erneut übertragen werden muss.

- **/GetErrorInformation**
Gibt den Fehlertyp des Remote-Berechnungsobjekts zurück.
Mögliche Fehler sind Analyse- und Rechenfehler. Fehler während der Analyse sollten nach der Erstellung des Objekts wie **/CreateComputation** überprüft werden. Fehler während der Berechnung sollten nach **/GetComputationResult** überprüft werden.
- **/GetComputationInformationXML**
Eine Hilfsfunktion zum Abrufen von Eigenschaften von einem Remote-Berechnungsobjekt, z.B. die Anzahl der Rechenkerne im Remote-System, die Anzahl der verwendeten Threads oder die Cache-Treffer in den bisherigen Berechnungen mit diesem Berechnungsobjekt.
- **/GetComputationResult**
Führt eine Berechnung auf **einem Remote-Berechnungsobjekt** mit den von **/CreateComputation** deklarierten Variablen und **mit einem Satz an Werten** $a_1, a_2, \dots, a_n$ für diese Variablen durch, d.h.

$$y = f(a_1, a_2, \dots, a_n)$$

- **/GetComputationResultExt**
Führt parallel mehrere Berechnungen auf **einem Remote-Berechnungsobjekt** mit den von **/CreateComputation** deklarierten Variablen und **mit mehreren Sätzen an Werten** $a_{11}, a_{12}, \dots, a_{1n}, a_{21}, a_{22}, \dots, a_{2n} \dots a_{m1}, a_{m2}, \dots, a_{mn}$ für diese Variablen durch, d.h.

$$\begin{aligned} y_1 &= f(a_{11}, a_{12}, \dots, a_{1n}) \\ y_2 &= f(a_{21}, a_{22}, \dots, a_{2n}) \\ &\dots \\ y_m &= f(a_{m1}, a_{m2}, \dots, a_{mn}) \end{aligned}$$

Es sollte **/GetComputationResultExt** verwendet werden, um viele Ergebnisse mit einem einzigen Aufruf auf einmal zu erhalten. Dies ist beispielsweise nützlich, wenn sehr schnell viele verschiedene Werte analysiert werden müssen.

- **/GetMultipleComputationsResults**
Führt parallel mehrere Berechnungen auf **mehreren Remote-Berechnungsobjekten** mit den von **/CreateComputation** deklarierten Variablen und **mit dem gleichen Satz an Werten** $a_1, a_2, \dots, a_n$ für diese Variablen durch, d.h.

$$\begin{aligned} y_1 &= f_1(a_1, a_2, \dots, a_n) \\ y_2 &= f_2(a_1, a_2, \dots, a_n) \\ &\dots \\ y_m &= f_m(a_1, a_2, \dots, a_n) \end{aligned}$$

Es sollte **/GetMultipleComputationsResults** verwendet werden, um Ergebnisse für mehrere Remote-Berechnungsobjekte gleichzeitig zu erhalten (z.B. für künstliche neuronale

Netzwerkfunktionen oder Entscheidungsbäume, bei denen in einem Schritt mehrere Funktionen mit gleichen Variablenwerten berechnet werden, um ein Objekt zu entscheiden).

Zusätzlich wird in der nächsten Version von *paceval*. diese API-Funktion unterstützt:

- `/GetMultipleComputationsResultsExt`
Führt parallel mehrere Berechnungen auf **mehreren Remote-Berechnungsobjekten** mit den von `/CreateComputation` deklarierten Variablen und **mit unterschiedlichen Sätzen an Werten** $a_{11}, a_{12}, \dots, a_{1n}, a_{21}, a_{22}, \dots, a_{2n} \dots a_{m1}, a_{m2}, \dots, a_{mn}$ für diese Variablen durch, d.h.

$$\begin{aligned} y_1 &= f_1(a_{11}, a_{12}, \dots, a_{1n}) \\ y_2 &= f_2(a_{21}, a_{22}, \dots, a_{2n}) \\ &\dots \\ y_m &= f_m(a_{m1}, a_{m2}, \dots, a_{mn}) \end{aligned}$$

Es sollte `/GetMultipleComputationsResultsExt` verwendet werden, um Ergebnisse für mehrere Remote-Berechnungsobjekte gleichzeitig zu erhalten und möglichst unterschiedliche Auswertungen auf einmal durchzuführen (z.B. für künstliche neuronale Netzwerkfunktionen oder Entscheidungsbäume, bei denen in einem Schritt mehrere Funktionen mit unterschiedlichen Variablenwerten berechnet werden, um mehrere Objekte gleichzeitig zu entscheiden).

Diese API-Funktionen, inklusive der neuen `/GetMultipleComputationsResultsExt`, können bereits jetzt mit dem Service, der zu Demonstrationszwecken Kubernetes nutzt, ausprobiert werden. Die API findet sich auf SwaggerHub unter

<https://app.swaggerhub.com/apis/paceval/paceval-service-cloud/>

3.4 Arbeitsweise des Zielsystems

Das Referenzdesign führt HTTP-Anfragen für Berechnungen mit Variablenwerten über die Ethernet Schnittstelle des ZC706 Entwicklungsboard durch.³⁸ Diese Anfragen werden vom *paceval*-service verarbeitet und per Foreign function interface (FFI) an die *paceval*. Bibliothek weitergegeben.

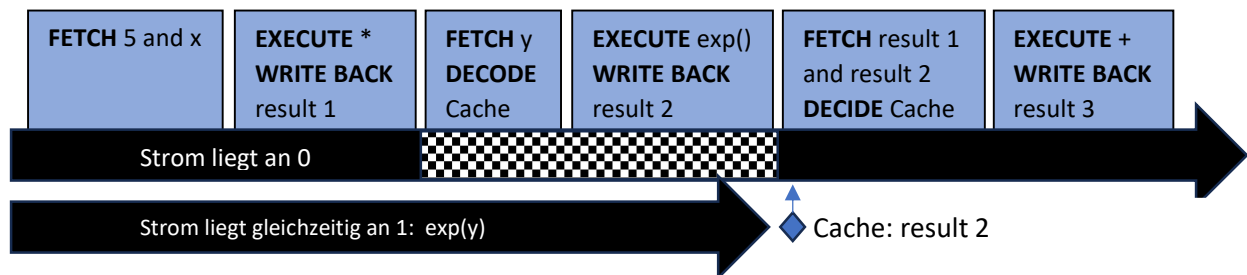
Für die Verarbeitung der Berechnung selbst gibt es nun die zwei Möglichkeiten, die zu automatisieren wären. Es wird entweder 1. die übliche Vorgehensweise gewählt, dass über Threads in Software die Berechnungen parallelisiert werden im Verarbeitungssystem (PS). Oder aber, dass 2. die neue Möglichkeit gewählt wird, dass die Berechnungen in Hardware parallelisiert werden in der Programmierbaren Logik (PL) (Abbildung 12).

³⁸ Da das gewählte ZC706 Entwicklerboard mit USB, Ethernet, SPI, SD/SDIO, I2C, CAN, UART und GPIO mannigfaltige Möglichkeiten zur Kommunikation bietet, könnte auch ohne weiteres eine andere Schnittstelle als Ethernet gewählt werden.

Eine Parallelisierung der Berechnungen in Hardware hat enorme Vorteile da gleichzeitig Strom an den unterschiedlichen Inputs der Schaltung anliegt. Nehmen wir zur Anschauung wieder die folgende Funktion und die anschließende Berechnung mit den angegebenen Variablenwerten:

$$f(x, y) = 5 * x + \exp(y) \quad \text{mit } x=2.2 \text{ und } y=3.3$$

Der Schritt „Create“, bzw. der API-Aufruf von /CreateComputation wird dann (vereinfacht) wieder diese verkettete Liste mit der Bibliothek von **paceval**. erzeugen und mit den Markern für mögliche Parallelisierung versehen. Allerdings wird nun diese Liste direkt auf der Programmierbaren Logik (PL) des FPGAs, also durch eine elektronische logische Schaltung abgearbeitet, analog wie durch die Threads bei der Softwareimplementierung³⁹. Diese wird insbesondere sehr schnell im Nanosekundenbereich schalten:



Der Schritt „Calculate“ wird dann die Berechnung auf 2 Schaltungen gleichzeitig durchführen. Die Schaltung 0 hat die Aufgabe, die gesamte Berechnung durchzuführen, d.h. sie läuft durch die gesamte Liste. Sollte Schaltung 1 weniger Takte für exp(3.3) benötigen als 5 * x, so wird das Ergebnis der parallelen Berechnung von exp(3.3) in Schaltung 1 für die finale Addition übernommen und die gesamte Berechnung, die Schaltung 1 abgedeckt hat, wird übersprungen.

Die 2. Vorgehensweise Berechnungen in Hardware zu parallelisieren (Abbildung 12), wurde in diesem Validierungsauftrag von dem Institut für Informatik in Oldenburg OFFIS untersucht.

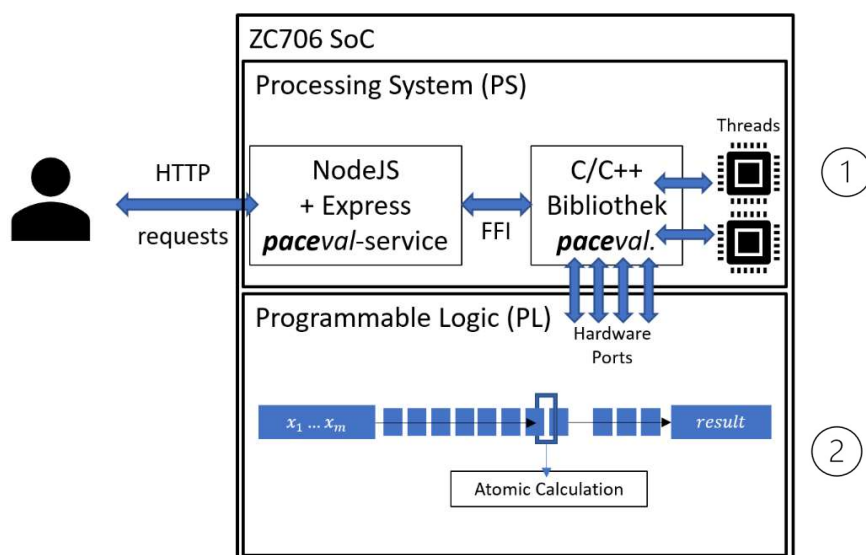


Abbildung 12: Blockdiagramm der hardware-basierten mathematischen Engine as a Service auf dem ZC706 SoC

³⁹ Die Softwareimplementierung durch Threads ist in Kapitel „3.3 Berechnungen mit der mathematischen Engine as a Service auf dem SoC“ beschrieben

4. Begleitende Technologien der *paceval*. Entwicklung

Das wie oben definierte Produkt mit FPGA lässt sich mit den bereits vorhandenen Technologien verbinden bzw. ergänzen. Die Homepage und insbesondere GitHub von *paceval*. bieten hier verschiedene Möglichkeiten in Quellcode an, die als Basis für eigene weiterführende Entwicklungen dienen können. Je nach Interessen kann ein Benutzer selber einen Berechnungsserver mit wenigen Schritten aufsetzen, ein Hyperscaler wie Google Cloud Platform könnte einen entsprechenden Service hochskaliert mit Kubernetes anbieten, Wissenschaftler oder Studenten würden sicherlich das direkte Python Interface nutzen. Der Kreativität an Integrationsmöglichkeiten sind hier keine Grenzen gesetzt. Im Folgenden werden diese begleitenden Technologien kurz dargestellt.

4.1 Die Bibliothek von *paceval*. und ihre Funktionen

Die Bibliothek von *paceval*. ist systemunabhängig in den Programmiersprachen C und C++ geschrieben und frei von Drittsoftware. Der Quellcode von *paceval*. ist nach dem ISO-Standard (ISO/IEC 14882:1998)⁴⁰ und läuft somit auf jedem Compiler seit den 1990er Jahren (Abbildung 13). Die Bibliothek selbst ist besonders klein (für ARM32 ca. 300 Kilobytes) und durch seine schmale Programmierschnittstelle⁴¹ mit geringem eigenen Programmieraufwand auf den wichtigsten Computersystemen sofort lauffähig. Das einfach gehaltene Interface ermöglicht die Verwendung in alle gängigen Programmiersprachen über das Foreign function interface (FFI)⁴² (Abbildung 14).

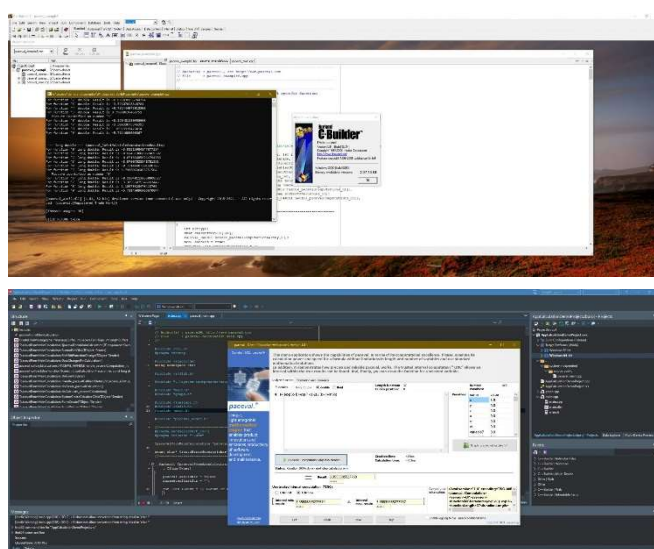


Abbildung 13: Applikationsentwicklung mit der Bibliothek von *paceval*. auf einem Compiler aus dem Jahr 2000 und einem heutigen Compiler

Ein Foreign function interface ist ein Mechanismus, durch den ein in einer Programmiersprache geschriebenes Programm Routinen aufrufen oder Dienste verwenden kann, die in einer anderen geschrieben sind.

Programmiersprachen und Entwicklungsumgebungen die sofort per FFI mit der Bibliothek von *paceval*. arbeiten können sind unter anderen .NET, ABAP, C, C#, COBOL, Delphi, Erlang, Fortran, Golang, Java, Maple, Julia, Kotlin, Lua, Mathematica, MATLAB, Mendix, Node.js, Octave, Pascal, Perl, PHP, Python, R, Ruby, Rust, Scratch, Swift, TypeScript oder Visual Basic. Stand heute gibt es Bibliotheken von *paceval*. offiziell für Windows (INTEL/AMD 32bit und 64bit), macOS (inkl. APPLE Silicon M1 und M2), Linux (INTEL/AMD und ARM), ARM (32bit und 64bit), iOS und Android. Es gibt darüber hinaus noch kundenspezifische Bibliotheken für Microcontroller von

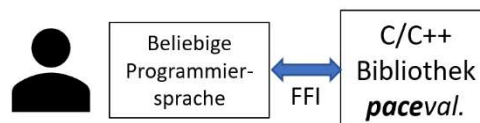


Abbildung 14: Programmierschnittstelle über Foreign function interface (FFI)

⁴⁰ Standard ISO/IEC 14882:1998 Programming languages C++, s. <https://www.iso.org/standard/25845.html>

⁴¹ Application programming interface (API) von *paceval*., s. <https://paceval.com/api>

⁴² Foreign function interface, s. https://en.wikipedia.org/wiki/Foreign_function_interface

Microchip AVR, TI, STMicroelectronics und NXP Semiconductors und neue offene Prozessorsysteme wie RISC-V⁴³.

Wissenschaftliche Programme, Ingenieursoftware, Standard-Branchenlösungen, gängige Datenbanken oder proprietäre Berechnungsprogramme benötigen üblicherweise einen polynomialen oder sogar exponentiellen Zeitaufwand (Abbildung 15). Allgemein gilt je komplexer die Berechnungsanfrage, desto höher ist der benötigte Zeitaufwand. Komplexe Berechnungen benötigen zu lange oder können gar nicht verarbeitet werden. Proprietäre Codes der üblichen Ingenieursoftware bieten zwar lineare Laufzeit. Diese lassen aber dann keine Flexibilität oder weitere Optimierungen der Laufzeit zu und müssen gepflegt werden. **paceval**. dagegen berechnet geschlossene mathematische Funktionen jeder Länge und mit einer beliebigen Anzahl von Variablen, üblicherweise in Millisekunden und systemunabhängig. Dies wird durch einen einzigartigen mathematischen Parser mit mehrfacher Caching-Funktionalität ermöglicht. Bei wiederholten Teilrechnungen kann so eine logarithmische Laufzeit erreicht werden.

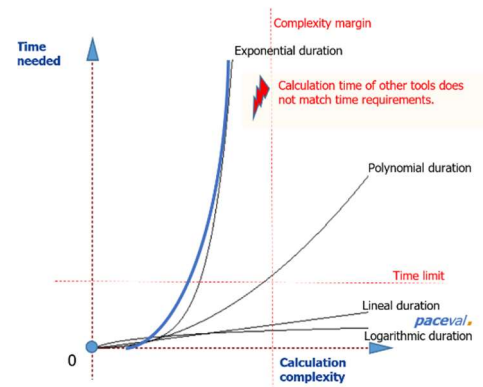


Abbildung 15: Laufzeiten von anderen Lösungen und **paceval**.

Computer rechnen nicht mathematisch genau. Jedoch je komplexer ein mathematisches Modell ist, umso wichtiger ist auch seine Präzision. Im Allgemeinen wird sich die Präzision rapide in Abhängigkeit von der Komplexität des zugrundeliegenden mathematischen Modells bzw. der Länge der Funktionen und der Anzahl der Variablen verschlechtern. Zudem hängt die Genauigkeit der Berechnungen stark von der verwendeten Plattform ab und es entstehen Rundungsfehler bei jeder Teilrechnung. Es ist deshalb häufig sinnvoll, den maximalen Fehler einer Berechnung zeitnah zu bestimmen. Nur dann kann man entscheiden, ob das Ergebnis, das System oder ein Produkt noch in einem Toleranzbereich operiert und entsprechende Maßnahmen ergreifen (Abbildung 16). **paceval**. berücksichtigt dies entsprechend mit einer eigenen Methodik, die „Trusted interval computing“ (TINC) genannt wird. In Abhängigkeit von der vorhandenen Plattform rechnet **paceval**. intern mit bis zu 19 Dezimalstellen Genauigkeit, und liefert zusätzlich zum Standard-Berechnungswert die obere und untere Intervallgrenze, in der sich der wahre Berechnungswert befindet. Bei **paceval**. ist die Möglichkeit, Ergebnisintervalle zu bestimmen, in allen mathematischen Teilfunktionen implementiert. Dadurch lässt sich eine im Vergleich zu anderen Lösungen einzigartige Genauigkeit bei den Berechnungen erzielen.

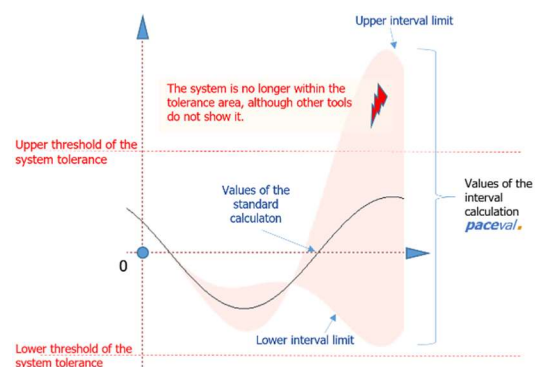


Abbildung 16: Problemstellung bei mathematischen Ungenauigkeiten

Auf die von **paceval**. unterstützten Operatoren, Teilfunktionen und Symbole wurde bereits im obigen Kapitel „3.3 Berechnungen mit der mathematischen Engine as a Service auf dem SoC“ eingegangen. Zur Anschauung soll nur kurz dargestellt werden, wie diese im Rahmen von

⁴³ RISC-V International besteht aus einer großen Mitgliedsorganisation, die die erste offene, kollaborative Gemeinschaft von Software- und Hardware-Innovatoren aufbaut, s. <https://riscv.org/>

sogenannter künstlicher Intelligenz genutzt werden können. Zwei wichtige mathematische Modelle hierbei sind Entscheidungsbäume und künstliche neuronale Netze.

- Das Lernen von Entscheidungsbäumen oder die Induktion von Entscheidungsbäumen ist einer der prädiktiven Modellierungsansätze, die in der Statistik, im Data Mining und im maschinellen Lernen verwendet werden (Abbildung 17).

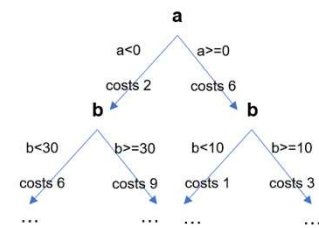


Abbildung 17: Entscheidungsbaum

- Ein künstliches neuronales Netzwerk ist eine miteinander verbundene Gruppe von Knoten, inspiriert von einer Vereinfachung von Neuronen in einem Gehirn. Neuronale Netze lernen (bzw. werden trainiert) durch die Verarbeitung von Beispielen (Abbildung 18).

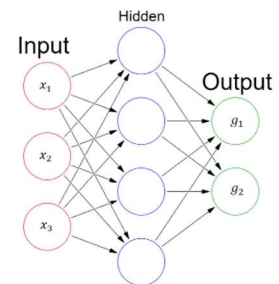


Abbildung 18: Neuronales Netz

Diese Techniken können auf eine einzige geschlossene mathematische Funktion abgebildet werden

$$y = f(x_1, x_2, \dots, x_n)$$

oder auf eine Menge geschlossener mathematischer Funktionen

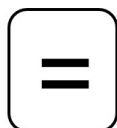
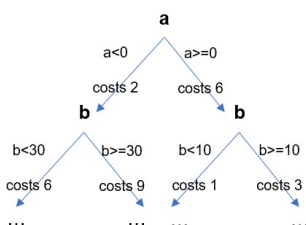
$$y_1 = f_1(x_1, x_2, \dots, x_n)$$

$$y_2 = f_2(x_1, x_2, \dots, x_n)$$

...

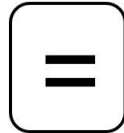
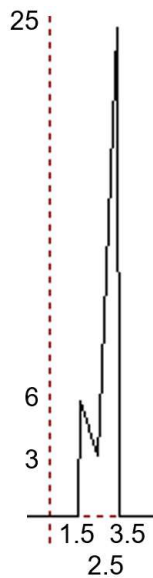
$$y_m = f_m(x_1, x_2, \dots, x_n)$$

Beispiel Entscheidungsbaum mit Variablen und Kosten:



$$f(a, b) = ((a < 0) \text{ AND } (b < 30)) * (2+6) \\ + ((a < 0) \text{ AND } (b \geq 30)) * (2+9) \\ + ((a \geq 0) \text{ AND } (b < 10)) * (6+1) \\ + ((a \geq 0) \text{ AND } (b \geq 10)) * (6+3) \\ \dots$$

Beispiel Approximation:



$$f(x) = ((x \geq 1.5) \text{ AND } (x \leq 2.5)) * (((3-6)/(2.5-1.5)) * (x-1.5) + 6) + ((x > 2.5) \text{ AND } (x \leq 3.5)) * (((25-3)/(3.5-2.5)) * (x-2.5) + 3)$$

Beispiel Matrixmultiplikation mit Variablen:

$$\begin{pmatrix} x & 6 & 4 \\ 7 & y & -7 \\ -2 & z & 12 \end{pmatrix} * \begin{pmatrix} 7 & 8 & -2 \\ 9 & u & v \\ w & -18 & 3 \end{pmatrix}$$



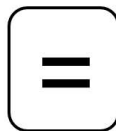
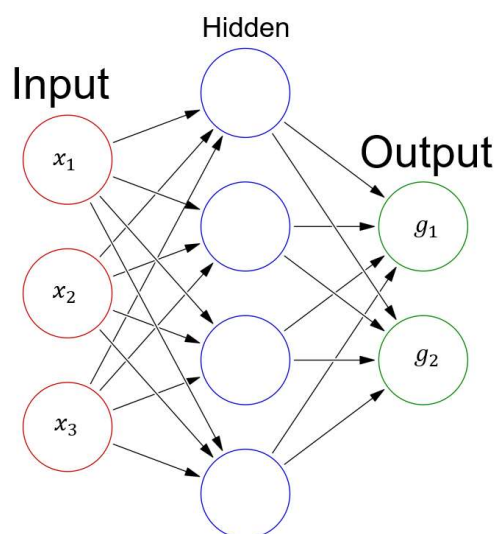
$$c_{1,1}(x, y, z, u, v, w) = x * 7 + 6 * 9 + 4 * w$$

$$c_{1,2}(x, y, z, u, v, w) = x * 8 + 6 * u + 4 * (-18)$$

...

$$c_{3,3}(x, y, z, u, v, w) = (-2) * (-2) + z * v + 12 * 3$$

Beispiel künstliche neuronale Netze:



$$g_1(x_1, x_2, x_3) = (((1.1544664390561268 * (1 / (1 + \exp((0.04934491206103983 * x_1)) + (0.12975707678433665 * x_2)) + (0.06215928752528358 * x_3)))$$

...

$$g_2(x_1, x_2, x_3) = \dots$$

Dies sind nur vier Beispiele. Weitere naheliegende wären gewichtete gerichtete Graphen, Differentialgleichungen, Skalarprodukte, geometrische Transformationen⁴⁴, Matrix- und Vektoroperationen, Hidden Markov Modelle für schnelle Gestenerkennung, Transformationen (DCT⁴⁵, FFT⁴⁶), Newtonverfahren, Singulärwertzerlegung, Polynome, Eigenwerte, Zyklische Redundanzprüfungen, Zero-Knowledge Proofs und allgemein nahezu alle Themen der Numerischen Mathematik⁴⁷.

Die Beispiele können natürlich beliebig komplex formuliert werden. Das Beispiel des Entscheidungsbaums mit Variablen und Kosten kann auf beliebig viele Äste und Variablen erweitert werden. Die Funktion wird umso länger, je mehr Variablen verwendet werden. Seien dies beispielsweise Werte von 20 Sensoren in einem System und habe jeder Sensor zwei Entscheidungen, so wären dies bereits $2^{20} = 1.048.576$ mögliche Entscheidungswege. Müsste man auf üblichem Wege ein Programm für alle diese Entscheidungswege schreiben, so würde dies sehr umfangreich und sehr schwer zu warten sein. Und ein solches Programm wäre zudem unglaublich komplex und fehleranfällig, wenn sich die Kosten ändern, Sensoren hinzukommen oder Entscheidungen dynamisch gefällt werden müssten. Mit **paceval**. ist dies dagegen kein Problem. Oder anders ausgedrückt: Anstatt mehrerer Hunderttausend oder Millionen an Zeilen Quellcode ist dies mit **paceval**. in wenigen Zeilen möglich.

Das kostenlose Software Development Kit von **paceval**. kann von der Internetseite als Windows Installationsprogramm geladen werden unter <https://paceval.com/product-demo>. Dieses enthält Demos und Beispiele für Windows, Apple, Linux, ARM, die neuesten Compiler und frühere Compiler-Versionen.

4.2 **paceval**-service und Docker Container

Der Dienst **paceval**-service ist wie auch im Kapitel „3.2 PetaLinux und die gewählten Softwarekomponenten auf dem SoC“ beschrieben, ein HTTP-Service, der üblicherweise an Port 8080 auf Eingaben wartet. Derselbe Node.js Quellcode, der nun für das Referenzdesign dieses

Validierungsauftrags verwendet wird, wurde ursprünglich für den Docker Container implementiert. Der Docker Container kapselt hiermit einen Dienst, der mathematische Berechnungen auf einem entfernten Computer oder Server ermöglicht, auf dem dieser Docker Container gestartet ist (Abbildung 19). Stand heute gibt es den **paceval**-service als Docker Container Image für ARM64 Prozessoren (bspw. APPLE M1/M2 und auch den

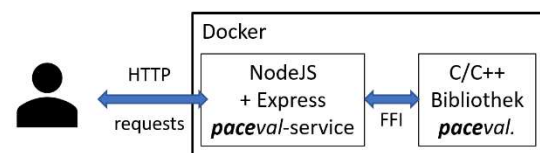


Abbildung 19: Funktionsweise Docker Container

⁴⁴ Unter geometrischen Transformationen versteht man das Verschieben, Vergrößern und Verkleinern, Drehen, Spiegeln usw. von Objekten innerhalb eines Koordinatensystems oder zwischen Koordinatensystemen.

⁴⁵ Die diskrete Kosinustransformation (DCT) ist eine Transformation der numerischen Mathematik. Sie wird z.B. für die verlustbehaftete Kompression von Audio- und Bilddaten verwendet.

⁴⁶ Die schnelle Fourier-Transformation (FFT) ist ein Algorithmus zur effizienten Berechnung der diskreten Fourier-Transformation (DFT). Mit ihr kann ein zeitdiskretes Signal in seine Frequenzanteile zerlegt und dadurch analysiert werden.

⁴⁷ Die numerische Mathematik, auch kurz Numerik genannt, beschäftigt sich als Teilgebiet der Mathematik mit der Konstruktion und Analyse von Algorithmen für kontinuierliche mathematische Probleme. Hauptanwendung ist dabei das näherungsweise Berechnen von Lösungen durch Approximationsalgorithmen mit Hilfe von Computern. S.

https://de.wikipedia.org/wiki/Numerische_Mathematik

beliebten Raspberry Pi) sowie für x64 Prozessoren von Intel und AMD.

Die Installation auf einem beliebigen System ist sehr einfach. Man führt diese Befehlszeilen im Terminal aus, um den Dienst mit Docker abzurufen und zu starten:

Linux für x64 Prozessoren

```
sudo docker pull paceval/paceval-service_linux_x64:latest
sudo docker run -p 8080:8080 -d paceval/paceval-service_linux_x64
```

Linux für ARM64 Prozessoren

```
sudo docker pull paceval/paceval-service_linux_arm64:latest
sudo docker run -p 8080:8080 -d paceval/paceval-service_linux_arm64
```

Danach steht der Dienst zur Verfügung und kann analog wie in Kapitel „3.3 Berechnungen mit der mathematischen Engine as a Service auf dem SoC“ für Berechnungen genutzt werden. Der **paceval**-service lauscht standardmäßig auf Port 8080. Der Port kann allerdings frei konfiguriert werden.

Berechnungsobjekte werden dann per HTTP-Request bspw. im Browser oder mit cURL erstellt und ausgeführt.

Um eine Berechnung direkt durchzuführen, ruft man im Browser einfach die folgende URL auf:

[http://localhost:8080/Demo/?functionString=sin\(x*cos\(x\)\)^\(1/y\)&numberOfVariables=2&variables=x;y&values=0.5;2&interval=yes](http://localhost:8080/Demo/?functionString=sin(x*cos(x))^(1/y)&numberOfVariables=2&variables=x;y&values=0.5;2&interval=yes)

Die komplette Dokumentation für den **paceval**-service findet sich auf SwaggerHub unter <https://app.swaggerhub.com/apis-docs/paceval/paceval-service/4.04>.

Der Quellcode des **paceval**-service ist in GitHub verfügbar unter <https://github.com/paceval>.

4.3 Hyperscaler und Skalierung mit Kubernetes

Wie vorher beschrieben, gibt es den **paceval**-service als Docker Container Image.

Für die Skalierung gibt es erste Helm Charts⁴⁸, die eine Orchestrierung mit Kubernetes auf einem Hyperscaler einrichten (Abbildung 20). Unter Hyperscalern versteht man im Allgemeinen die drei großen Cloud-Anbieter Amazon, Microsoft und Google. Die Amazon Web Services (AWS), Microsoft Azure und die Google Cloud Platform decken zusammen etwa 75 Prozent des Gesamtmarktes bei Public-Cloud-Angeboten ab. Für das Hyperscaling sind tausende Server und Storage-Systeme über leistungsfähige Netzwerke

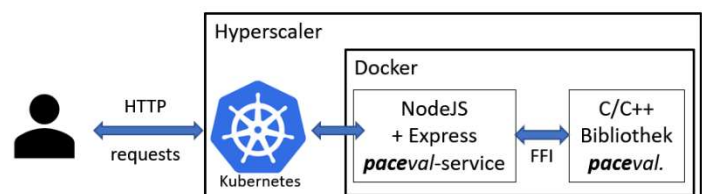


Abbildung 20: Skalierung des **paceval**-service bei einem Hyperscaler

⁴⁸ Helm ist ein Paketmanager zum einfachen Installieren, Veröffentlichen, Verwalten, Updaten und Skalieren von vorkonfigurierten Kubernetes-Anwendungen. Helm benötigt Anweisungen (genannt Helm Charts), um zu wissen, wie genau eine Kubernetes-Anwendung aufgebaut ist.

miteinander verbunden. Durch das Hinzufügen weiterer Systeme lassen sich die Ressourcen fast beliebig erweitern. Hyperscaling-Anbieter bedienen sehr viele Kunden gleichzeitig. Sie stellen große Rechenleistung und hohe Speicherkapazität zur Verfügung. Neben den klassischen Servicemodellen des Cloud Computings wie Infrastructure-as-a-Service (IaaS), Platform-as-a-Service (PaaS) und Software-as-a-Service (SaaS) bieten sie zahlreiche weitere Services und Tools an.

Dies sind die Vorteile für Anwender bei der Nutzung eines Hyperscalers:

- Ressourcen lassen sich fast beliebig erweitern und den eigenen Bedürfnissen anpassen
- Hohe Redundanz, Sicherheit der eigenen Dienste und Ausfallsicherheit
- Keine eigene Hard- und Software-Investitionen notwendig
- Kosteneffiziente Nutzung der IT-Ressourcen
- Transparente, leicht kalkulierbare Kosten
- Hohe Flexibilität durch Auswahl eines geeigneten Hyperscalers

Dies sind die Nachteile für den Anwender bei der Nutzung eines Hyperscalers:

- Datensicherheit, Datenhoheit und Souveränität sorgt immer wieder für Probleme
- Hohe Anforderungen an IT-Mitarbeitende bzgl. der Möglichkeiten eines Hyperscalers
- Abhängigkeit und Bindung an die Preismodelle des Anbieters (Vendor Lock-In)

Der Vendor Lock-In kann vermieden werden durch die parallele Nutzung mehrerer Hyperscaling-Anbieter. Dies erhöht dann allerdings nochmals die notwendigen hohen Anforderungen an die eigenen IT-Mitarbeiter. Die Kosten für die Nutzung der Infrastruktur mehrerer Anbieter gleichzeitig sind zudem nicht zwangsläufig geringer. Wenn das Wissen um die Möglichkeiten des jeweiligen Hyperscalers bei den IT-Mitarbeitern vorhanden ist, so ist insbesondere die hohe Redundanz, Sicherheit der eigenen Dienste und Ausfallsicherheit, die quasi mitgeliefert wird, nicht zu unterschätzen.

Alle Hyperscaler haben ein umfangreiches Angebot im Bereich Container und Kubernetes (K8s). Docker ist dabei der de-facto Standard der Container Technologie und Kubernetes derjenige für die Orchestrierung davon. Kubernetes kommt ins Spiel, wenn man mehrere Container orchestrieren muss. Die Container sind dabei verpackt in sogenannte Pods, wobei in der Regel ein Container einem Pod entspricht. Pods sind die kleinste managebare Einheit für Kubernetes.

Dieses Dokument „Guide for deploying the Kubernetes native *paceval*-service on any cluster“ beschreibt, wie man einen eigenen *paceval*-service auf Kubernetes der Google Cloud Platform (GCP) mit leicht verständlichen Schritten einrichtet:

https://github.com/paceval/paceval/blob/main/examples_sources/NodeJS_examples/k8s/doc/Readme.md

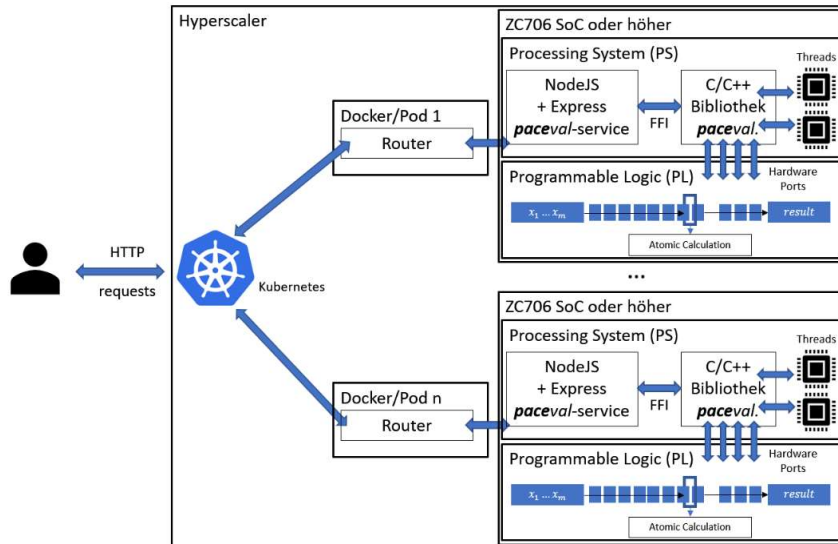


Abbildung 21: Schema der Skalierung für die Hyperscaler mit dem Referenzdesign und Kubernetes

Die Anweisungen können für jeden anderen Hyperscaler wie Amazon Web Services (AWS) oder Microsoft Azure übernommen und angepasst werden. Die Umsetzung für eine Interaktion mit dem Referenzdesign oder höher (bspw. basierend auf der Zync UltraScale+ Plattform⁴⁹) aus diesem Validierungsauftrag wäre für die Hyperscaler simpel über selbstgebaute Router in Software (mit Node.js oder Golang) zu lösen⁵⁰ (Abbildung 21).

4.4 Python Interface

Da Python sich großer Beliebtheit erfreut, gibt es naturgemäß auch ein Python Modul von *paceval*. Dieses Python Modul kapselt Aufrufe zu der URL eines beliebigen *paceval*-service (Abbildung 22). Die Handhabung ist, wie bei Python üblich, sehr einfach.

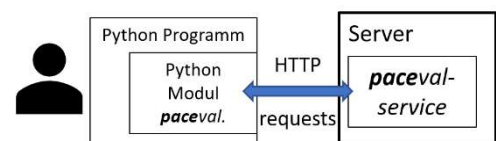


Abbildung 22: Python Programmierung mit dem *paceval*-service

Das folgende Beispiel macht Abfragen gegen den frei verfügbaren Server von den Entwicklern von *paceval*. selbst:

```
import paceval
import json
from types import SimpleNamespace

#paceval. Server
url = "http://paceval-service.com"
```

⁴⁹ Zynq UltraScale+ platform, s. <https://www.xilinx.com/products/silicon-devices/soc/xa-zynq-ultrascale-mpsoc.html#productAdvantages>

⁵⁰ Hyperscaler wäre hier auch stellvertretend für ein beliebiges Rechenzentrum, bei dem dieselbe Umsetzung erfolgen kann.

```
demoResponse = paceval.Demo(url, "-sin(x*cos(x))^(1/y)", "2", x;y", "0.5;2",
"yes").text
```

Natürlich könnte das Python Modul auch mit einem Hyperscaler und Kubernetes arbeiten. Dann ergäbe sich das folgende Bild (Abbildung 23):

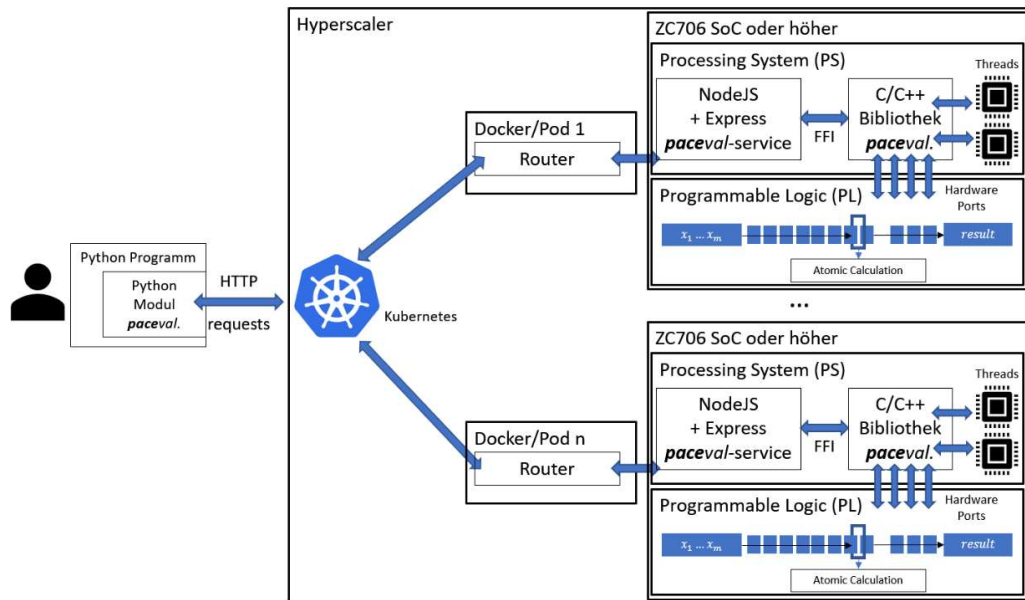


Abbildung 23: Schema wie das Python Programm per Python Modul, mit dem Referenzdesign bei einem Hyperscaler interagiert

Die komplette Dokumentation für das Python Modul von **paceval** findet sich auf des Software-Verzeichnis Python Package Index (PyPI) unter <https://pypi.org/project/paceval>.

Der Quellcode des Python Moduls von **paceval** ist in GitHub verfügbar unter <https://github.com/paceval>.

4.5 Eclipse Diafanis Projekt

Das Referenzdesign wäre allerdings nicht nur für Hyperscaler interessant. Da das gewählte ZC706 SoC auch für Automotive bzw. ebenfalls allgemein für Produktlösungen aus dem Feld der Internet of Things (IoT)⁵¹ ist, wurde das Eclipse Diafanis™ Projekt⁵² erzeugt. Dieses Projekt gehört zu Eclipse IoT der Eclipse Foundation. Dies ist eine gemeinnützige Gesellschaft mit der Aufgabe, die Eclipse-Open-Source-Gemeinschaft und ihre Projekte zu unterstützen. Die Stiftung koordiniert darüber hinaus alle

⁵¹ Das Internet of Things (IoT) ist die Bezeichnung für das Netzwerk physischer Objekte („Things“), die mit Sensoren, Software und anderer Technologie ausgestattet sind, um diese mit anderen Geräten und Systemen über das Internet zu vernetzen, sodass zwischen den Objekten Daten ausgetauscht werden können. Diese Geräte reichen von normalen Haushaltsgegenständen bis hin zu anspruchsvollen Industriewerkzeugen. Experten erwarten, dass die Zahl der vernetzten IoT-Geräte bis 2025 auf 22 Mrd. ansteigen wird.

⁵² Diafanis ist griechisch für transparent. Gemeint sind hier übertragen transparente, d.h. nachvollziehbare mathematische Funktionen auch und insbesondere bei KI wie **paceval**, die ermöglicht.

komplementären Produkte und Dienstleistungen, die im internen Sprachgebrauch auch als das Ökosystem bezeichnet werden.

In den letzten Jahren hat sich das IoT zu einer der wichtigsten Technologien des 21. Jahrhunderts entwickelt. Nun, da wir Alltagsgegenstände wie Küchengeräte, Autos, Thermostate oder allgemein Sensorik über eingebettete Geräte mit dem Internet verbinden können, ist eine nahtlose Kommunikation zwischen Mitarbeitern, Prozessen und Objekten möglich.

Das Referenzdesign erfüllt alle Ansprüche, die an ein IoT Gerät gestellt werden:

- Verfügbarkeit für kostengünstige Sensortechnologie mit geringem Stromverbrauch
- Konnektivität über eine Vielzahl von möglichen Netzwerkprotokollen (bspw. Ethernet oder CAN)
- Interaktionen mit Cloud-Computing-Plattformen der Rechenzentren, um eine Skalierung durchzuführen, ohne diese tatsächlich verwalten zu müssen
- Künstliche Intelligenz (KI) und Analysen auf dem Gerät selbst (sog. Edge Computing⁵³)
- Langer Lebenszyklus des ZC706 SoC (formell bis mindestens 2035 von AMD)
- Anforderungen der Automobilindustrie AEC-Q100⁵⁴ sind erfüllt und getestet beim ZC706

Das Referenzdesign ermöglicht ebenfalls Anwendungen der industriellen IoT (IIoT)⁵⁵. In letzter Zeit haben viele Branchen Maschine-zu-Maschine-Kommunikation (M2M) eingesetzt, um eine drahtlose Automatisierung und Steuerung zu erreichen. Mit dem Aufkommen von Cloud-Technologien und weiteren ergänzenden Technologien wie KI und Analysen können viele Branchen ein neues Niveau bei der Automatisierung erreichen und so neue Umsatz- und Geschäftsmodelle schaffen. Das IIoT wird manchmal als die vierte Welle der industriellen Revolution oder als Industrie 4.0 bezeichnet.

Das Referenzdesign kann bei den folgenden häufigen Anwendungen für das IIoT genutzt werden:

- Intelligente Fertigung
- Vernetzte Wirtschaftsgüter und vorbeugende und vorausschauende Wartung
- Intelligente Stromnetze
- Intelligente Städte
- Vernetzte Logistik
- Intelligente digitale Lieferketten (sog. Supply Chains)

Eclipse Diafanis ist eine mathematische Engine als Dienst (Mathematical Engine as a Service). Dieser ist aktuell quasi eine 1:1 Kopie des bestehenden *paceval*-service. Die grundlegende Idee ist, dass die Eclipse-Open-Source-Gemeinschaft das Projekt akzeptiert und unterstützt, so dass dieses durch die Eclipse Foundation einen Multiplikator erfährt und somit auch das Referenzdesign bzw. die darauf in der Zukunft existierenden Produkte.

Die Lösung, die Eclipse Diafanis bietet, ist analog zu den vorher beschriebenen.

⁵³ Edge Computing bezeichnet im Gegensatz zum Cloud Computing die dezentrale Datenverarbeitung am Rand des Netzwerks, der sogenannten Edge, s. https://de.wikipedia.org/wiki/Automotive_Electronics_Council.

⁵⁴ AEC-Q100 ist ein Standard der Automobilindustrie, der die empfohlenen Anforderungen und Verfahren zur Qualifizierung neuer Produkte und größerer Änderungen festlegt, s. https://de.wikipedia.org/wiki/Automotive_Electronics_Council

⁵⁵ Das industrielle IoT (IIoT) bezieht sich auf die Anwendung der IoT-Technologie in industriellen Umgebungen, insbesondere in Bezug auf die Instrumentierung und Steuerung von Sensoren sowie auf Geräte, die Cloud-Technologien einsetzen.

Das Problem: Viele vernetzte Geräte oder eben sogenannte IoT/IloT-Umgebungen erfordern komplexe mathematische Berechnungen, um korrekt zu funktionieren oder Entscheidungen zu treffen. Das kann zum Beispiel eine selbstfliegende Drohne, die während des Fluges Objekte in aufgezeichneten Videos oder Bildern in Echtzeit lokal auswertet. Eine Lösung wäre einfach mit einer lokalen Integration des Referenzdesigns aus diesem Validierungsauftrag möglich (Abbildung 24 links).

Sollte ein IoT/IloT-Gerät nicht die Rechenleistung haben, um solche mathematischen Berechnungen selbst durchzuführen oder die Batteriekapazität nicht ausreichen. So kann der Eclipse Diafanis Dienst diesem ermöglichen komplexe mathematische Operationen aus der Ferne und sehr schnell durchzuführen, wodurch ein erhöhter Stromverbrauch des Geräts selbst vermieden wird (Abbildung 24 rechts).

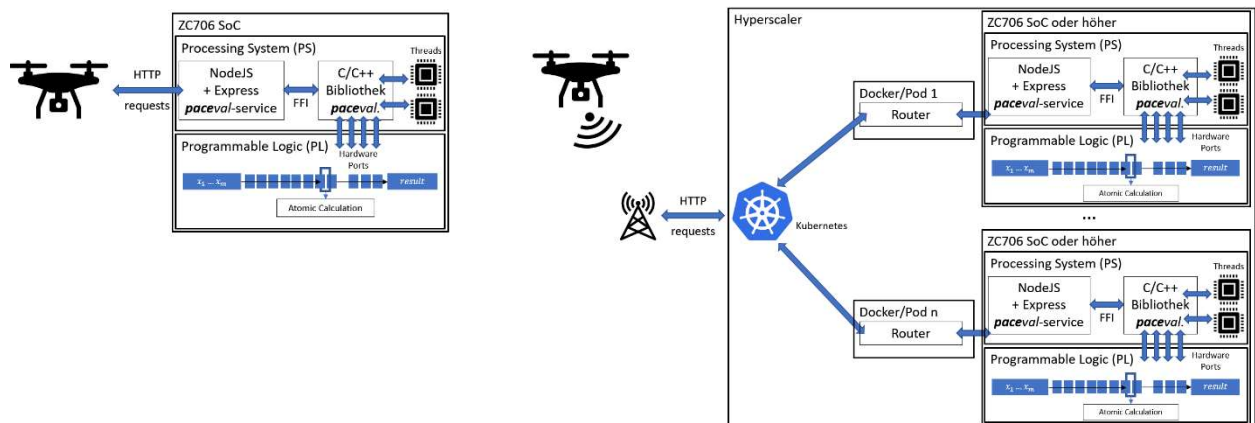


Abbildung 24: Das batteriebetriebene Gerät hier eine Drohne interagiert lokal mit dem Referenzdesign (links) oder über einen Hyperscaler (rechts)

Das Eclipse Diafanis™ Projekt findet sich auf <https://projects.eclipse.org/projects/iot.diafanis>.

Die API Dokumentation mit SwaggerHub findet sich unter <https://app.swaggerhub.com/apis/diafanis/diafanis-service/>.

Der Quellcode des Eclipse-Diafanis Dienstes ist in GitHub verfügbar unter <https://github.com/eclipse/diafanis>.

5. Abschnitt 1: Voraussetzung zur Durchführung der Validierung

Die notwendigen Voraussetzungen, die eine Durchführung der Validierung ermöglichen, wurden mit den Meilenstein 0 und Meilenstein 1 erreicht. Wie im Kapitel „6.2 Arbeiten seit dem Zwischenbericht“ dargelegt, wurde eine Beauftragung von OFFIS notwendig. Dies betrifft vor allem den Meilenstein 2, wie auch in Kapitel „6.2 Arbeiten seit dem Zwischenbericht“ erklärt:

- **Meilenstein 2 – Grundlegende mathematische Funktionen beschleunigen**

6. Abschnitt 2: Ergebnisse

6.1 Erzielte Ergebnisse bis zum Zwischenbericht

Arbeitspakete AP0 und AP1 wurden abgeschlossen. Dabei konnten die assoziierten **Meilensteine 0 und 1 vollumfänglich erreicht** werden.

Erreichte Meilensteine:

Meilenstein 0 – Projektstart

Arbeitspaket 0 (AP0): Kauf und Inbetriebnahme der für die Validierung benötigten Hardware (FPGAs + Test Server)

Notwendig ist hierfür auch eine Vor-Ort Unterstützung von OFFIS (Institut für Informatik) in Oldenburg.

Der Kauf und die Inbetriebnahme des ZC706 Evaluierungsboards und des APPLE Mac Studio als 64-bit Testserver verlief unproblematisch. Gleiches gilt auch für die zu verwendende Software und Werkzeuge (s. auch „6.3 Arbeitsbeschreibung AP0 „Kauf und Inbetriebnahme der für die Validierung benötigten Hardware“). Demzufolge hat auch keine Vor-Ort Unterstützung von OFFIS (Institut für Informatik) in Oldenburg stattgefunden.

Meilenstein 1 – Portierung von *paceval*. auf FPGA

Arbeitspaket 1 (AP1): Lauffähigkeit von *paceval*. auf den ARM Prozessoren auf dem FPGA
Parallelisierung sicherstellen und erste Messungen der Geschwindigkeit und Stromaufnahme bei Durchführung des MNIST Benchmark.

Das in Kapitel „3. Die eingesetzte Technologie“ beschriebene und zum Zwischenbericht umgesetzte vorläufige Referenzdesign verteilt die Last der Berechnungen gleichmäßig auf die vorhandenen ARM-Prozessoren. Die hierbei genutzte 32-bit Bibliothek von *paceval*. für ARM-Prozessoren verwendet intern eine leichtgewichtige Thread-API für C/C++, die gemeinsam vom IEEE⁵⁶ und der

⁵⁶ Institute of Electrical and Electronics Engineers (IEEE) ist ein 1963 gegründeter Berufsverband für Elektrotechnik und Elektrotechnik (und verwandte Disziplinen). Seine Ziele sind die pädagogische und technische Weiterentwicklung der Elektrotechnik, der Telekommunikation, der Computertechnik und ähnlicher Disziplinen.

Open Group für Unix⁵⁷ entwickelt und unter dem Namen Portable Operating System Interface (POSIX) standardisiert wurde (POSIX.1c, Threads extensions IEEE Std 1003.1c-1995)⁵⁸.

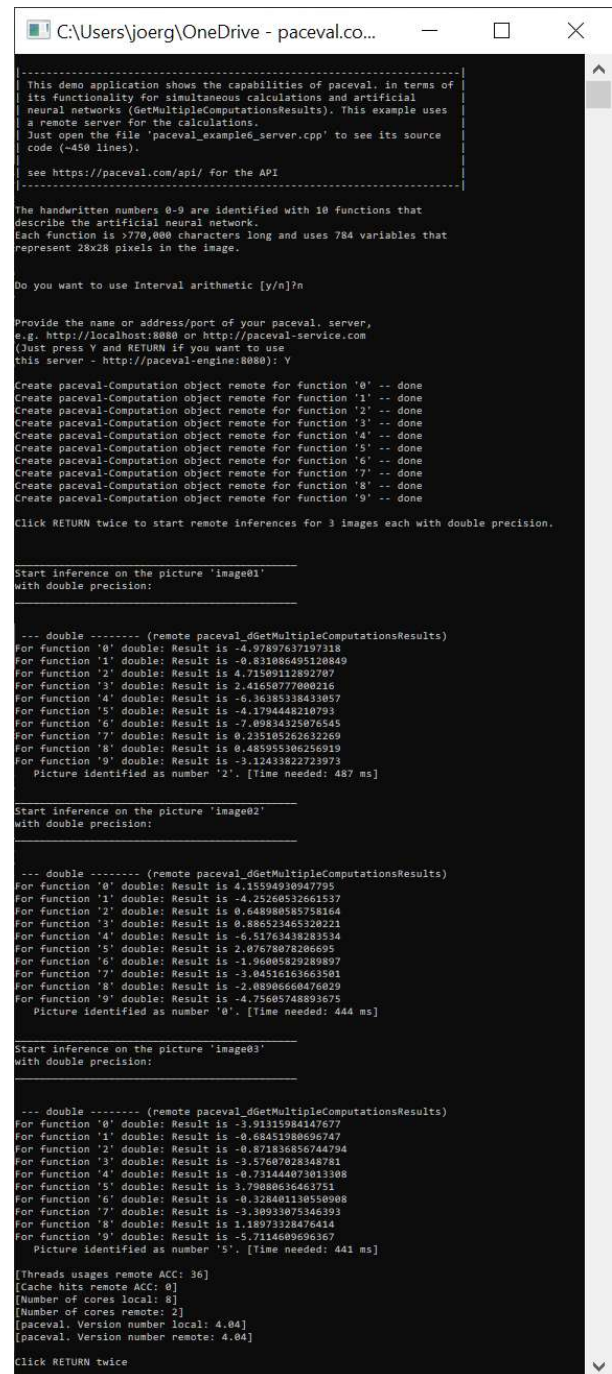
Diese Funktionen, wie sie in „3.3 Berechnungen mit der mathematischen Engine as a Service auf dem SoC“ beschrieben sind, profitieren von Threads:

- /GetComputationResult
- /GetComputationResultExt
- /GetMultipleComputationsResults
- /GetMultipleComputationsResultsExt

Durchgeführte Messungen des ZC706 Evaluierungsboards mit dem vorläufigen Referenzdesign (d.h. ohne Nutzung des FPGA) ergaben eine Leistungsaufnahme von maximal 8 Watt. Hierbei wurde das komplette ZC706 Evaluierungsboard gemessen, und nicht nur das ZC706 SoC. 8 Watt entsprechen über die Laufzeit von einem Jahr Gesamtenergiekosten von weniger als \$10. Dies wäre eine CO₂ Emission des vorläufigen Referenzdesigns von weniger als 1,8kg pro Jahr⁵⁹.

Nach Messungen, die sich in anderen Untersuchungen finden lassen, wäre insbesondere bei FPGA-Nutzung eine Leistungsaufnahme von weit weniger als 8 Watt mit dem ZC706 SoC alleine möglich. Dies deckt sich auch mit den Ergebnissen von OFFIS in Anhang C: „OFFIS-Machbarkeitsstudie für die Umsetzung der paceval-Software in FPGA-Hardware“.

Die auf dem PetaLinux des ZC706 erfolgten Zeitmessungen des MNIST Benchmarks⁶⁰ zeigten für eine Objekterkennung im Mittel circa 450ms (Abbildung 25). Allerdings kann die genutzte 32-bit Bibliothek von **paceval**. nicht die im Verarbeitungssystem (PS) integrierten zwei ARM Cortex-A9 MPCore-Anwendungsprozessoren des ZC706 SoC wirklich ausnutzen. Es werden zwar Threads erzeugt und verwendet, jedoch erfolgt



```
-----
This demo application shows the capabilities of paceval, in terms of
its functionality for simultaneous calculations and artificial
neural networks (getMultipleComputationsResults). This example uses
a remote server for the calculations.
Just open the file 'paceval_example6_server.cpp' to see its source
code (~450 lines).

see https://paceval.com/api/ for the API
-----

The handwritten numbers 0-9 are identified with 10 functions that
describe the artificial neural network.
Each function is ~770,000 characters long and uses 784 variables that
represent 28x28 pixels in the image.

Do you want to use Interval arithmetic [y/n]?n

Provide the name or address/port of your paceval. server,
e.g. http://localhost:8080 or https://paceval-service.com
(Just press y and RETURN if you want to use
this server - http://paceval-engine:8080): y

Create paceval-Computation object remote for function '0' -- done
Create paceval-Computation object remote for function '1' -- done
Create paceval-Computation object remote for function '2' -- done
Create paceval-Computation object remote for function '3' -- done
Create paceval-Computation object remote for function '4' -- done
Create paceval-Computation object remote for function '5' -- done
Create paceval-Computation object remote for function '6' -- done
Create paceval-Computation object remote for function '7' -- done
Create paceval-Computation object remote for function '8' -- done
Create paceval-Computation object remote for function '9' -- done

Click RETURN twice to start remote inferences for 3 images each with double precision.

Start inference on the picture 'image01'
with double precision:

--- double ----- (remote paceval_dGetMultipleComputationsResults)
For function '0' double: Result is -4.9789763719318
For function '1' double: Result is -0.831086495128849
For function '2' double: Result is 4.71509112892787
For function '3' double: Result is 2.41650777800216
For function '4' double: Result is -6.36385338433057
For function '5' double: Result is -4.1794448210793
For function '6' double: Result is -7.09834325076545
For function '7' double: Result is 0.23510832632269
For function '8' double: Result is 0.485955386256919
For function '9' double: Result is -3.12433822723973
Picture identified as number '2'. [Time needed: 487 ms]

Start inference on the picture 'image02'
with double precision:

--- double ----- (remote paceval_dGetMultipleComputationsResults)
For function '0' double: Result is 4.15594930947795
For function '1' double: Result is -4.25260532661537
For function '2' double: Result is 0.648989838738164
For function '3' double: Result is 0.886523455320221
For function '4' double: Result is -6.51763438283534
For function '5' double: Result is 2.07678078206695
For function '6' double: Result is -1.96080329289897
For function '7' double: Result is -3.0451613663581
For function '8' double: Result is -2.08906660476029
For function '9' double: Result is -4.75605748893675
Picture identified as number '0'. [Time needed: 444 ms]

Start inference on the picture 'image03'
with double precision:

--- double ----- (remote paceval_dGetMultipleComputationsResults)
For function '0' double: Result is -3.91315984147677
For function '1' double: Result is -0.68453980696747
For function '2' double: Result is -0.871836856744794
For function '3' double: Result is -3.57607028348781
For function '4' double: Result is -0.731444073013308
For function '5' double: Result is 3.7980836463751
For function '6' double: Result is -0.328401130550908
For function '7' double: Result is -3.38933075346393
For function '8' double: Result is 1.1897328476414
For function '9' double: Result is -5.7114609696367
Picture identified as number '5'. [Time needed: 441 ms]

[Threads usages remote ACC: 36]
[Cache hits remote ACC: 0]
[Number of cores local: 8]
[Number of cores remote: 2]
[paceval. Version number local: 4.04]
[paceval. Version number remote: 4.04]

Click RETURN twice
```

Abbildung 25: Ausführung des MNIST Benchmarks über den HTTP-Service auf dem ZC706 SoC

⁵⁷ Die Open Group ist ein 1996 gegründetes globales Konsortium, das sich zum Ziel gesetzt hat, das Erreichen von Geschäftszielen zu ermöglichen, indem es offene, anbieterneutrale Technologiestandards und Zertifizierungen entwickelt.

⁵⁸ Der Standard ermöglicht einem Programm, mehrere verschiedene Arbeitsabläufe zu steuern, die sich zeitlich überschneiden. Jeder Arbeitsablauf wird als Thread bezeichnet, und die Erstellung und Steuerung dieser Abläufe erfolgt durch Aufrufe der POSIX-Threads-API.

⁵⁹ Die dargestellten Energiekosten und CO₂ Emissionen sind abhängig vom Ort, dem Anbieter und dem gewählten Energiemix.

⁶⁰ Der MNIST Benchmark ist die Erkennung von handgeschriebenen Ziffern.

aufgrund der Grundlast auf dem Betriebssystem kein Caching, wodurch ein Großteil der Geschwindigkeit erzielt werden könnte.⁶¹

Und hier kommt die Idee ins Spiel, das FPGA für die Verarbeitung der verketteten Listen zu verwenden, dort auch die Parallelisierung durchzuführen und die ARM-Prozessoren lediglich für die Generierung der verketteten Listen und die Verwaltung der Berechnungsobjekte sowie der externen Schnittstellen zu verwenden. Dies wird in den nachfolgenden Abschnitten zu den „Arbeiten seit dem Zwischenbericht“ näher erklärt.

6.2 Arbeiten seit dem Zwischenbericht

Arbeitspakete AP2, AP3 und AP4 wurden abgeschlossen. Dabei konnten die assoziierten **Meilensteine 2, 3 und 4 vollumfänglich erreicht** werden.

Erreichte Meilensteine:

Meilenstein 2 – Grundlegende mathematische Funktionen beschleunigen

Arbeitspaket 2 (AP2): Klärung Fragen 1. und 2., s.o. „Unteraufgaben/-fragen“

Umsetzung von mathematischen Operationen und Teilfunktionen in Hardware auf dem FPGA (+/-/*/:exp).

Parallelisierung sicherstellen und zweite Messungen der Geschwindigkeit und Stromaufnahme bei Durchführung des MNIST Benchmark.

Als Bestandteile dieses Meilensteins werden die folgenden Unteraufgaben/-fragen beantwortet:

1. Welche Teile der *paceval*. Engine können in Hardware umgesetzt werden? Können beispielsweise Elementaroperationen wie +/-/*/:exp und parallele Ausführung von (Teil-)Funktionen umgesetzt werden?
2. Wie ist das Zusammenspiel zwischen der Hardware Implementierung und Komponenten, die weiterhin in Software vorhanden sind (z.B. mit Compiler)? Wie kann dies im Hinblick auf eine optionale Performance am besten gestaltet werden?

Im November 2022 haben hierzu erste Gespräche mit dem Institut für Informatik OFFIS vor Ort in Oldenburg stattgefunden. Am OFFIS werden neue Formen der computergestützten Informationsverarbeitung in Hard- und Softwaresystemen erforscht und Forschungsergebnisse in anwendungsnahe Entwicklungen umgesetzt. OFFIS wurde hier schon zu Beginn des Projektes von SPRIN-D als Entwicklungspartner vorgeschlagen.

Ziel des ersten und auch des zweiten Gespräches, das dann im Januar 2023 erfolgt ist, war, einen Plan zu entwickeln, wie OFFIS bei der Umsetzung von *paceval*. in Hardware unterstützen kann. Dies betrifft vorrangig Meilenstein 2⁶² – Grundlegende mathematische Funktionen beschleunigen. Vonnöten ist hierzu vor allem die Konfiguration des FPGA mit einer Hardwarebeschreibungssprache wie VHDL. Die so erzeugten Schaltungen des FPGA würden dann mittels Ports geeignet aus der Bibliothek von *paceval*. aufgerufen werden.

⁶¹ Es sei hier darauf hingewiesen, dass die 4 Anwendungsprozessoren des ZCU104 SoC genutzt werden, wie in Kapitel „6.7 Arbeitsbeschreibung AP4: Dokumentation und Beantwortung aller obenstehenden Fragen zum Validierungsauftrag“ beschrieben. Beim ZCU104 ergibt sich dadurch eine Geschwindigkeit bei der Ausführung von MNIST von 114ms.

⁶² Die Beauftragung von OFFIS betrifft potenziell auch „Parallelisierung verbessern“ des Meilenstein 3 – Optimierung der Funktionalität. Dies wäre noch im weiteren Verlauf des Validierungsprojektes zu klären.

Konkret hat OFFIS im März 2023 diese Schnittstellen von dem Team von **paceval**. erhalten (s. auch detailliert hierzu Anhang A: „paceval-Übergabe Quellcode an OFFIS“):

11. eine Compiler Projektumgebung, um selbstständig eine 32-bit **paceval**.-Bibliothek für ARM-Prozessoren für das ZC706 SoC zu erzeugen
12. eine Beschreibung der speziellen API im Quellcode, die mit der Hardwarebeschreibungssprache interagieren soll, insbesondere
 - a. API-Aufrufe, um benötigte Objekte wie die verkettete Liste pro Thread, die Liste der Variablen, die Liste der Ergebnisplatzhalter u.a. im Speicher für die programmierbare logische Schaltung (PL) zu erzeugen
 - b. den Quellcode für die Abarbeitung der Liste, welche von den Threads der ARM-Prozessoren aufgerufen wird
 - c. den Quellcode für die Operationen und Teilfunktionen

Basierend auf dieser Übergabe und Sichtung durch OFFIS wurde das Angebot „Machbarkeitsstudie für die Umsetzung der paceval-Software in FPGA-Hardware“ (Angebotsnummer 202303P047-00) von OFFIS an das Team von **paceval**. erstellt. Das Team von **paceval**. hat daraufhin mit Rücksprache mit SPRIN-D einen Antrag auf Änderung des Forschungs- und Entwicklungsvertrags eingereicht, der von SPRIN-D genehmigt wurde. Inhalt der Änderung war auch eine Verlängerung vorrangig für die Ergebnisse von OFFIS bis zum 31.7.2023. Die Ergebnisse von OFFIS finden sich in diesem Abschlussbericht in „6.5 Arbeitsbeschreibung AP2: Grundlegende mathematische Funktionen beschleunigen“.

Meilenstein 3 – Optimierung der Funktionalität

Arbeitspaket 3 (AP3): Klärung Fragen 3. und 4., s.o. „Unteraufgaben/-fragen“

Parallelisierung verbessern, Schnittstellen zum Hostsystem festlegen. System möglichst verallgemeinern, um die Benutzerfreundlichkeit für andere herzustellen.

Abschätzung wie andere neuronale Netze mit der Lösung arbeiten.

Die Optimierung der allgemeinen Funktionalität war ein fortlaufender Prozess. Es wurden verschiedenste Experten für Maschinelles Lernen, künstliche Intelligenz, Kubernetes, Parallelisierung und REST-APIs konsultiert. Insbesondere wurde auch eine überraschende Methodik entwickelt, um abzuschätzen, wie andere neuronale Netze mit der Lösung arbeiten. Die Idee, die bekannte Technologie der Programmiersprache Julia zu nutzen, um neuronale Netze zu erstellen, um sie in geschlossene, textuelle mathematische Ausdrücke zu exportieren, ist völlig neuartig (s. auch Anhang B: „Expertenpapier-Umwandlung von neuronalen Netzen in geschlossene Ausdrücke“).

Als Bestandteile dieses Meilensteins werden auch die folgenden Unteraufgaben/-fragen beantwortet:

3. Wie kann hinsichtlich der Ausführungsgeschwindigkeit die Schnittstelle zwischen Speicher (z.B. RAM, Caches) und der HW Implementierung implementiert werden?
4. Wie verhält sich eine HW Implementierung in einem Multi-User-Context in dem viele unterschiedliche Funktionen gleichzeitig ausgeführt werden müssen? Sind hier ähnliche Mechanismen wie sie heute z.B. in Standardprozessoren und Betriebssystemen benutzt werden notwendig?

Meilenstein 4 – Dokumentation und Beantwortung aller obenstehenden Fragen zum Validierungsauftrag

Arbeitspaket 4 (AP4): Klärung Fragen 5. und 6., s.o. „Unteraufgaben/-fragen“

Finale Messungen der Geschwindigkeit und Stromaufnahme bei Durchführung des MNIST Benchmark.

Der hier vorliegende Abschlussbericht zusammen mit Anhang C „OFFIS-Machbarkeitsstudie für die Umsetzung der paceval-Software in FPGA-Hardware“ umfasst alle Erkenntnisse zu dem Referenzdesign. Insbesondere konnten durch die Entwicklungsarbeiten und die Messungen von OFFIS zusätzliche und fehlende Informationen von Mai bis zum Projektende Juli 2023 beigefügt werden.

Als Bestandteile dieses Meilensteins werden auch die folgenden Unteraufgaben/-fragen beantwortet:

5. Welche Energieeinsparungen sind im Vergleich zu einer reinen Software Implementierung zu erwarten?
6. Welche Probleme konnten im Rahmen des Auftrags nicht gelöst werden und was sind mögliche Folgeaktivitäten für die weitere Entwicklung von *paceval*?

6.3 Arbeitsbeschreibung AP0 „Kauf und Inbetriebnahme der für die Validierung benötigten Hardware“

Das Evaluierungsboard ZC706 ließ sich unproblematisch über den Distributor Farnell bestellen. Allerdings dauerte die Lieferung länger als ursprünglich von Farnell angegeben. Zudem war das Evaluierungsboard ZC706 teurer als noch ursprünglich im August 2022 von Farnell angegeben. Die Lieferung erfolgte Ende Oktober 2022, so dass umgehend Anfang November mit der Inbetriebnahme begonnen werden konnte.

Die Inbetriebnahme des Evaluierungsboards ZC706 war ebenfalls unproblematisch. Sowohl zur Inbetriebnahme des Boards, den Werkzeugen als auch zum PetaLinux, existieren ausreichend beschreibende Dokumente und Foren. Allerdings ist es anfänglich schwierig, sich einen Überblick zu verschaffen und die richtigen Dokumente und Inhalte zu finden. Dies scheint allerdings üblich bei Hardwareinbetriebnahmen zu sein.

Die Bestellung des APPLE Mac Studio als 64-bit Test- und Demoserver war über APPLE selbst auch unproblematisch. Der Rechner konnte Mitte November vom APPLE Store in München abgeholt werden. Die Installation und Konfiguration von Parallels Desktop als Virtualisierungssoftware für den

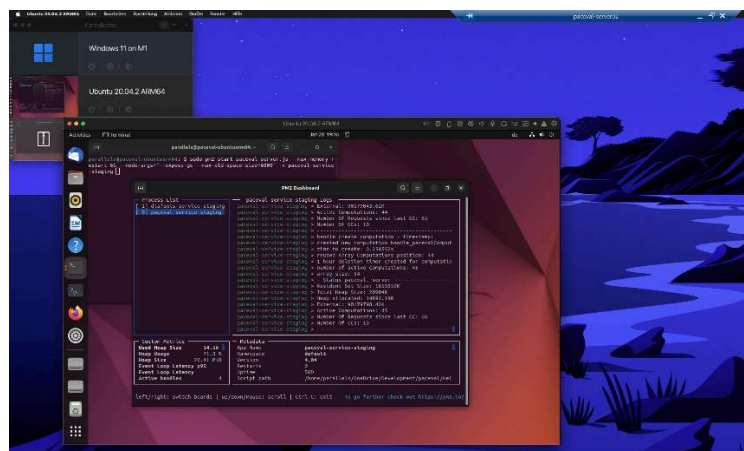


Abbildung 26: Die Oberfläche des APPLE Mac Studio Testservers mit dem *paceval-service* Dienst

paceval-service⁶³ (s. Kapitel „4.2 **paceval**-service und Docker Container“) war reibungslos. Der Server stellt **paceval**. als Dienst seit November 2023 fehlerfrei und kostenlos für jedermann über das Internet zur Verfügung (Abbildung 26).

6.4 Arbeitsbeschreibung AP1 „Lauffähigkeit von **paceval**. auf den ARM-Prozessoren auf dem FPGA“

In diesem Arbeitspaket wurde zunächst der Quellcode der systemunabhängigen Bibliothek von **paceval**. so umgestellt, dass elementare Teilbereiche externen Partnern zur Verfügung gestellt werden konnten, ohne den gesamten Quellcode offenlegen zu müssen. Die Bibliothek ist grundsätzlich per se als Closed Source zu betrachten. D.h. die nicht veröffentlichten Standards und die Behandlung des Quelltextes gilt als Betriebsgeheimnis. Im Gegensatz dazu sind alle Bestandteile, die um die Bibliothek herum bspw. zur möglichen Integration derselben auf kundenspezifischer Hardware oder für Rechenzentren existieren als Quelltext auf GitHub für jeden öffentlich verfügbar.

Diese Umstellungen der Bibliothek von **paceval**. betrafen vor allem

- die Erzeugung und Behandlung der Threads
- die Verarbeitung der verketteten Liste und der Teilfunktionen⁶⁴

Da das Augenmerk dieses Validierungsauftrags auf effektive Berechnungen vor allem im Umfeld von künstlicher Intelligenz liegt, wurden noch hierfür notwendige Operatoren hinzugefügt. Anwender der Bibliothek von **paceval**. haben diese üblicherweise selber über die API Funktion `paceval_SetCallbackUserFunction()`⁶⁵ mit Hilfe Ihres eigenen Quellcodes hinzugefügt. Da das FPGA als Produkt aber ein in sich geschlossenes System⁶⁶ darstellt, sind diese nun Bestandteil der vordefinierten Operatoren.

Folgende Operatoren wurden der Bibliothek von **paceval**. hinzugefügt:

- Minimum und Maximum
min, max
- Modulo⁶⁷ symmetrische und mathematische Variante
%, mod

Dieses Vorgehen hat auch den Vorteil, dass die geplante Umsetzung als Hardwarebeschreibungssprache auch dann diese Operatoren umfasst.

Zudem wurde die API der Bibliothek von **paceval**. noch um die folgende Funktion erweitert:

- `paceval_dGetMultipleComputationsResultsExt()`

Hiermit können in einem Aufruf mehrere Funktionen mit mehreren Werten für Variablen ausgewertet werden, s.u. Da das Referenzdesign dieses Validierungsauftrags eine Hardwareschnittstelle (Ethernet) mit Variablenwerten aufrufen wird, sollte der Datentransfer über

⁶³ SwaggerHub für **paceval**-service, s. <https://app.swaggerhub.com/apis-docs/paceval/paceval-service/4.04>

⁶⁴ diese Umstellung wird auch notwendig für die externe Beauftragung des OFFIS, s. „6.2 Arbeiten seit dem Zwischenbericht“

⁶⁵ Die API Dokumentation von **paceval**., s. <https://paceval.com/api/>

⁶⁶ Anmerkung: Das in sich abgeschlossene System ist notwendig, um sicherheitsrelevanten Manipulationen vorzubeugen.

⁶⁷ Modulo oder „Division mit Rest“, s. https://de.wikipedia.org/wiki/Division_mit_Rest

diese Schnittstelle möglichst klein in der Größe der Daten sein und auch selten stattfinden. Die neue Funktion `GetMultipleComputationsResultsExt()` komplettiert hier dann die bereits vorhandenen API Funktionen:

- `paceval_dGetComputationResult()`
Führt eine Berechnung auf **einem Berechnungsobjekt** mit den von `paceval_CreateComputation()` deklarierten Variablen und **mit einem Satz an Werten** für diese Variablen durch.
- `paceval_dGetComputationResultExt()`
Führt parallel mehrere Berechnungen auf **einem Berechnungsobjekt** mit den von `paceval_CreateComputation()` deklarierten Variablen und **mit mehreren Sätzen an Werten** für diese Variablen durch.
- `paceval_dGetMultipleComputationsResults()`
Führt parallel mehrere Berechnungen auf **mehreren Berechnungsobjekten** mit den von `paceval_CreateComputation()` deklarierten Variablen und **mit dem gleichen Satz an Werten** für diese Variablen durch.
- `paceval_dGetMultipleComputationsResultsExt()`
Führt parallel mehrere Berechnungen auf **mehreren Berechnungsobjekten** mit den von `paceval_CreateComputation()` deklarierten Variablen und **mit unterschiedlichen Sätzen an Werten** für diese Variablen durch.

Dann wurden alle im Abschnitt „3. Die eingesetzte Technologie“ dargestellten Funktionalitäten umgesetzt bzw. wenn möglich, an bereits existierende Lösungen angepasst. Zum Abschluss wurde alles noch mit den im Abschnitt „4. Begleitende Technologien der *paceval*. Entwicklung“ beschriebenen, bereits existierenden Funktionalitäten getestet.

6.5 Arbeitsbeschreibung AP2: Grundlegende mathematische Funktionen beschleunigen

In diesem Arbeitspaket wurden mit Hilfe der Beauftragung von OFFIS die folgenden Unteraufgaben/-fragen beantwortet:

1. Welche Teile der *paceval*. Engine können in Hardware umgesetzt werden? Können beispielsweise Elementaroperationen wie $+/-/*/:exp$ und parallele Ausführung von (Teil-) Funktionen umgesetzt werden?
2. Wie ist das Zusammenspiel zwischen der Hardware Implementierung und Komponenten, die weiterhin in Software vorhanden sind (z.B. mit Compiler)? Wie kann dies im Hinblick auf eine optionale Performance am besten gestaltet werden?

Die Ausführungen in Anhang C „OFFIS-Machbarkeitsstudie für die Umsetzung der *paceval*-Software in FPGA-Hardware“ beantworten auch die 1. Frage: „Welche Teile der *paceval*. Engine können in Hardware umgesetzt werden? Können beispielsweise Elementaroperationen wie $+/-/*/:exp$ und parallele Ausführung von (Teil-) Funktionen umgesetzt werden?“

Die verknüpfte Liste kann vollständig auf der Programmierbaren Logik (PL) des FPGA verarbeitet werden, wie in Abschnitt „3.4 Funktionsweise des Zielsystems“ beschrieben. Alle von der Bibliothek von **paceval**. unterstützten Operatoren, Unterfunktionen und Symbole können auch parallel implementiert und ausgeführt werden, wie in Abschnitt „3.3 Berechnungen mit der mathematischen Engine as a Service auf dem SoC“ beschrieben.

Ebenfalls wird die 2. Frage beantwortet: „Wie ist das Zusammenspiel zwischen der Hardware Implementierung und Komponenten, die weiterhin in Software vorhanden sind (z.B. mit Compiler)? Wie kann dies im Hinblick auf eine optionale Performance am besten gestaltet werden?“.

Nach der Untersuchung durch OFFIS ist es offensichtlich, dass die optimale Leistung erreicht wird, wenn die verkettete Liste vollständig auf der Programmierbaren Logik (PL) des FPGA verarbeitet wird. Das bedeutet, dass die eigentliche Berechnung des „Calculate“-Schritts immer wieder in der Programmierbaren Logik (PL) des FPGA erfolgt, was teilweise in Nanosekunden erfolgen kann.

Besonders komfortabel ist es auch, die einmalige Erstellung der verknüpften Liste mit der Bibliothek von **paceval**. im Verarbeitungssystem (PS) durchzuführen. Dieser Prozess des Erstellens der verknüpften Liste im Speicher und des Setzens der Markierungen für eine mögliche Parallelisierung ist im Quellcode aufwendig und komplex. Da die Softwarebibliothek in C und C++ geschrieben ist, lässt sie sich sehr schnell für alle gängigen Prozessorarchitekturen wie ARM32, ARM64 oder RISK-V kompilieren, die mit einem FPGA interagieren. Es können dann auch ohne Umwege die bestehenden Schnittstellen zur Bibliothek von **paceval**. genutzt werden, wie sie in „4. Begleitende Technologien der paceval. Entwicklung“ beschrieben sind.

6.6 Arbeitsbeschreibung AP3: Optimierung der Funktionalität

In diesem Arbeitspaket wurden in einem fortwährenden Prozess die folgenden Aspekte bearbeitet:

- Parallelisierung verbessern
- Schnittstellen zum Hostsystem festlegen und System möglichst verallgemeinern, um die Benutzerfreundlichkeit für andere herzustellen
- Abschätzung wie andere neuronale Netze mit der Lösung arbeiten

Auch wurden in diesem Arbeitspaket mit Hilfe der Beauftragung von OFFIS die folgenden Unteraufgaben/-fragen beantwortet:

3. Wie kann hinsichtlich der Ausführungsgeschwindigkeit die Schnittstelle zwischen Speicher (z.B. RAM, Caches) und der HW Implementierung implementiert werden?
4. Wie verhält sich eine HW Implementierung in einem Multi-User-Context in dem viele unterschiedliche Funktionen gleichzeitig ausgeführt werden müssen? Sind hier ähnliche Mechanismen wie sie heute z.B. in Standardprozessoren und Betriebssystemen benutzt werden notwendig?

Bezüglich der Verbesserung der Parallelisierung wurde durch die ersten Gespräche mit dem Institut für Informatik OFFIS schnell deutlich, dass möglichst viel Verarbeitung auf der Programmierbaren

Logik (PL) des FPGA durchgeführt werden muss. Das Advanced eXtensible Interface (AXI)⁶⁸ zwischen dem Verarbeitungssystem (PS) und der Programmierbaren Logik (PL) sollte möglichst wenig genutzt werden, da dieses die Verarbeitung deutlich verlangsamt. In der Bibliothek von *paceval* wurde deshalb die Funktion zur Sammelbearbeitung `/GetMultipleComputationsResultsExt` hinzugefügt. Diese API-Funktion führt parallel mehrere Berechnungen auf mehreren Remote-Berechnungsobjekten mit den von `/CreateComputation` deklarierten Variablen und mit unterschiedlichen Sätzen an Werten für diese Variablen. Die Details hierzu sind in Abschnitt „3.3 Berechnungen mit der mathematischen Engine as a Service auf dem SoC“.

Wie in Abschnitt „3.2 PetaLinux und die gewählten Softwarekomponenten auf dem SoC“ dargestellt, ist der auf dem Verarbeitungssystem (PS) bzw. auf dem PetaLinux laufende *paceval*-service ein HTTP-Service. Dieser wird wie üblich über GET und POST angesprochen. Die Schnittstellen für diesen Service sind genau dieselben, die für den bekannten *paceval*-service als Docker Container genutzt werden und die hier auf SwaggerHub dokumentiert sind unter <https://app.swaggerhub.com/apis-docs/paceval/paceval-service/>.

Der neu entwickelte Kubernetes Cluster, der auf der Google Cloud Platform zu Demonstrationszwecken arbeitet, verwendet bereits die API-Funktion `/GetMultipleComputationsResultsExt`. Die Beschreibung der Schnittstellen findet sich auf SwaggerHub unter <https://app.swaggerhub.com/apis/paceval/paceval-service-cloud/>.

Für neuronale Netze haben wir eine einfache Möglichkeit entwickelt, diese auf die übliche Weise zu generieren und sie dann in geschlossene Ausdrücke zu exportieren, wie sie von *paceval* verarbeitet werden können. Die Vorgehensweise ist in Anhang B „Expertenpapier-Umwandlung von neuronalen Netzen in geschlossene Ausdrücke“ beschrieben. Da somit neuronale Netze einfach aus vielen mathematischen Funktionen bestehen, die ebenfalls als Berechnungsobjekte von der Bibliothek von *paceval* erzeugt werden können, ist die vorgestellte Lösung dieses Referenzdesigns ebenfalls geeignet, alle anderen neuronale Netze zu verarbeiten sofern genügend Speicher für die verketteten Liste auf der Programmierbaren Logik (PL) des FPGA zur Verfügung steht.

Um eine Alternative zu dem ZC706 SoC zu haben, der ein 32-bit System darstellt, wurden zusammen mit OFFIS die Spezifikationen zu 64-bit Systeme mit FPGA durchgeschaut. Die geeignetste 64-bit FPGA-Technologie wäre der Zynq UltraScale+ MPSoC⁶⁹ XCZU7EV von AMD Xilinx. Der MPSoC XCZU7EV verfügt über ein Verarbeitungssystem (PS) mit Quad-Core ARM Cortex-A53 und einen Dual-Core ARM Cortex-R5 Echtzeitprozessor. Zudem hat der MPSoC XCZU7EV mit 4GB mehr Speicher für die Programmierbaren Logik (PL) als das ZC706 SoC mit 1GB, was für besonders große neuronale Netze nötig sein kann. Das Referenzdesign kann unproblematisch vom jetzigen ZC706 SoC auf einen UltraScale+ MPSoC für leistungsfähigere Produkte übertragen werden.

Damit kann auch die 3. Frage beantwortet werden: „Wie kann hinsichtlich der Ausführungsgeschwindigkeit die Schnittstelle zwischen Speicher (z.B. RAM, Caches) und der HW Implementierung implementiert werden?“

⁶⁸ AMBA AXI4 (Advanced eXtensible Interface 4), s. <https://docs.xilinx.com/r/en-US/ug1483-model-composer-sys-gen-user-guide/AXI-Interface>

⁶⁹ MPSoC steht hier für Multiprocessor System-on-a-Chip

Sammelbearbeitungen wie `/GetComputationResultExt`, `/GetComputationResultExt`, `/GetMultipleComputationsResults` und `/GetMultipleComputationsResultsExt` entlasten das Advanced eXtensible Interface (AXI) zwischen dem Verarbeitungssystem (PS) und der programmierbaren Logik (PL). Die Verarbeitungen der verketteten Listen erfolgt somit ausschließlich in der Programmierbaren Logik (PL) des FPGA.

Auch die 4. Frage erschließt sich dann aus der gewählten Methodik und Architektur: „Wie verhält sich eine HW Implementierung in einem Multi-User-Context in dem viele unterschiedliche Funktionen gleichzeitig ausgeführt werden müssen? Sind hier ähnliche Mechanismen wie sie heute z.B. in Standardprozessoren und Betriebssystemen benutzt werden notwendig?“.

Analog zu der allgemeinen Vorgehensweise der Bibliothek von *paceval*, wie in Abschnitt „3.3 Berechnungen mit der mathematischen Engine as a Service auf dem SoC“ beschrieben, wird beim Aufruf von `/CreateComputation` als Referenz ein Token zurückgegeben, das eindeutig das Berechnungsobjekt und somit auch die verkettete Liste darstellt, die in der Programmierbaren Logik (PL) des FPGA erzeugt wurde. Mit diesen Tokens können dann viele Berechnungen als Sammelbearbeitung gleichzeitig auf den Berechnungsobjekten über das Verarbeitungssystem (PS) in die programmierbare Logik (PL) des FPGA mit den verketteten Listen durchgeführt werden. Mit dieser Methodik sind somit keine weiteren Mechanismen von Standardprozessoren und Betriebssystemen auf dem FPGA notwendig, da diese im Verarbeitungssystem (PS) bspw. dem PetaLinux behandelt werden.

Berechnungsobjekte werden in der jetzigen Implementierung nach 1 Stunde automatisch gelöscht, wenn diese nicht benutzt werden und entsprechend dann der Speicher frei gegeben wie in Kapitel „3.3 Berechnungen mit der mathematischen Engine as a Service auf dem SoC“ beschrieben. Würde nun nicht mehr ausreichend Speicher für das Verarbeitungssystem (PS) oder für die programmierbare Logik (PL) zur Verfügung stehen für das Erzeugen eines neuen Berechnungsobjektes, so wird ein Fehler zurückgegeben. Dies ließe sich, wenn nötig, sehr einfach erweitern, indem man eine API-Funktion `/DeleteComputation` noch anbietet, die bei Übergabe einer gültigen Referenz auf ein existierendes Berechnungsobjekt dieses sofort aus dem Speicher löscht.⁷⁰

6.7 Arbeitsbeschreibung AP4: Dokumentation und Beantwortung aller obenstehenden Fragen zum Validierungsauftrag

Allgemein lässt sich sagen, dass das Referenzdesign ein voller Erfolg ist. Nach dem Bericht von OFFIS ist die Geschwindigkeit und die Stromaufnahme mit der Möglichkeit mehrfache Parallelität zu erreichen geeignet ein ASIC Design erfolgreich zu erstellen und auch als Produkt zu vermarkten.

Damit kann auch die 5. Frage beantwortet werden: „Welche Energieeinsparungen sind im Vergleich zu einer reinen Software Implementierung zu erwarten?“

Ein moderner Office-PC mit Mehrkernprozessor und normaler Grafikkarte leistet im Normalbetrieb beispielsweise rund 135 Watt. Würde man nun diesen Office-PC als Server für mathematische Berechnungen verwenden, so wäre die Energieeinsparung um mehr als den Faktor 18 wenn man

⁷⁰ Das Vorgehen mit `/CreateComputation` und `/DeleteComputation` für den HTTP-Service wäre auch analog zu der bestehenden API von *paceval*, die zu `paceval_CreateComputation()` auch ein `paceval_DeleteComputation()` anbietet, s. <https://paceval.com/api/>

stattdessen das Referenzdesign mit maximal 7,2 Watt nimmt (wobei hier wohlgermerkt das komplette Entwicklerboard von OFFIS gemessen wurde).

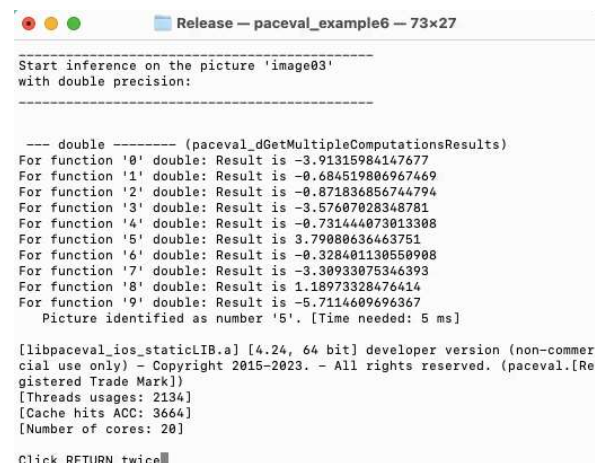
Aber es geht noch besser! Denn betrachtet man die geschätzte Messung der Geschwindigkeit und des Stromverbrauchs bei der Ausführung eines neuronalen Netzwerks MNIST LeNet zur Erkennung von einer handgeschriebenen Zahl, die auf einem Standard-Computersystem mit Grafikkarte maximal 2 Millisekunden⁷¹ bei doppelter Genauigkeit dauern würde, so ergibt sich folgender Vergleich:

	Standard 3.0 GHz CPU + NVIDIA GeForce GTX TITAN X	<i>paceval</i> . 4.24 auf CPU APPLE MAC STUDIO M1 20 Cores	<i>paceval</i> . 4.24 auf FPGA PS Zynq Ultrascale+ XCZU7EV 4 Cores	<i>paceval</i> . in Hardware/ FPGA PL Zynq Ultrascale+ XCZU7EV 96 FPU*s	<i>paceval</i> . in Hardware/ASIC 256 FPU*s (Werte abgeschätzt)
Stromaufnahme	400 Watt (CPU+GPU) ⁷²	68 Watt	3,4 Watt	3,4 Watt	< 3,4 Watt
Bearbeitungszeit MNIST LeNet	< 2 ms	5 ms	114 ms	< 75 ms	< 15 ms
Anschaffungskosten	< \$ 2.000	\$ 3.300	\$ 850	\$ 850	\$ 700 ⁷³ + Gewinnmarge
Laufende Energiekosten ⁷⁴	\$ 700/Jahr	\$ 120/Jahr	\$5/Jahr	\$5/Jahr	< \$5/Jahr
CO2 Emission	560 kg/Jahr	95 kg/Jahr	0,9 kg/Jahr	0,9 kg/Jahr	< 0,9 kg/Jahr
Kühlung	aktiv	aktiv	passiv	passiv	passiv
Gehäusegröße	> 8.000 cm ³	3.700 cm ³	< 50 cm ³	< 50 cm ³	< 35 cm ³

*Werte für FPGA PL interpoliert aus Anhang C: „OFFIS-Machbarkeitsstudie für die Umsetzung der *paceval*-Software in FPGA-Hardware“

paceval. für ARM64 auf dem APPLE Mac Studio nutzt nur die CPU und benötigt für MNIST LeNet 5 Millisekunden bei gemessenen 68 Watt (Abbildung 27). Dies ist sehr positiv im direkten Vergleich zur GPU, auch was die Effizienz angeht.

Mit der *paceval*.-eigenen Intervallarithmetik „Trusted interval computing“ (TINC) wird mit 75 Millisekunden aufgrund des höheren Rechenbedarfs erwartungsgemäß mehr Zeit benötigt (Abbildung 28). Trotzdem ist dies noch ein sehr respektabler Wert.



```

Release — paceval_example6 — 73x27

Start inference on the picture 'image03'
with double precision:

----- double ----- (paceval_dGetMultipleComputationsResults)
For function '0' double: Result is -3.91315984147677
For function '1' double: Result is -0.684519806967469
For function '2' double: Result is -0.871836856744794
For function '3' double: Result is -3.57607028348781
For function '4' double: Result is -0.731444073013308
For function '5' double: Result is 3.79080636463751
For function '6' double: Result is -0.328401130550908
For function '7' double: Result is -3.30933075346393
For function '8' double: Result is 1.18973328476414
For function '9' double: Result is -5.7114609696367
Picture identified as number '5'. [Time needed: 5 ms]

[libpaceval_ios_staticLIB.a] [4.24, 64 bit] developer version (non-commercial use only) - Copyright 2015-2023. - All rights reserved. (paceval.[Registered Trade Mark])
[Threads usages: 2134]
[Cache hits ACC: 3664]
[Number of cores: 20]

Click RETURN twice

```

Abbildung 27: Messung *paceval*. auf APPLE Mac Studio

⁷¹ Quelle: <https://arxiv.org/pdf/1709.01686.pdf>

⁷² Quelle: <https://www.pcgameshardware.de/Titan-X-Grafikkarte-265165/Tests/Test-Benchmark-Tuning-Overclocking-vs-1080-1206879/>

⁷³ Die Herstellungskosten für den ASIC sind nach bestem Wissen geschätzt.

⁷⁴ Die dargestellten Energiekosten und CO₂ Emissionen sind abhängig vom Ort, dem Anbieter und dem gewählten Energiemix.

```

-----
Start inference on the picture 'image03'
with double precision:
-----

--- double ----- (paceval_dGetMultipleComputationsResults)
For function '0' double: Result is -3.91315984147677 : Interval [-3.91315984148631; -3.91315984146724]
For function '1' double: Result is -0.684519806967469 : Interval [-0.684519806976312; -0.684519806958628]
For function '2' double: Result is -0.871836856744794 : Interval [-0.871836856750928; -0.871836856738658]
For function '3' double: Result is -3.57607028348781 : Interval [-3.5760702834949; -3.57607028348072]
For function '4' double: Result is -0.731444073013308 : Interval [-0.731444073019618; -0.731444073006996]
For function '5' double: Result is 3.79080636463751 : Interval [3.79080636462975; 3.79080636464528]
For function '6' double: Result is -0.328401130550908 : Interval [-0.328401130559065; -0.32840113054275]
For function '7' double: Result is -3.30933075346393 : Interval [-3.30933075347093; -3.30933075345694]
For function '8' double: Result is 1.18973328476414 : Interval [1.18973328476011; 1.18973328476816]
For function '9' double: Result is -5.7114609696367 : Interval [-5.71146096964449; -5.7114609696289]
Picture identified as number '5'. [Time needed: 75 ms]

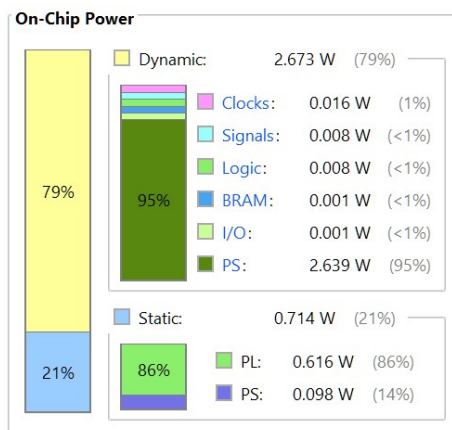
[libpaceval_ios_staticLIB.a] [4.24, 64 bit] developer version (non-commercial use only) - Copyright 2015-2023.
- All rights reserved. (paceval.[Registered Trade Mark])
[Threads usages: 2134]
[Cache hits ACC: 2529]
[Number of cores: 20]

Click RETURN twice

```

Abbildung 28: Messung paceval. mit „Trusted interval computing“ (TINC) auf APPLE Mac Studio

paceval. für ARM64 ausgeführt auf dem PetaLinux im Verarbeitungssystem (PS) auf dem AMD Xilinx ZCU104 mit XCZU7EV benötigt für MNIST LeNet 114 Millisekunden (Abbildung 29). Der Preis von \$850 wurde aus Anhang C: „OFFIS-Machbarkeitsstudie für die Umsetzung der paceval-Software in FPGA-Hardware“ Kapitel „4.2 Kosteneffizienz“ übernommen. Das Programm Xilinx Vivado gibt für das AMD Xilinx ZCU104 in der genutzten Konfiguration eine total On-Chip Power von 3,4 Watt an:



```

--- float ----- (paceval_fGetMultipleComputationsResults)
For function '0' float: Result is 4.15595
For function '1' float: Result is -4.252608
For function '2' float: Result is 0.6489806
For function '3' float: Result is 0.8865231
For function '4' float: Result is -6.517635
For function '5' float: Result is 2.076781
For function '6' float: Result is -1.960058
For function '7' float: Result is -3.045161
For function '8' float: Result is -2.089067
For function '9' float: Result is -4.750057
Picture identified as number '0'. [Time needed: 114 ns]

--- double ----- (paceval_dGetMultipleComputationsResults)
For function '0' double: Result is 4.15594930947795
For function '1' double: Result is -4.25260532661537
For function '2' double: Result is 0.648980585758164
For function '3' double: Result is 0.886523465320221
For function '4' double: Result is -6.51763498205334
For function '5' double: Result is 2.07678078206695
For function '6' double: Result is -1.96005829288997
For function '7' double: Result is -3.0451616363501
For function '8' double: Result is -2.08906660476029
For function '9' double: Result is -4.7505748893675
Picture identified as number '0'. [Time needed: 114 ns]

```

Abbildung 29: Messung paceval. auf AMD Xilinx ZCU104

Die folgenden Angaben wurden aus dem Bericht von OFFIS aus Anhang C: „OFFIS-Machbarkeitsstudie für die Umsetzung der paceval-Software in FPGA-Hardware“ interpoliert: Es wäre dann zu erwarten, dass eine Umsetzung von **paceval.** auf der Programmierbaren Logik (PL) maximal 75 Millisekunden für die Ausführung von MNIST LeNet benötigt. Dabei werden alle 96 Berechnungseinheiten verwendet. Dann wäre auch zu erwarten, dass eine Umsetzung als ASIC mit 256 Berechnungseinheiten mindestens 5-mal schneller ist und somit maximal 15 Millisekunden für die Ausführung von MNIST LeNet benötigt. Auch ist zu erwarten, dass der ASIC mit **paceval.** hierfür weitaus weniger als 3,4 Watt verbraucht.

Zu erwarten ist ebenfalls, dass ein Standard-Computersystem mit Grafikkarte anfangs immer noch schneller ist. Allerdings ist auch zu erwarten, dass für den ASIC je nach gewählter Technologie, z.B. Nanometer, das Processing/Watt und insbesondere das Processing/€ weitaus besser sein werden.

Dies führt unmittelbar zur 6. Frage: „Welche Probleme konnten im Rahmen des Auftrags nicht gelöst werden und was sind mögliche Folgeaktivitäten für die weitere Entwicklung von *paceval*.?“

Es gibt keine Probleme, die im Rahmen des Validierungsauftrags nicht gelöst wurden. Wir empfehlen daher, die Umsetzung des Referenzdesigns im 1. Schritt auf einem FPGA mit allen mathematischen Operatoren, Teilfunktionen und Symbolen. Und im 2. Schritt die Umsetzung als ASIC mit der geeigneten Technologie für die Anforderungen von IoT und IIoT.

6.8 Voraussichtlicher Nutzen insbesondere Verwertbarkeit des Ergebnisses

Für die Hyperscaler wird sicherlich auch das Referenzdesign dieses Validierungsauftrags in Kombination mit Kubernetes interessant sein. Denn bei der im Validierungsauftrag anvisierten signifikanten Verbesserung der Energieeffizienz bei künstlicher Intelligenz oder allgemein Mathematik durch optimierte Software und Hardware würde ein Stromeinsparungspotential bei den Hyperscalern bzw. Rechenzentren allgemein bestehen. Nach den Ergebnissen einer Studie⁷⁵ lag der Energieverbrauch von Rechenzentren in der EU im Jahr 2018 bei 76,8 TWh. Die Schätzungen ergeben einen Anstieg bis 2030 auf 98,52 TWh. Dies entspricht einem Anstieg von 28 %. Der weltweite Energieverbrauch der Rechenzentren liegt sogar um ein Vielfaches höher und wird bis 2030 noch erheblich steigen (Abbildung 28).

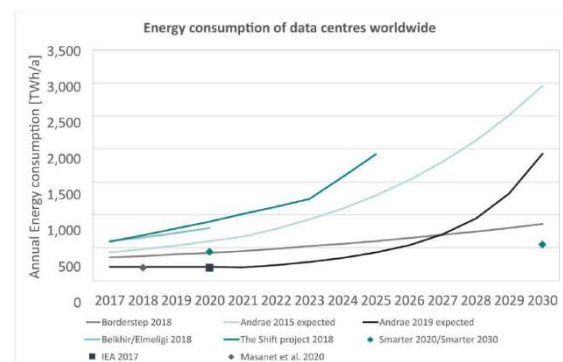


Abbildung 28: Weltweiter Energieverbrauch der Rechenzentren und der Netzwerkinfrastruktur

Die durch den Energieverbrauch bedingten Preistreiber für die Rechenzentren sind insbesondere der Strompreis, aber auch die noch steigenden Anforderungen, die CO₂ Emissionen zu verringern. Ein Server in einem Rechenzentrum hat eine Stromaufnahme von durchschnittlich mindestens 500 Watt. Dies ergibt laufende Energiekosten pro Server von mindestens \$850 pro Jahr, wobei die Gesamtkosten zumeist noch weitaus höher liegen werden. Die CO₂ Emission eines einzelnen Servers liegt dann auch bei mindestens 700kg pro Jahr. Wie gesagt sind dies alles Mindestwerte, da auch eine geeignete Kühlung, Luftzufuhr und andere Energieverbraucher und damit CO₂ Erzeuger hinzukommen.

Dies bedingt alles erhebliche Kosten bei den Betreibern der Rechenzentren und schlägt sich durch die Umlagen der Hyperscaler auf die Benutzer nieder. Es sollte also klar ein Wettbewerbsvorteil entstehen, wenn ein Hyperscaler das hier dargestellte Referenzdesign einsetzt.

⁷⁵ Quelle Energieverbrauch von Rechenzentren, s.

<https://www.bundestag.de/resource/blob/863850/423c11968fcb5c9995e9ef9090edf9e6/WD-8-070-21-pdf-data.pdf>

Ebenso das erklärte Ziel mit *paceval*. in Hardware die Grundlage für ein Produkt zu schaffen, das nicht nur neuronale Netze ausführen kann, sondern allgemein geschlossene mathematische Funktionen, sollte ein sehr großes Potential entfalten. Damit lassen sich dann nämlich alle finanztechnischen, stochastischen, ingenieur- und naturwissenschaftlichen Funktionen und darüber hinaus alle Modelle für maschinelles Lernen darstellen. Und dies auf einer energieeffizienten und universell einsetzbaren Plattform.

Vergleicht man allein die Vorteile von dem für das Referenzdesign gewählten ZC706 SoC mit FPGA gegenüber Grafikprozessoren (GPUs), so ergibt sich das folgende Bild (Abbildung 29 und Tabellen unten):

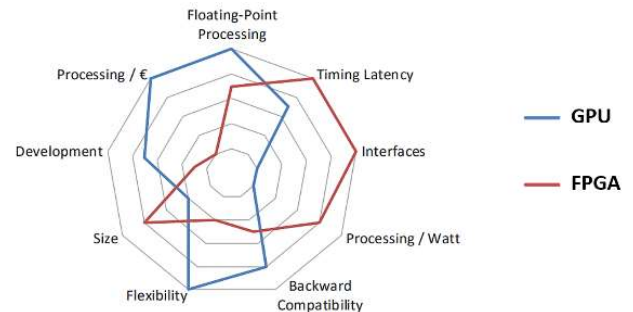


Abbildung 29: Quantitativer Vergleich der Funktionalitäten von GPUs und FPGAs

ZC706 SoC mit FPGA besser: Was wir behalten wollen (und können)

Größe	Der geringere Stromverbrauch des ZC706 SoC erfordert weniger Gegenmaßnahmen zur Wärmeableitung bei der Umsetzung eines Produktes in kleineren Dimensionen.
Interfaces	GPUs werden über PCI Express verbunden, während das ZC706 SoC die Flexibilität einer Verbindung mit jedem anderen Gerät über fast jede physische Standard- oder benutzerdefinierte Schnittstelle ermöglicht.
Leistungsfähigkeit per Watt	Bei der Messung von GFlops ⁷⁶ pro Watt ist ein FPGA um ein Vielfaches besser als eine GPU, mit einer anständigen Gesamtverarbeitungsleistung und einer hervorragenden Energieeffizienz
Timing-Latenz	Auf einem FPGA implementierte Algorithmen bieten ein deterministisches Timing in der Größenordnung von Nanosekunden, mit Latenzen, die um eine Größenordnung geringer sind als bei GPUs.

GPU besser: Was wir ändern wollen (und nach bisherigem Stand können) mit diesem Validierungsauftrag

Rückwärts Kompatibilität	Software, die für ältere GPUs entwickelt wurde, funktioniert in den neuen Geräten. FPGA VHDL kann auf neuere Plattformen verschoben werden, jedoch mit einigen Überarbeitungen.
Flexibilität	Dem FPGA mangelt es an Flexibilität, um die Hardwareimplementierung des synthetisierten Codes zu ändern, was für GPU-Entwickler kein Problem darstellt.
Development	Die wichtigsten Algorithmen der künstlichen Intelligenz werden direkt für GPUs entwickelt, und FPGA-Entwickler sind schwierig und teuer einzustellen.
Leistungsfähigkeit per Kosten	Geräte der Mittelklasse können in der gleichen Größenordnung verglichen werden, aber eine GPU gewinnt, wenn man nur den Kaufpreis pro GFlop betrachtet.

⁷⁶ Hinter der Abkürzung GFlop/s verbirgt sich eine Maßeinheit für die Geschwindigkeit von Mikroprozessoren pro Sekunde.

GPUs machen eine Sache wirklich gut: Operationen, die genau denselben Code auf einer großen Anzahl verschiedener Daten gleichzeitig ausführen. Das resultiert aus den Aufgaben, für die GPUs vorrangig gebaut sind. Die parallele Berechnung von Licht, Schatten und Farbe für Millionen verschiedener Pixel auf einem Bildschirm, wenn etwas gerendert wird. Es gibt jedoch tatsächlich nur sehr wenige trivial parallelisierbare Aufgaben außerhalb des Offensichtlichen (z.B. Grafik oder grundlegende Aufgaben der linearen Algebra für Musterlernen) bei denen GPUs mit ihrer „brute force“⁷⁷ parallelen Gleitkommaverarbeitung dominieren können. Zusammenfassend lässt sich sagen, dass GPUs für eine ziemlich spezifische Teilmenge von Problemen extrem leistungsfähig sind.

Warum können wenige CPUs trotzdem viel schneller sein als eine GPU? Der Hauptgrund besteht darin, dass man zwei Möglichkeiten hat, die Dinge schneller zu machen. Man kann Prozessoren schneller machen oder mehr Prozessoren hinzufügen. Eine GPU im sehr allgemeinen Sinne besteht aus vielen langsamen Prozessoren, während eine CPU aus einigen wenigen schnellen besteht.

Man stelle sich vor, man baue nun einen schmalen, hohen Turm. Dieser Turm besteht aus 10.000 Ziegeln. Der GPU-Weg würde darin bestehen, dass 10.000 ziemlich langsame, aber energiegeladene Bauarbeiter auf einmal auftauchen und der Turm sofort gebaut würde. Das funktioniert allerdings nicht. Denn in Wirklichkeit würden sie natürlich alle so energiegeladen in einer Warteschlange stecken, da viele Bauarbeiter warten müssen, bis die unterste Schicht gebaut ist, bevor sie Steine für die zweite platzieren. Für nahezu alle mathematischen Probleme ist es tatsächlich schneller, ein paar sehr schnelle energiegeladene Bauarbeiter zu verwenden. Dies ist der CPU-Weg. Wir übertragen nun diesen CPU-Weg auf das FPGA. Denn dort besteht grundsätzlich auch die Möglichkeit, Schaltungen zu parallelisieren.

Und fraglich ist auch, wer noch in Zukunft neuronale Netze über GPUs finanzieren mag, wenn nicht genügend und zumal teure Energie da ist zudem, wenn eine Alternative in Geschwindigkeit und Funktionalität existiert.

Zusammenfassend in einem Bild wollten wir mit diesem Validierungsauftrag das folgende Ziel (Target) schaffen, die Funktionalitäten signifikant zu verbessern und die Vorteile von GPU und FPGA in einem potenziellen mathematischen Produkt zu vereinen (Abbildung 30):

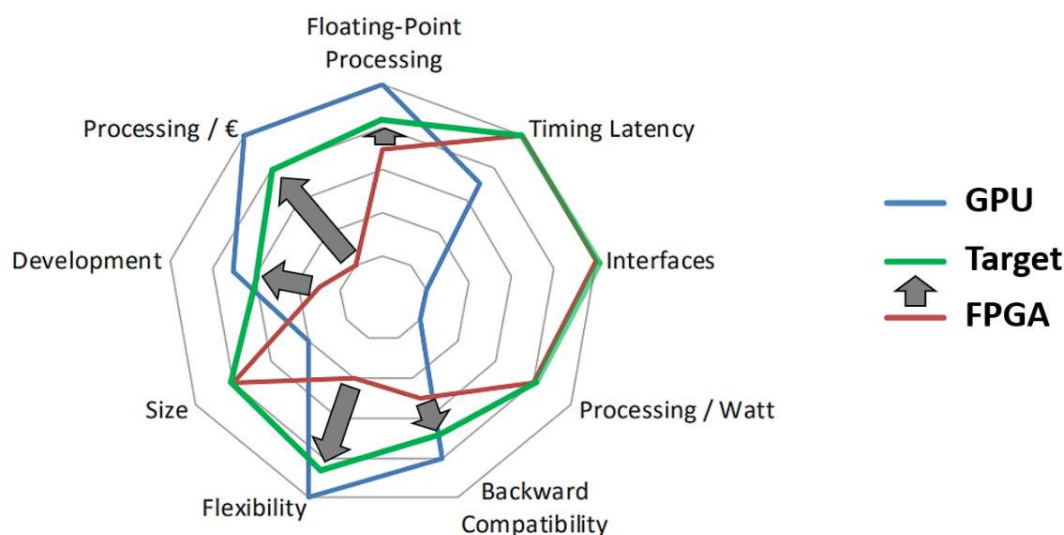


Abbildung 30: Quantitativer Vergleich der Funktionalitäten von GPUs, FPGAs und dem gesteckten Ziel dieses Validierungsauftrags.

⁷⁷ Die Brute-Force-Methode bzw. Methode der rohen Gewalt, auch Exhaustionsmethode, ist eine Lösungsmethode für Probleme aus den Bereichen Informatik, Kryptologie und Spieltheorie.

6.9 Während der Durchführung des Forschungs- und Entwicklungsauftrags der Auftragnehmerin bekannt gewordener Fortschritt auf dem Gebiet des Auftrags bei anderen Stellen

Nach bestem Wissen und Gewissen sind uns keinerlei ähnliche Projekte oder Fortschritte auf dem Gebiet des Validierungsauftrags bekannt. Es gibt bekanntermaßen spezialisierte Lösungen für konkrete mathematische Problemstellungen. Allerdings existiert kein universell einsetzbares System, wie es insbesondere das hier entwickelte Referenzdesign bietet:

- Transparente in textueller Form gegebene mathematische Funktionen
- Skalierbarkeit für kleine (z.B. IoT/IIoT und Sensorik) bis zu sehr großen Umgebungen (z.B. Hyperscaler) mit 32bit und 64bit FPGAs
- Einfach einzusetzende Intervallarithmetik für die Fehlerabschätzung

Denn im Allgemeinen sind heutige KI-basierte Entscheidungen eine sogenannte „Blackbox“. Die Definition einer Blackbox von Norbert Wiener, einem US-amerikanischen Mathematiker und Philosophen, der als Begründer der Kybernetik bekannt ist, lautet:

„Ich werde unter einer Blackbox einen Apparat verstehen, [...] der eine bestimmte Operation durchführt [...], für den wir jedoch nicht unbedingt Informationen über die Struktur haben, durch die diese Operation durchgeführt wird.“⁷⁸

Diese Blackbox für KI wird i.A. stillschweigend akzeptiert. Jedoch, wie Tommi Jaakkola, Professor für Elektrotechnik und Informatik am MIT, es kurz und knapp formuliert:

„Ob es sich um eine Investitionsentscheidung, eine medizinische Entscheidung oder vielleicht eine militärische Entscheidung handelt, Sie wollen sich nicht nur auf eine „Blackbox“-Methode verlassen.“⁷⁹

Jedoch, im Gegenteil, werden von Google und anderen Marktführern für Produkte im Bereich KI zusätzlich zu den Blackbox Verfahren noch Gleitkommazahlen mit niedriger Genauigkeit propagiert:

„Eine Verwendung von Gleitkommazahlen mit niedriger Genauigkeit ist eine gängige Methode, mit der sich die Zeit bis zur Konvergenz verkürzen kann, ohne dass die Genauigkeit beeinträchtigt wird.“⁸⁰

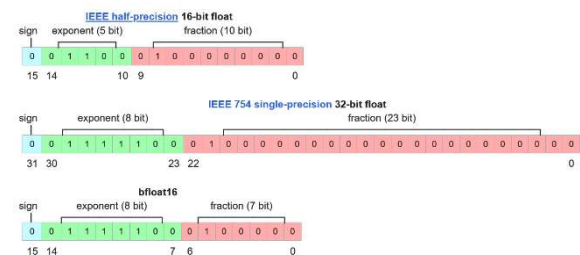


Abbildung 31: Vergleich bfloat16 mit 7bit Genauigkeit der Mantisse gegenüber 23bit bei IEEE 754 float

⁷⁸ Original: „I shall understand by a black box a piece of apparatus [...] which performs a definite operation [...], but for which we do not necessarily have any information of the structure by which this operation is performed.“, s.

https://uberty.org/wp-content/uploads/2015/07/Norbert_Wiener_Cybernetics.pdf

⁷⁹ Original: „Whether it’s an investment decision, a medical decision, or maybe a military decision, you don’t want to just rely on a ‚black box‘ method“, s. <https://www.technologyreview.com/s/604087/the-dark-secret-at-the-heart-of-ai/>

⁸⁰ „Google Cloud: Das numerische bfloat16-Format“, s. <https://cloud.google.com/tpu/docs/bfloat16?hl=de>

Die Argumentation ist verständlich, wenn man bedenkt, dass GPUs mit doppelter Präzision im Durchschnitt nur weniger als 20% so schnell sind wie mit einfacher Präzision.⁸¹ Den Beweis für die Allgemeingültigkeit, dass die Genauigkeit nicht beeinträchtigt wird, bleiben Google und andere schuldig. Es wäre sich hier vor Augen zu führen, was die Reduktion von 6-9 signifikanten Dezimalstellen bei IEEE 754 32-bit float hin zu 2-3 signifikanten Dezimalstellen bei bfloat16 für die Genauigkeit der Berechnungen und für die daran anschließende Bewertung oder Entscheidung in Gänze bedeutet (Abbildung 31).

Zum Vergleich: Schon alleine die verhältnismäßig einfache MNIST handgeschriebene Zahlenerkennung führt bei IEEE 754 32-bit float d.h. 6-9 Dezimalstellen Genauigkeit zu einem möglichen Fehler bei der dritten Dezimalstelle (Abbildung 32). Bei bfloat16 mit nur 2-3 signifikanten Dezimalstellen Genauigkeit wäre schon bei solch einem einfachen neuronalen Netz bei der ersten Dezimalstelle ein möglicher Fehler zu vermuten. Das ist fatal. Denn dadurch werden mit hoher Wahrscheinlichkeit Objekte ohne Hintergrundwissen potenziell häufig gänzlich falsch bewertet.

Dies würde naturgemäß insbesondere bei komplexeren Modellen, bei denen sich die Ungenauigkeiten mit den Rundungsfehlern der einzelnen Teilberechnungen noch weiter aufschaukeln, zu offensichtlich potenziell noch häufigeren Fehlinterpretationen bei der Objekterkennung führen. Diese dann existierende, latent vorhandene signifikante Fehlinterpretation wird noch verstärkt mit einer umgebenen Blackbox auf potenziell kritische Entscheidungen angewendet. Das Risiko für eine Gewährleistung ist somit immens und verbleibt allein beim Hersteller des finalen Produkts. Noch nicht einmal betrachtet wurde bei diesem Gedankengang, wie ein Produkt basierend auf einer solchen Methodik, wie sie heute als „Standard“ bezeichnet wird, überhaupt eine gegebenenfalls notwendige Zertifizierung beziehungsweise anschließende regulatorische Zulassung erhalten soll.

All diesem gegenüber bietet das in diesem Validierungsauftrag gewählte Referenzdesign klare Vorteile bis hin zu einem Pfad für eine mögliche regulatorische Zulassung eines darauf basierenden Produkts.



Abbildung 32: MNIST Erkennung durchgeführt mit *paceval*. und eingeschalteter Intervallarithmetik für float (23bit Mantisse), double (52bit Mantisse) und long double (≥63bit Mantisse) Genauigkeit

⁸¹ Quelle: https://de.wikipedia.org/wiki/General_Purpose_Computation_on_Graphics_Processing_Unit

6.10 Problematik der regulatorischen Zulassung eines Produktes mit KI und Verfahren zur Lösung

In diesem Kapitel wollen wir anschaulich das Problem der Zertifizierung von KI, bzw. genauer das Problem der regulatorischen Zulassung eines Produktes, das ein neuronales Netz beinhaltet, darstellen. Es ist hier gleich, ob es sich dabei um ein Softwareprodukt bspw. eine Personalsoftware, die aus tausenden Bewerbern auf eine Stelle den für die Aufgabenstellung besten auswählen soll oder um ein Produkt in Hardware handelt bspw. eine Komponente in einem Auto, die Verkehrsschilder erkennt. Die Problemstellung ist in beiden Fällen gleich. Denn neuronale Netze unterliegen bekanntermaßen Halluzinationen und Illusionen sowie der Abhängigkeit von den ausgewählten Trainings- und Testdaten bezüglich Ihrer Genauigkeit der Entscheidungen.

Bei der Halluzination handelt es sich um eine Wahrnehmung, für die es keine nachweisbare äußere Reizbasis gibt. Solche Wahrnehmungen können in jedem Sinnesbereich auftreten. Das bedeutet zum Beispiel, dass physisch nicht wahrnehmbare Gegenstände gesehen oder Stimmen gehört werden, ohne dass jemand spricht. Das Problem der Halluzination hat sicherlich schon jeder Nutzer von einem stimmenbasierten Produkt mit KI erlebt.

Bei einer Illusion hingegen wird eine tatsächlich vorhandene Realität als verändert wahrgenommen. Beispielsweise scheint sich ein tatsächlich vorhandenes, stationäres Objekt zu bewegen oder Gesichter scheinen in unregelmäßigen Mustern erkennbar zu sein (Abbildung 33).



Abbildung 33: Illusion von Muffins und Hundegesichtern

Üblicherweise lassen sich schnell neuronale Netze generieren, die auf Fragestellungen konstant bis zu einer gewissen Genauigkeit korrekte Antworten liefern. Nehmen wir zur Anschauung eine Genauigkeit von 80% an, die ein Produkt erreicht hat, wobei es natürlich je nach Problemstellungen bessere oder schlechtere

Ausgangsgenauigkeiten gibt (Abbildung 34). Zur Erhöhung der Genauigkeit werden

immer wieder neue Trainingsdatensätze hinzugefügt, das neuronale Netz erneut trainiert und gegen ausgewählte Trainingsdatensätze verprobt. Schwierig und langwierig wird es hingegen, eine höhere Genauigkeit zu erreichen. Die Industrie fordert üblicherweise Genauigkeiten von mehr als 95%.

Und es gibt noch ein zweites Problem: Die Genauigkeit definiert sich gegen die vorher ausgewählten Trainingsdaten. Wurden diese falsch gewählt bspw. waren bei der Personalsoftware nicht auch Daten von Minderheiten gegeben oder bei der Komponente zur Verkehrsschilderkennung nicht auch individuell verschmutzte Schilder, so wird sich die Genauigkeit potenziell rapide wieder nach unten verschieben.

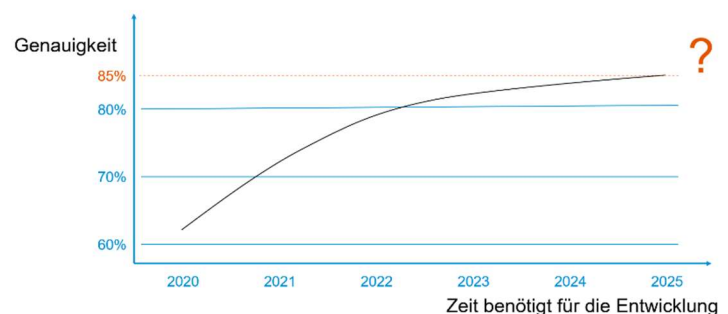


Abbildung 34: Standardansatz beim maschinellen Lernen

Fallen solche Fehlerinterpretationen durch nicht ausreichende Trainings- und Testdaten, Illusionen oder Halluzinationen bei einem Produkt auf, so muss nachgebessert werden. Diese Nachbesserung geschieht bei neuronalen Netzen dadurch, dass zusätzliche Trainingsdatensätze hinzugefügt werden und ein neues neuronales Netz erzeugt wird. „Neu“ ist dabei wörtlich zu verstehen, da sich das nun generierte neuronale Netz anders verhält in Bezug auf Illusionen oder Halluzinationen. Es kann deshalb auch nicht davon ausgegangen werden, dass Entscheidungen, die beim alten neuronalen Netz korrekt gefällt wurden, auch von dem neuen korrekt gefällt werden, solange diese nicht den Testdaten entsprechen. Und das ist naturgemäß üblicherweise der Fall, da man nicht für jede Eventualität einen Testdatensatz hat. Oder anders ausgedrückt, man kann nicht für jeden möglichen Bewerber und für jeden möglichen Schmutz auf einem Verkehrsschild im Vorhinein einen Testdatensatz erstellen. Die Überdeckung des Entscheidungsraums hat sich durch das erneute Lernen also signifikant geändert (Abbildung 35).



Abbildung 35: Umfang des erneuten Lernens

Bei einem neuronalen Netz kann man demnach nicht beantworten: Was hat es gelernt?

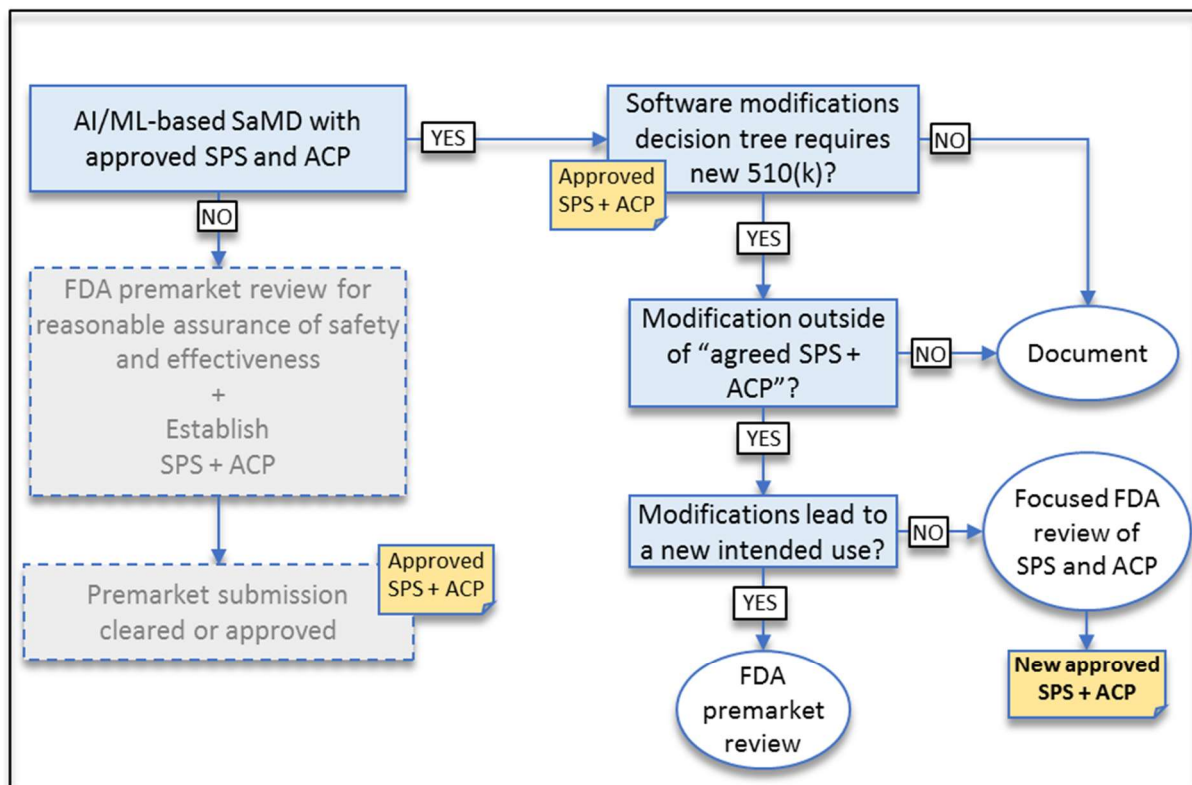
Und man kann auch nicht beantworten: Was hat es NICHT gelernt?

Das führt zwangsläufig zu einem Problem bei der regulatorischen Neuzulassung bei Änderungen.

Wenn das Produkt eine Blackbox ist und man der Zulassungsstelle nicht nachweislich die Verbesserung belegen kann, so wird eine komplette Neuzulassung des Produkts notwendig sein, da auch die potentiell negativen Auswirkungen entstanden durch das erneute Lernen nicht vollumfänglich erklärbar sind. Denn verständlicherweise ist es so, dass man nur eine Zulassung durch die jeweilige Behörde nach einer Änderung erwarten kann, wenn man die Änderungen in einem Dokument auch stichhaltig erklären kann.

Abbildung 36 zeigt hier stellvertretend für alle Behörden den Zulassungsprozess 510(k)⁸² der US-Behörde FDA für medizinische Geräte mit neuronalen Netzen. Obwohl das ursprüngliche künstliche neuronale Netzwerk bereits zertifiziert wurde, führt ein simples hinzufügen eines Trainingsdatensatzes dazu, dass das neu trainierte künstliche neuronale Netzwerk das Zertifizierungsverfahren erneut durchlaufen muss. Dies ist zeitaufwendig und kostspielig. Der Weg über das die Änderung erklärende Dokument ist offensichtlich zu empfehlen.

⁸² US-FDA Premarket Notification 510(k), s. <https://www.fda.gov/medical-devices/premarket-submissions-selecting-and-preparing-correct-submission/premarket-notification-510k>



Legend

Proposed regulatory pathway for new AI/ML-based SaMD

Proposed regulatory pathway for modifications for AI/ML-based SaMD

Endpoint for AI/ML modification

Abbildung 36: Vorgehensweise der US-FDA bei Änderungen an zuvor genehmigten medizinischen Geräten mit neuronalen Netzen

Wie kann man aber nun zu einem solchen erklärenden Dokument gelangen? Die Antwort liegt auf der Hand, wenn man die Änderung in einer menschenlesbaren Form vorliegen hat. Das wäre bei Software eines Produkts dann der Quelltext, wobei man den ursprünglichen Quelltext gegen den neuen Quelltext vergleichen kann und die Auswirkungen eingrenzen und erklären kann.

Bei neuronalen Netzen hat man üblicherweise keinen Quelltext. Allerdings besteht wie in diesem Bericht vorgestellt die Möglichkeit, das zertifizierte neuronale Netz in lesbare textuelle mathematische Ausdrücke zu exportieren. Dann werden nötige Änderungen durch geeignete mathematische Ausdrücke als Patches eingefügt. Der Patch bietet eine effiziente, transparente und zielgerichtete Lösung, die die Entscheidungen, die vom künstlichen neuronalen Netzwerk getroffen werden, effizient verbessert, ohne dass das neuronale Netzwerk neu trainiert werden muss.⁸³

Da der Patch mithilfe transparenter mathematischer Verfahren erfolgt ist und somit einer Behörde erklärt werden kann, ist eine vollständige Zertifizierung nicht mehr erforderlich. Da zudem die mathematischen Ausdrücke textuell sind, können die Patches mit den bei der Softwareentwicklung üblichen Werkzeugen zur Versionierung verwaltet werden (Abbildung 37).

⁸³ Für Beispiele, wie solche Patches aussehen, sei auf das Patent US20210279567 METHOD AND SYSTEM FOR ADJUSTING A MACHINE LEARNING OUTPUT von Koenning et al. verwiesen und auch auf die Beispiele für mathematische Ausdrücke in Kapitel „4.1 Die Bibliothek von paceval. und ihre Funktionen“

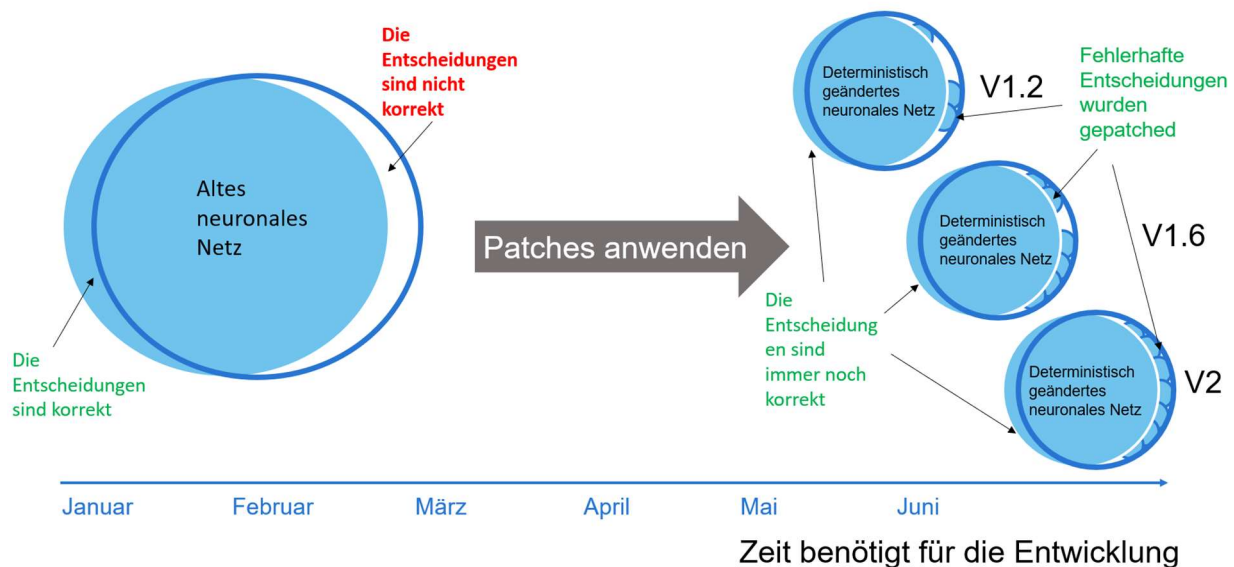


Abbildung 37: Kontinuierliche Optimierung des neuronalen Netzes mit Patches

Natürlich ist dies potentiell aufwendiger als einfach einen neuen Datensatz hinzuzufügen und das neuronale Netz einfach schnell neu zu erstellen. Allerdings hat man durch das vorgestellte Verfahren wenigstens die Wahl, ob man nun die Kosten für eine Korrektur bei der eigenen Entwicklung einsetzt und schnell eine Verbesserung ausliefern kann.

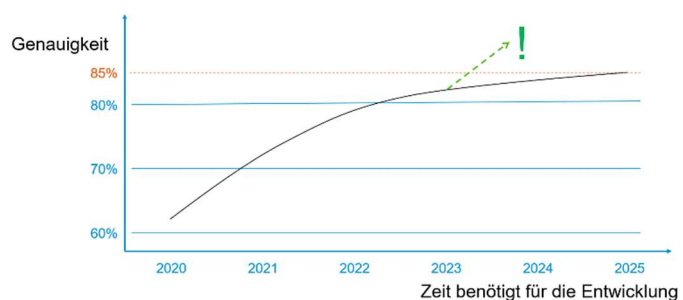


Abbildung 38: Schnellere Produktion und Veröffentlichungen durch Patches

Oder ob man die Kosten investiert, um das neue neuronale Netz erneut durch eine kostspielige und aufwendige Zertifizierung mit einer Behörde zu bringen. Zudem ist zu bezweifeln, dass man ein neuronales Netz auf anderem Wege bis zu der Qualität in der Genauigkeit bekommt, die in der Industrie gefordert wird. Denn eines ist klar: Softwareentwicklung, die industriellem Standard entspricht ist immer aufwendig in der Qualitätssicherung. Insbesondere sollten Entscheidungen, sofern sie die Gesundheit, Sicherheit, Ethik und Moral oder banal den Geldbeutel betreffen möglichst frei von Halluzinationen und Illusionen sein, bzw. es sollte einen Weg geben, die Korrektheit der Entscheidungen kontinuierlich und schnell zu verbessern (Abbildung 38).

Das beschriebene Verfahren ist auch konform zur Forderung der Organisation für wirtschaftliche Zusammenarbeit und Entwicklung (OECD)⁸⁴ und des Europäischen Parlaments⁸⁵:

- KI-Systeme brauchen **Transparenz und verantwortungsvolle Offenlegung**
- KI-Systeme müssen über **ihren gesamten Lebenszyklus robust, sicher und geschützt** funktionieren
- Organisationen und Einzelpersonen, die KI-Systeme entwickeln, einsetzen oder betreiben, **werden zur Verantwortung gezogen im Falle von falschen Entscheidungen**

⁸⁴ OECD AI Principles, s. <https://oecd.ai/en/ai-principles>

⁸⁵ European Parliamentary Research Service "A governance framework for algorithmic accountability and transparency", s. [https://www.europarl.europa.eu/RegData/etudes/STUD/2019/624262/EPRS_STU\(2019\)624262_EN.pdf](https://www.europarl.europa.eu/RegData/etudes/STUD/2019/624262/EPRS_STU(2019)624262_EN.pdf)

6.11 Erfolgte oder geplante Veröffentlichungen des Forschungs- und Entwicklungsergebnisses

Es ist geplant, die Ergebnisse mit und ohne Non-Disclosure Agreement (NDA) interessierten Unternehmen und Institutionen zur Verfügung zu stellen.

7. Abschnitt 3: Erfolgskontrollbericht

- Wissenschaftlich-technisches Ergebnis des Auftrags, erreichte Nebenergebnisse und gesammelte wesentliche Erfahrungen:
 - **Wissenschaftlich-technisch wurden alle Fragestellungen in diesem Auftrag erfüllt bzw. beantwortet. Zudem wurde als wesentliche Erfahrung eine komplette Werkzeugkette dargelegt:** Angefangen von dem neuartigen Export von neuronalen Netzen (Anhang B „Expertenpapier-Umwandlung von neuronalen Netzen in geschlossene Ausdrücke“), über die Realisierung als IoT/IIoT Formfaktor in Hardware (Anhang C „OFFIS-Machbarkeitsstudie für die Umsetzung der paceval-Software in FPGA-Hardware“), bis hin zur Bereitstellung einer Alternative zu bestehenden Lösungen mit GPUs als Serverlösung für die Hyperscaler (Abschnitt „4.3 Hyperscaler und Skalierung mit Kubernetes“ und Abschnitt „6.8 Voraussichtlicher Nutzen insbesondere Verwertbarkeit des Ergebnisses“).
- Erfindungen/Schutzrechtsanmeldungen und erteilte Schutzrechte, die von der Auftragnehmerin oder von am Auftrag Beteiligten gemacht oder in Anspruch genommen wurden:
 - **Nach bestem Wissen und Gewissen wurden in diesem Projekt zum Validierungsauftrag keine Schutzrechtsanmeldungen in Anspruch genommen.** Die Bibliothek von *paceval*. ist frei von Drittanbietersoftware. Gleiches gilt für die Nutzung der Programmier-/Scriptsprachen C/C++, Node.js, Julia, PHP, GoLang oder Python in diesem Projekt. Die genutzten Umgebungen von Google Cloud Platform, Kubernetes, AMD Xilinx Vivado oder PetaLinux wurden im Rahmen der bestehenden Lizenzen verwendet. Gleiches gilt für die Dienste von PetaLinux, die in Abschnitt „3.2 PetaLinux und die gewählten Softwarekomponenten auf dem SoC“ gelistet sind wie node, npm, PM2, Lighttpd, PHP, Samba, ntpd und swap.
 - In diesem Projekt zum Validierungsauftrag wurden keine Erfindungen/Schutzanmeldungen von dem Team von *paceval*. oder den am Auftrag Beteiligten gemacht. **Es gibt sicherlich vielfältige Möglichkeiten der Verwertung der in diesem Projekt entwickelten Methodiken durch Patente. Dies wäre von einem Patentanwalt zu prüfen.** Das Team von *paceval*. hat hier positive Erfahrungen mit
 - der Kanzlei Müller-Boré & Partner, Patentanwälte PartG mbB, Friedenheimer Brücke 21, 80639 München und
 - dem Patentanwalt Dr. Jochen Reich, Herrnstraße 15, 80539 München
- Wissenschaftliche und wirtschaftliche Anschlussfähigkeit für eine mögliche notwendige nächste Phase bzw. die nächsten innovatorischen Schritte:

- Wir möchten hier auf den ausführlichen Bericht des Instituts für Informatik OFFIS in Anhang C verweisen, insbesondere „Anhang C-6. Zusammenfassung“.
Zitat: „Damit ergibt sich für eine ASIC-Realisierung ein klares Potential, für komplexe mathematische Berechnungen konkurrenzfähig zu existierenden Beschleuniger-Produkten (beispielsweise Grafikkarten der namhaften Hersteller) zu sein.
Es ist dabei wichtig festzuhalten, dass selbst mit der aktuellen Architektur und Implementierung bereits konkurrenzfähige und bzw. überlegene Kennwerte erreicht werden konnten.“
- Somit wäre die wissenschaftliche und wirtschaftliche Anschlussfähigkeit, die konkrete Weiterführung der Entwicklung von OFFIS hin zu einem Prototypen mit FPGA bis zu einer Realisierung als ASIC. Dies sollte allerdings nicht unter der Leitung des Teams von *paceval*. erfolgen, da dieses Experten mit Erfahrung in Software und Cloud-Computing umfasst aber eben keine Experten der Hardwareentwicklung. **Das Team von *paceval*. legt deshalb nahe, das Projekt und die Inhalte an Murata (Japan), NXP (Niederlande) oder Infineon (Deutschland) zu übergeben. Diese Hersteller wären dann in der Lage, unterschiedlichste Systemlösungen für die Industriebereiche zu entwickeln und ggf. zu patentieren.**
- Das hat zudem Vorteile für den Hersteller eines Produktes basierend auf diesem Projekt des Validierungsauftrags. Geht man davon aus, dass ein ASIC mit dem neuartigen Coprozessor dieses Projekts im Jahr 2028 verfügbar sein könnte, und betrachtet man nun allein das Potential von künstlicher Intelligenz in Hardware im Jahr 2028, so ist dann das Marktpotenzial bei \$ 70.27 Milliarden (Abbildung 39). Je nach Studie ist im Jahr 2028 der Enterprise IoT Markt bei mehr als \$ 500 Milliarden. **Man kann dann davon ausgehen, dass ein neuartiger mathematischer Coprozessor einen internationalen Markt von mindestens \$ 8 Milliarden in IoT/IIoT adressieren kann. Wohlgedacht gehen alle Studien davon aus, dass kein europäischer Chiphersteller eine signifikante Rolle in 2028 bei künstlicher Intelligenz in Hardware einnehmen wird. Ein solcher ASIC wäre im Jahr 2028 somit ein sogenannter „Game Changer“, der auch europäischen Chipherstellern eine Chance bietet, erhebliche Gewinne mit einer lizenzfreien Hardware zu erzielen.**
- Die Umsetzung als ASIC würde der Softwareindustrie in Europa Planungssicherheit und Resilienz in der Lieferkette geben wie Anhang D „SAP-Empfehlungsschreiben“ darlegt.
Zitat: „Die begrenzte Effizienz der Software-Berechnungen verhindert derzeit jedoch noch einen breiteren Einsatz dieser Techniken (intelligenter Technologien wie Analytics und künstlicher Intelligenz). Hier sind die Fähigkeiten heutiger Prozessoren noch unzureichend da, insbesondere die Infrastrukturkosten, wie aktuell auch die gestiegenen Stromkosten einen erheblichen Kostenfaktor bei unseren Kunden bedeuten. Hardwarebeschleuniger sind aus unserer Sicht ein geeignetes Mittel, diesem Mangel entgegenzutreten. **Als potenzielle Nutzer derartiger Hardware würde uns der Open-Hardware-Ansatz zudem Planungssicherheit und Resilienz in der Lieferkette geben.“**
- Auch wenn die Umsetzung des „Trusted interval computing“ (TINC)⁸⁶ von *paceval*. nicht Bestandteil dieses Validierungsauftrags war, so sollte dies auch entsprechend in Hardware realisiert werden. Es lässt sich dann das Potential von falschen Entscheidungen aufgrund von Rundungsungenauigkeiten einschätzen, wie in Kapitel

⁸⁶ Trusted interval computing (TINC) ist beschrieben in Kapitel „4.1 Die Bibliothek von *paceval*. und ihre Funktionen“

„6.9 Während der Durchführung des Forschungs- und Entwicklungsauftrags der Auftragnehmerin bekannt gewordener Fortschritt auf dem Gebiet des Auftrags bei anderen Stellen“ dargestellt. Eine Implementierung von TINC in Hardware selbst stellt kein Problem dar.

- Es wäre auch sinnvoll, das Potenzial dieser Technologie für Quantencomputing zu klären. **Mit der Quantentechnologie ist es möglich, die oft iterative Berechnung mathematischer Teilergebnisse nochmals deutlich zu beschleunigen.**



ARTIFICIAL INTELLIGENCE IN HARDWARE MARKET SIZE, 2021 TO 2030 (USD BILLION)

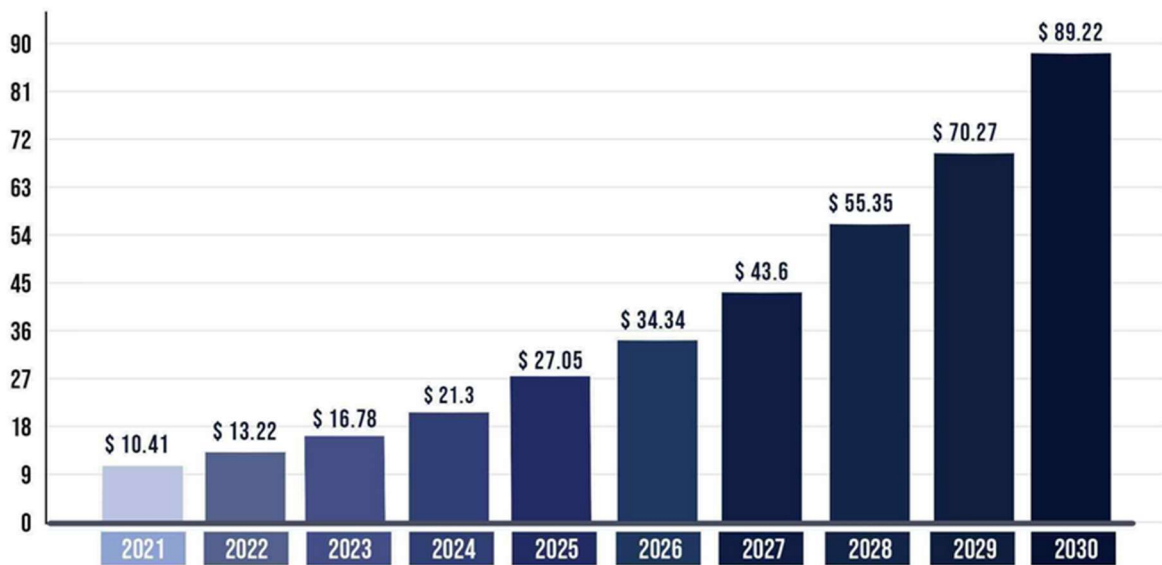


Abbildung 39: Erwartetes Marktpotential für KI-Hardware (Quelle: Precedence Research)

- Einhaltung der Kosten- und Zeitplanung:
 - Der Zeitplan wurde vollumfänglich eingehalten
 - Die Personalkosten wurden vollumfänglich eingehalten.

Anhang A: paceval-Übergabe Quellcode an OFFIS

An OFFIS wurde ein Referenzquellcode übergeben, der weitestgehend der Arbeitsweise der *paceval*. Bibliothek entspricht⁸⁷. Der Referenzquellcode ist so gestaltet, dass er zusätzlich zu der Standardberechnung der *paceval*. Bibliothek auch Berechnungen mit neuem Quellcode durchführt. Dieser neue Quellcode wurde extra für die Anpassungen von OFFIS erzeugt, entspricht dem ISO-Standard (ISO/IEC 14882:1998) und umfasst die folgenden Quellcodedateien:

- `paceval_fFPGAHandling.h` (circa 75 Zeilen Quellcode ohne Kommentare)
- `paceval_fFPGAHandling.cpp` (circa 450 Zeilen Quellcode⁸⁸ ohne Kommentare)

Dieser Quellcode wurde für die gemeinsamen und neuen Datenstrukturen erweitert:

- `paceval_cCommonData.h`

Arbeitsweise des neuen Quellcodes

Wir stellen die Arbeitsweise an einem einfachen Beispiel vor, bei dem auch alle Datenstrukturen Verwendung finden. Die C-Funktion `paceval_activityMain_FPGA_DoComputation(...)` ist hier die einzig wichtige Funktion für die Abarbeiten der mathematischen Berechnung. Alle anderen Funktionen des Referenzquellcodes werden zur Erzeugung und Verwaltung der benötigten Datenstrukturen verwendet. Nehmen wir als Beispiel diese Berechnung für die folgende Funktion:

$$f(x, y) = 5 * x + \exp(y) \quad \text{mit } x=2.2 \text{ und } y=3.3$$

Wenn die C-Funktion `paceval_activityMain_FPGA_DoComputation(...)` zur Berechnung aufgerufen wird, so verwenden wir diese vorgefüllte verkettete Liste `handle_nodeDataArray` für die durchzuführenden Teilberechnungen und dazu diese verkettete Liste `handle_nodeCacheStatusArray` für die Werte des Cache (wir benutzen hier aktuell nur die konstanten Werte bei `SET_TO_VALUE_SIGN`):

Knoten	0	1	2	3	4	5
handle_nodeDataArray						
eOperator	SET_TO_VALUE_SIGN	SET_TO_VARIABLE_SIGN	MULTIPLICATION_SIGN	SET_TO_VARIABLE_SIGN	EXPONENTIAL_FUNCTION	ADDITION_SIGN
#value field 1	-1 (nicht verwendet)	#0 (Wert Variable x)	#0	#1 (Wert Variable y)	#1	#0
#value field 2	-1 (nicht verwendet)		#1			
#result field	#0	#1	#0	#1	#1	#1
handle_nodeCacheStatusArray						
resultCache StatusFor Stack	CACHE_CALCULATION_ALWAYS_VALID					
resultCachedAs DoubleForStack	„5“ (als double)					
resultCachedAs FloatForStack	„5“ (als float)					

⁸⁷ Was in dem Referenzquellcode nicht enthalten ist, ist unter anderem die Unterstützung von long double, jedwede Caching-Funktionalität und Intervallarithmetik.

⁸⁸ Der neue Quellcode ist in C-Sprachsyntax. Er beinhaltet lediglich für die Lesbarkeit einige wenigen new- und delete-Operatoren von C++. Wir gehen davon aus, dass die Speicherverwaltung von OFFIS für das FPGA komplett neu angepasst wird.

Die Werte für die Variablen werden mit `variablesValues_in` an die C-Funktion `paceval_activityMain_FPGA_DoComputation(...)` übergeben und sind wie folgt:

variablesValues_in		
	Position	Wert (als double)
Wert Variable x	0	2.2
Wert Variable y	1	3.3

In `paceval_activityMain_FPGA_DoComputation(...)` wird die Berechnung für die Präzision double (`PACEVAL_CALCULATION_PRECISION_DOUBLE`) dann wie folgt abgearbeitet:

Schritt 1: Beim Knoten 0 ist der Operator `SET_TO_VALUE_SIGN`. Das bedeutet, dass ein konstanter Wert, der im Cache zum Knoten 0 gespeichert ist, als Operand in ein Feld von `handle_valueFieldArray` für eine Teilberechnung kopiert werden soll. Konkret wird der konstante Wert „5“ in das Feld 0 als Operand zur weiteren Verwendung kopiert.

handle_valueFieldArray		
	Position	Wert (als double)
	0	5 ← SET_TO_VALUE_SIGN
	1	0

Schritt 2: Beim Knoten 1 ist der Operator `SET_TO_VARIABLE_SIGN`. Das bedeutet, dass ein Variablenwert, der an die C-Funktion `paceval_activityMain_FPGA_DoComputation(...)` übergeben wurde, als Operand in ein Feld von `handle_valueFieldArray` für eine Teilberechnung kopiert werden soll. Konkret wird der Variablenwert „2.2“ für die Variable an der Stelle 0 (der Variablenbezeichner x) in das Feld 1 als Operand zur weiteren Verwendung kopiert.

handle_valueFieldArray		
	Position	Wert (als double)
	0	5
	1	2.2 ← SET_TO_VARIABLE_SIGN

Schritt 3: Beim Knoten 2 ist der Operator `MULTIPLICATION_SIGN`. Das bedeutet, dass zwei Operanden zur Multiplikation verwendet werden und das Ergebnis der Teilberechnung in ein Feld von `handle_valueFieldArray` geschrieben wird. Konkret wird das Feld 0 (s.o. Schritt 1 Wert „5“) als 1. Operand mit dem Feld 1 (s.o. Schritt 2 Wert „2.2“) als 2. Operand multipliziert und das Ergebnis „11“ in das Feld 0 hineinkopiert.

handle_valueFieldArray		
	Position	Wert (als double)
	0	11 ← MULTIPLICATION_SIGN
	1	2.2

Schritt 4: Beim Knoten 3 ist der Operator `SET_TO_VARIABLE_SIGN`. Analog zur Beschreibung in Schritt 2 wird hier konkret der Variablenwert „3.3“ für die Variable an der Stelle 1 (der Variablenbezeichner y) in das Feld 1 als Operand zur weiteren Verwendung kopiert.

handle_valueFieldArray		
	Position	Wert (als double)
	0	11
	1	3.3 ← SET_TO_VARIABLE_SIGN

Schritt 5: Beim Knoten 4 ist der Operator `EXPONENTIAL_FUNCTION`. Das bedeutet, dass ein Operand zur Berechnung der Exponentialfunktion verwendet wird und das Ergebnis der Teilberechnung in ein Feld geschrieben wird. Konkret wird das Feld 1 (s.o. Schritt 4 Wert „3.3“) als Operand für die Exponentialfunktion genommen und das Ergebnis „27.1126389206579“ in das Feld 1 hineinkopiert.

handle_valueFieldArray		
	Position	Wert (als double)
	0	11
	1	27.1126389206579 ← EXPONENTIAL_FUNCTION

Schritt 6: Beim Knoten 5 ist der Operator `ADDITION_SIGN`. Analog zur Beschreibung in Schritt 3 wird hier konkret das Feld 0 (s.o. Schritt 3 Wert „11“) als 1. Operand mit dem Feld 1 (s.o. Schritt 5 Wert „27.1126389206579“) als 2. Operand addiert und das Ergebnis „38.1126389206579“ in das Feld 0 hineinkopiert.

handle_valueFieldArray	
	Wert (als double)
Position	
0	38.1126389206579 ← ADDITION_SIGN
1	27.1126389206579

Abschluss: Es gibt keinen weiteren Knoten abzuarbeiten und das Ergebnis der Berechnung steht im letzten Feld, in das geschrieben wurde. Konkret ist dies Feld 0 mit dem Ergebnis „38.1126389206579“.

Die Abarbeitung über die Teilfunktionen läuft immer nach diesem Schema ab für jede beliebige mathematische Funktion.

Die Aufrufsequenz der C-Funktionen für die FPGA-Berechnungen und die wichtigsten Werte-/Variablenparameter

Die *paceval*. Bibliothek ruft in dieser Sequenz die C-Funktionen in `paceval_fFPGAHandling.cpp` auf.

1. `paceval_activityStart_FPGA_CreateComputation(...)` – 1-mal
: Diese C-Funktion wird aufgerufen, wenn ein *paceval*. Computation-Objekt⁸⁹ über die *paceval*. Bibliothek erzeugt wird.
2. `paceval_activityEnd_FPGA_CreateComputation(...)` – 1-mal
: Diese C-Funktion wird aufgerufen, wenn ein *paceval*. Computation-Objekt über die *paceval*. Bibliothek erzeugt wurde.
3. `paceval_activityStart_FPGA_GetComputationResult(...)` – n-mal
: Diese C-Funktion wird aufgerufen, wenn ein *paceval*. Computation-Objekt eine Berechnung durchführen möchte.
4. `paceval_activityMain_FPGA_DoComputation(...)` – n-mal
: Diese C-Funktion ist verantwortlich für das Abarbeiten der mathematischen Berechnung.
5. `paceval_activityMain_FPGA_ResetCachedNodeData(...)` – wenn nötig
: Diese C-Funktion ist verantwortlich im Falle eines Abbruchs oder Fehlers in `paceval_activityMain_FPGA_DoComputation(...)` den ungültigen Cache zurückzusetzen. Dies betrifft nicht den Cache für konstante Werte (s. `SET_TO_VALUE_SIGN`).
6. `paceval_activityEnd_FPGA_GetComputationResult(...)` – n-mal
Diese C-Funktion wird am Ende der Standardberechnung und der Berechnung über das FPGA aufgerufen. Es können dann die Ergebnisse verglichen werden.
7. `paceval_activityStart_FPGA_DeleteComputation(...)` – 1-mal
: Diese C-Funktion wird aufgerufen, wenn ein *paceval*. Computation-Objekt über die

⁸⁹ Ein Computation-Objekt ist das von der Bibliothek von *paceval*. verwaltete Berechnungsobjekt.

paceval. Bibliothek gelöscht wird. Dann ist insbesondere aller allozierter Speicher wieder freizugeben.

Im Folgenden aufgelistet sind die wichtigsten Werte-/Variablenparameter der C-Funktionen in `paceval_fFPGAHandling.cpp` nach der Reihenfolge Ihrer Verwendung:

const PACEVAL_HANDLE handle_pacevalComputation_in

: der Zeiger auf das *paceval*.Computation-Objekt, das in der Bibliothek von *paceval*. verwaltet wird

const unsigned long positionNode_in

: die Position in der verketteten Liste der Knoten, die gelesen werden soll

long* maxValueFields_out

: die maximale Feldnummer, die in dem Knoten verwendet wird

paceval_sFPGANodeData* handle_nodeDataStruct_out

: die Datenstruktur, die zum Holen und Halten der Knoteninhalte dient
(s. `paceval_cCommonData.h`)

paceval_sFPGANodeCacheStatusForStacks*

handle_nodeDataCacheStatusStructs_out

: die Datenstruktur, die zum Holen und Halten der Cacheinhalte dient
(s. `paceval_cCommonData.h`)

void* pointerToFPGAMemory_in

: die Adresse auf den Speicherstelle, die sich gemerkt werden soll am *paceval*. Computation-Objekt, das in der *paceval*. Bibliothek verwaltet wird

const unsigned long functionLengthString_in

: die Länge der Funktion, die bearbeitet wird über die *paceval*. Bibliothek

const unsigned long numberOfVariables_in

: die Anzahl der Variablen der Funktion

const bool useInterval_in

: soll Intervall-Arithmetik verwendet werden (true) oder nicht (false)

const bool success_in

: war das Erzeugen des *paceval*. Computation-Objekts über die *paceval*. Bibliothek erfolgreich (true) oder nicht (false)

const unsigned long numberOfNodes_in

: die Anzahl der Knoten des *paceval*. Computation-Objekts

const unsigned long numberOfStacks_in

: die Anzahl der maximal benötigten Stacks für die Berechnungen

const paceval_eCalculationPrecisionTypes useCalculationPrecision_in

: die Präzision (float oder double), mit der die Berechnung durchgeführt werden soll

const unsigned long stackNumber_in

: der Stack, auf dem die Berechnung durchgeführt werden soll

const unsigned char computationCircle_in

: der aktuelle Berechnungszyklus 1 oder 2 (wird zukünftig im Verlauf des Projekts für die Gültigkeit des Teilergebnisses im Cache benötigt)

const void* variablesValues_in

: der Zeiger auf die Werte für die Variablen (float oder double je nach useCalculationPrecision_in)

double* resultAsDouble_out

: der Platzhalter für das Ergebnis der Berechnung als double wenn alles geklappt hat

float* resultAsFloat_out

: der Platzhalter für das Ergebnis der Berechnung als float wenn alles geklappt hat

paceval_eErrorTypes* errorType_out

: im Fehlerfall der Typ des aufgetretenen Fehlers (s. paceval_main.h)

unsigned long* errorNodePosition_out

: im Fehlerfall die Position des aufgetretenen Fehlers (wird zukünftig im Verlauf des Projekts für die Meldung an den Benutzer durch die *paceval*. Bibliothek verwendet)

const bool validResultStandard_in

: gibt an, ob das Ergebnis der Standardberechnung gültig ist

const void* resultStandard_in

: das Ergebnis der Standardberechnung (float oder double je nach useCalculationPrecision_in)

const bool validResultFPGA_in

: gibt an, ob das Ergebnis der Berechnung über das FPGA (s. insbesondere paceval_activityMain_FPGA_DoComputation(...)) gültig ist

const void* resultFPGA_in

: das Ergebnis der Berechnung über das FPGA (float oder double je nach useCalculationPrecision_in)

Die Erzeugung und Verwaltung der benötigten Datenstrukturen

Wie oben dargestellt können, die verketteten Listen `handle_nodedataArray` (für die durchzuführenden Teilberechnungen) und `handle_nodeCacheStatusArray` (für die Werte des Cache) hier beliebig lang werden. Die Anzahl der benötigten Felder bzw. Zwischenspeicher in `handle_valueFieldArray` für die Operanden und Ergebnisse kann als Tiefe der Klammerebenen bzw. Verschachtelung der Funktion verstanden werden.

Wichtig ist das Verständnis, dass `handle_nodeCacheStatusArray` sowie `handle_valueFieldArray` mit Stacks arbeiten müssen. Je nach Aufgabe werden nämlich ausgiebig parallele Berechnungen durch Threads gestartet.⁹⁰

Alle diese Listen werden in der C-Funktion

`paceval_activityEnd_FPGA_CreateComputation(...)` angelegt und über

⁹⁰ An dieser Stelle sei auf „3.3 Berechnungen mit der mathematischen Engine as a Service auf dem SoC“ verwiesen und die Funktionen, die ausgiebig mit Threads arbeiten (s. `/GetComputationResult`, `/GetComputationResultExt`, `/GetMultipleComputationsResults` und `/GetMultipleComputationsResultsExt`).

`paceval_cComputation_setFPGAMemoryAllocation(...)` am zugehörigen Berechnungsobjekt (`handle_pacevalComputation`) der **paceval**. Bibliothek sich gemerkt. Die Allokation des Speichers im Referenz Quellcode wird sicherlich von OFFIS für den FPGA zugänglichen Speicher zu ändern sein. Analog gilt dies für die Freigabe des Speichers in der C-Funktion `paceval_activityStart_FPGA_DeleteComputation(...)`.

Im Falle eines Berechnungsfehlers in

`paceval_activityMain_FPGA_DoComputation(...)` muss der Cache zurückgesetzt werden. Dafür ist die Funktion `paceval_activityMain_FPGA_ResetCachedNodeData(...)` verantwortlich.

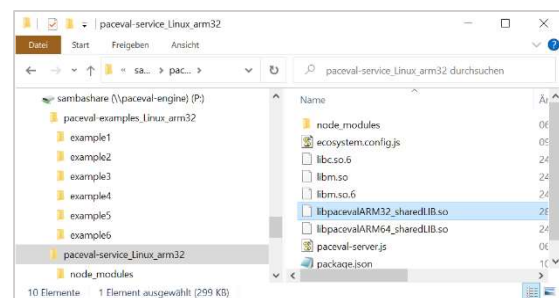
Da idealerweise die Standardberechnung der **paceval**. Bibliothek und die Berechnung über das FPGA zu denselben Ergebnissen führen sollen, können diese Ergebnisse mittels der C-Funktion `paceval_activityEnd_FPGA_GetComputationResult(...)` verglichen werden.

Verwendung im Validierungsprojekt

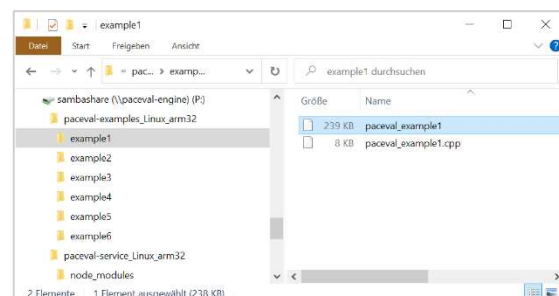
Mit dem Quellcode und den im folgenden bereitgestellten Eclipse IDE Projektumgebungen kann OFFIS selbst eine **paceval**. Bibliothek und das `paceval_example1` sowie `paceval_example6` zur Verwendung auf dem Referenzdesign erzeugen.

Die jeweils erzeugte eigene **paceval**. Bibliothek kann direkt im Referenzdesign verwendet werden, indem man

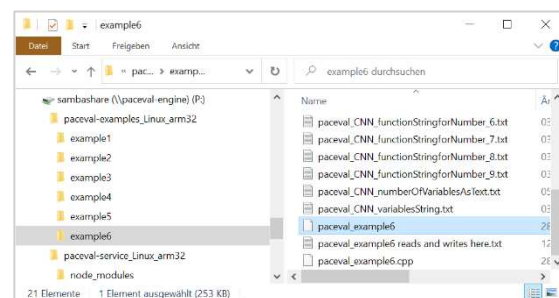
1. die selbst erzeugte shared Library „`libpacevalARM32_sharedLIB.so`“ nach `/paceval-service_Linux_arm32` kopiert (s. Bild):



2. oder das selbst erzeugte Beispiel in „`paceval_example1`“ in `/paceval-examples/example1` kopiert (s. Bild):



3. oder das selbst erzeugte Beispiel „`paceval_example6`“ in `/paceval-examples/example6` kopiert (s. Bild):



Erzeugen der ARM32 Beispiele zum Testen und der *paceval*. shared library

Der Eclipse Workspace (s. Bild) zum Erzeugen von „paceval_example1“, „paceval_example6“ und der shared library „libpacevalARM32_sharedLIB.so“ kann hier heruntergeladen werden:

https://paceval-my.sharepoint.com/:u:/p/info/EW3u_fbezuJKto2pChrl9ksB-TU8APggEOoJPtTdk8MlxA?e=yiby4A



Alle Beispiele als Netzwerkversionen zum Testen gegen das Referenzdesign

Auf dem GitHub von *paceval*. sind alle sechs Beispiele paceval_example1_server, ..., paceval_example6_server verfügbar zur sofortigen Verwendung, um diese gegen einen Server mit *paceval*. auszuführen.

Dies ist der Link zu dem Verzeichnis auf dem *paceval*. GitHub:

https://github.com/paceval/paceval/tree/main/examples_sources/C%2B%2B_examples/Windows/Microsoft%20Visual_Studio/x64_and_x86

Da auch das Referenzdesign mit Petalinux einen solchen Server darstellt, können diese ebenfalls ohne Änderungen verwendet werden. Die Beispiele verbinden sich standardmäßig mit dem Testserver von *paceval*. auf <http://paceval-service.com>. Zum Testen gegen das Referenzdesign muss lediglich die Adresse und sofern nötig der Port in der „paceval.ini“ umgesetzt werden (beispielsweise auf <http://paceval-engine:8080>).⁹¹

Das ZC706 *paceval*. Petalinux Image

Da OFFIS mehrere ZC706- oder ähnliche Entwicklerboards hat, stellen wir hier auch das Petalinux des Referenzdesigns von *paceval*. zur Verfügung. Im Kapitel „3.2 PetaLinux und die gewählten Softwarekomponenten auf dem SoC“ finden sich ausgiebig Informationen zu der gewählten Umsetzung.

Dies ist der Link zum Image für die SD-Karte:

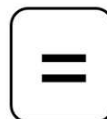
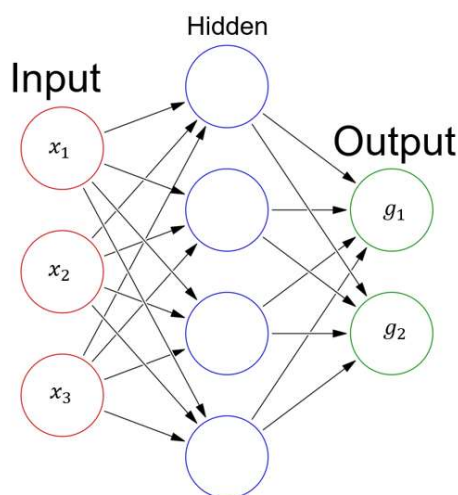
https://paceval-my.sharepoint.com/:u:/p/info/EYIJN_h77PtIvKGL40GG7TIB5uT8WlaD9z5RCL60dzW9jg?e=mumPG4

⁹¹ Weitere und ergänzende Informationen sind im Kapitel „3.3 Berechnungen mit der mathematischen Engine as a Service auf dem SoC“.

Anhang B: Expertenpapier-Umwandlung von neuronalen Netzen in geschlossene Ausdrücke⁹²

Inhalt

1 Einleitung und Ziele dieses Whitepapers.....	66
1.1 Tensoren	67
1.2 Automatische Differenzierung.....	68
1.3 Architekturen Neuronaler Netze	68
2. Methoden	70
2.1 Dynamische Verfolgung mit der Programmiersprache Julia	70
2.2 Einzelheiten zur Implementierung des Programmcodes.....	70
2.3 Überblick über den gesamten Arbeitsablauf.....	73
3. Verbesserungen und Lösungen.....	74
3.1 Exponentielles Wachstum der exportierten Formeln.....	74
3.2 Wachstum der Formeln mit zunehmender Größe der Eingaben	74



$$\begin{aligned} g_1(x_1, x_2, x_3) = & (((1.1544664390561268 * (1 / (1 + \\ & \exp((0.04934491206103983 * x_1)) \\ & + (0.12975707678433665 * x_2)) \\ & + (0.06215928752528358 * x_3))) \\ & \dots \\ g_2(x_1, x_2, x_3) = & \dots \end{aligned}$$

⁹² In der Mathematik ist ein Ausdruck in geschlossener Form ein mathematischer Ausdruck, der eine endliche Anzahl von Standardoperationen verwendet. Geschlossene Ausdrücke sind von Interesse, wenn man versucht, allgemeine Lösungen für Probleme zu entwickeln. Sie können Konstanten, Variablen, bestimmte bekannte Operationen (z. B. + - × ÷) und Funktionen (z. B. Exponent, Logarithmus, trigonometrische Funktionen und inverse hyperbolische Funktionen) enthalten, aber normalerweise keinen Grenzwert oder ein Integral.

1 Einleitung und Ziele dieses Whitepapers

Nicht alle Software- oder Hardwaresysteme bieten eine Schnittstelle, die in der Lage ist, Tensoren zu manipulieren, und die Einbindung neuronaler Netze in solche Systeme kann eine teure Herausforderung sein. Es ist dann von Vorteil, geschlossene Formeln für die Ausgaben solcher Netze zu erhalten. D.h. man möchte eine geschlossene mathematische Formel für jedes der Ausgangsneuronen eines Netzes haben, wobei nur die Eingänge des Netzes als Parameter und die trainierten Gewichte des Netzes als Konstanten in die Formel eingegeben werden.

Geschlossene Ausdrücke für die Ausgänge eines neuronalen Netzes können ebenfalls mehrere Vorteile bieten, was sie für bestimmte Anwendungen attraktiv macht.

- Erstens **können geschlossene Ausdrücke mit mathematischen Formeln eine effiziente Berechnung ermöglichen**, da sie die Beziehung zwischen den Eingangs- und Ausgangsvariablen direkt darstellen, ohne iterative Prozesse zu erfordern. Dies kann besonders in Szenarien von Vorteil sein, in denen die Rechenressourcen begrenzt sind oder in denen Vorhersagen mit geringer Latenzzeit entscheidend sind, wie z. B. in Echtzeitsystemen oder eingebetteten Geräten.
- Zweitens **können geschlossene Ausdrücke mit mathematischen Formeln die Integration von neuronalen Netzen mit anderen mathematischen Modellen oder Systemen vereinfachen** und eine nahtlose Kommunikation und Analyse zwischen verschiedenen Komponenten ermöglichen. Solche geschlossenen Ausdrücke können durch die Analyse der Architektur eines neuronalen Netzes und die manuelle Erstellung von Formeln für diese spezifische Architektur erstellt werden.
- Drittens erwartet der Industriestandard eine Genauigkeit von 99,9 % oder mehr für Entscheidungen. Da es sich bei neuronalen Netzen um Blackboxes⁹³ handelt, gibt es keine transparente Methodik, um Entscheidungen im Bereich der künstlichen Intelligenz zu optimieren und eine Verschlechterung zu vermeiden. Insbesondere soll vermieden werden, dass Entscheidungen, die in der Vergangenheit erfolgreich getroffen wurden, heute anders entschieden werden. Diese Blackbox für KI wird allgemein stillschweigend akzeptiert. Tommi Jaakkola, Professor für Elektrotechnik und Informatik am MIT, bringt es jedoch auf den Punkt: „Ob es sich um eine Investitionsentscheidung, eine medizinische Entscheidung oder vielleicht eine militärische Entscheidung handelt, man möchte sich nicht nur auf eine „Blackbox“-Methode verlassen.“⁹⁴
Mathematische Funktionen im geschlossenen Textformat können eine Möglichkeit bieten, Industriestandards zu erreichen (z. B. beim Patchen der mathematischen Funktionen).
- Viertens werden Entscheidungen mit künstlicher Intelligenz aufgrund der GPU-Technologie oft nur mit begrenzter Präzision getroffen. Beispielsweise würde für bfloat16 mit einer Genauigkeit von nur 2-3 signifikanten Dezimalstellen selbst ein einfaches neuronales Netzwerk einen möglichen Fehler in der ersten Dezimalstelle vermuten. Dies kann fatal sein, da es dazu führt, dass Objekte in einer Blackbox ohne Hintergrundwissen möglicherweise völlig falsch bewertet werden.

⁹³ Norbert Wiener, ein amerikanischer Mathematiker und Philosoph, bekannt als Begründer der Kybernetik: „Ich werde unter einer Blackbox einen Apparat verstehen, [...] der eine bestimmte Operation durchführt [...], für den wir jedoch nicht unbedingt Informationen über die Struktur haben, durch die diese Operation durchgeführt wird.“, s.

https://uberty.org/wp-content/uploads/2015/07/Norbert_Wiener_Cybernetics.pdf

⁹⁴ Quelle: <https://www.technologyreview.com/s/604087/the-dark-secret-at-the-heart-of-ai/>

Eine mathematische Engine⁹⁵, die neuronale Netze verarbeitet und den IEEE-Standard für Gleitkomma-Arithmetik⁹⁶ wie 32-Bit (Float), 64-Bit (Double), 80-Bit (Extended) oder künftig 128-Bit (Quadruple) unterstützt, **ist offensichtlich viel präziser und zuverlässiger in den mathematischen Berechnungen.**

In diesem Whitepaper wird die Methode beschrieben, mit der jedes neuronale Netz in geschlossene Ausdrücke mit mathematischen Formeln exportiert werden kann.

1.1 Tensoren

Moderne Frameworks zur Implementierung künstlicher neuronaler Netze (wie PyTorch, TensorFlow, Flux.ml und JAX) arbeiten alle nach dem Prinzip der Durchführung von Berechnungen auf mehrdimensionalen Arrays, die Tensoren genannt werden. Sowohl in der Forschung zum maschinellen Lernen als auch in der Mathematik sind Tensoren mehrdimensionale Arrays, die eine Vielzahl von mathematischen Objekten wie Skalare, Vektoren und Matrizen darstellen können. Die Verwendung und Interpretation von Tensoren in diesen beiden Bereichen ist jedoch etwas unterschiedlich.

Im mathematischen Sinne sind Tensoren algebraische Objekte, die bestimmten Transformationsregeln folgen, wenn das Koordinatensystem geändert wird. Sie können durch mehrdimensionale Arrays dargestellt werden, aber ihre wahre Natur liegt in der Art und Weise, wie sie sich unter verschiedenen Basen transformieren. Tensoren werden in verschiedenen Bereichen der Mathematik und Physik verwendet, um eine Vielzahl von Phänomenen zu untersuchen, darunter Krümmung in der Differentialgeometrie, Spannung und Dehnung in der Kontinuumsmechanik und elektromagnetische Felder in der Physik.

In der Forschung zum maschinellen Lernen werden Tensoren hauptsächlich als Datenstrukturen zur effizienten Darstellung und Verarbeitung mehrdimensionaler Daten verwendet. Sie sind die grundlegenden Bausteine für viele maschinelle Lernmodelle, insbesondere für Deep-Learning-Architekturen wie neuronale Netze. In diesem Zusammenhang werden Tensoren oft als mehrdimensionale Arrays behandelt, und der Schwerpunkt liegt eher auf ihren numerischen Eigenschaften und ihrer effizienten Berechnung als auf ihren Transformationseigenschaften unter verschiedenen Koordinatensystemen.

Frameworks für maschinelles Lernen wie TensorFlow und PyTorch bieten numerisch optimierte Tensoroperationen, die das Rückgrat verschiedener Algorithmen für maschinelles Lernen bilden.

⁹⁵ Eine Mathematische Engine ist ein Teil eines Computerprogramms oder einer Computerhardware, die als Engine bezeichnet wird und für die effiziente Verarbeitung mathematischer Modelle verantwortlich ist. Ein mathematisches Modell ist eine Abstraktion eines realen Szenarios, Systems oder Ereignisses, das mathematische Sprache verwendet, um das Verhalten, die Dynamik und die Entwicklung des Szenarios, Systems oder Ereignisses zu beschreiben und vorherzusagen.

⁹⁶ Der IEEE-Standard für Gleitkomma-Arithmetik (IEEE 754) ist ein technischer Standard für Gleitkomma-Arithmetik, der 1985 vom Institute of Electrical and Electronics Engineers (IEEE) festgelegt wurde. Der Standard befasste sich mit vielen Problemen, die in den verschiedenen Gleitkomma-Implementierungen auftraten und deren zuverlässige und portierbare Verwendung erschwerten.

1.2 Automatische Differenzierung

Alle Frameworks ermöglichen nicht nur die Manipulation von und Berechnungen mit Tensoren, sondern bieten auch eine Methode zur automatischen Differenzierung. Die automatische Differenzierung ist eine unverzichtbare Technik beim Training künstlicher neuronaler Netze (ANN), da sie die effiziente Berechnung von Gradienten zur Aktualisierung der Modellparameter ermöglicht.

Im Zusammenhang mit ANNs werden Tensoren verwendet, um sowohl die Daten als auch die Parameter des Modells darzustellen, was die Verarbeitung multidimensionaler Informationen im gesamten Netz erleichtert.

Bei der automatischen Differenzierung wird die Kettenregel aus der Infinitesimalrechnung verwendet, um den Gradienten einer skalaren Verlustfunktion in Bezug auf die Parameter des Modells zu berechnen, die normalerweise als Tensoren dargestellt werden. Die Verlustfunktion quantifiziert die Diskrepanz zwischen den Vorhersagen des Netzes und den tatsächlichen Zielwerten, und ihre Minimierung ist das Hauptziel des Trainings von ANNs. Durch die Berechnung der Gradienten kann der Lernalgorithmus, z. B. der stochastische Gradientenabstieg oder seine Varianten, die Parameter des Modells so anpassen, dass die Verlustfunktion minimiert wird, was letztlich die Vorhersageleistung des Netzes verbessert.

Moderne Deep-Learning-Frameworks wie TensorFlow, PyTorch und Flux.ml bieten effiziente, integrierte Autodifferenzierungsfunktionen, die den Trainingsprozess von ANNs rationalisieren und den Umgang mit komplexen Architekturen ermöglichen. Aus praktischer Sicht wird die Berechnung der Gradienten von modernen Frameworks hinter den Kulissen durchgeführt: Die Berechnungen werden auf den Tensoren durchgeführt (den sogenannten "forward pass") und einen vollautomatischen, so genannten "backward pass", der den Gradienten aller durchgeführten Berechnungen in Bezug auf bestimmte Tensoren (im Allgemeinen die Tensoren, die die Modellgewichte enthalten) berechnet.

1.3 Architekturen Neuronaler Netze

Künstliche neuronale Netze bestehen aus einer Reihe miteinander verbundener Schichten, von denen jede bestimmte Funktionen erfüllt, um die Eingabedaten in sinnvolle Darstellungen für genaue Vorhersagen umzuwandeln.

Diese Schichten können grob in drei allgemeine Typen eingeteilt werden: Eingabe-, versteckte und Ausgabeschichten.

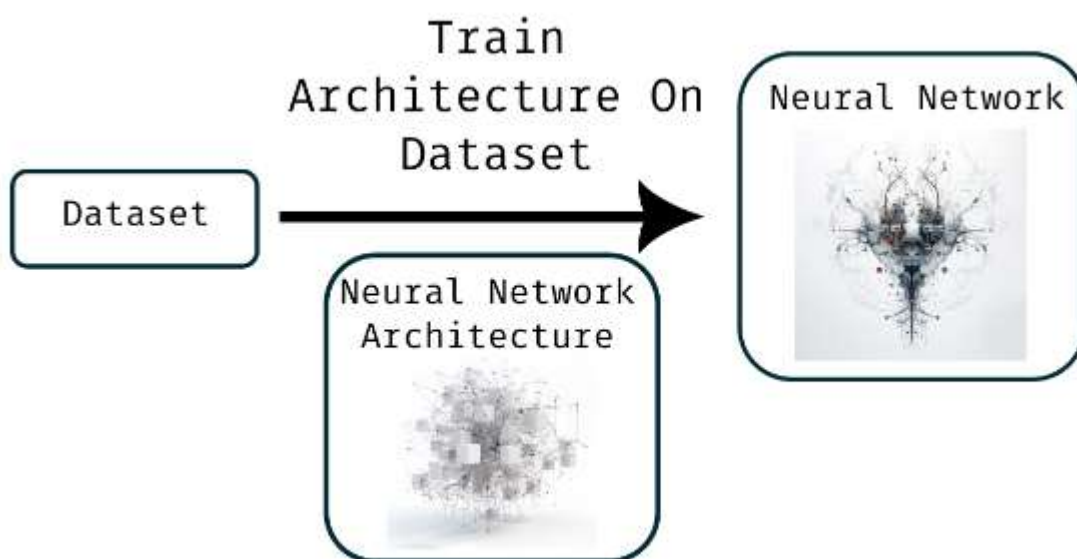
Die Eingabeschicht ist für den Empfang der Rohdaten zuständig, während die Ausgabeschicht (im Falle einer Klassifizierungsaufgabe) die endgültigen Klassifizierungswahrscheinlichkeiten oder -entscheidungen erzeugt. Versteckte Schichten, die zwischen der Eingabe- und der Ausgabeschicht liegen, bestehen aus einer Vielzahl von Schichttypen, die jeweils dazu dienen, bestimmte Muster und Merkmale in den Daten zu erfassen. In traditionellen neuronalen Netzwerkarchitekturen fließen die Informationen streng linear von einer Schicht zur nächsten. Modernere Architekturen enthalten jedoch oft mehrere Pfade, wobei die Informationen durch verschiedene parallele Schichten fließen und an bestimmten Stellen wieder integriert werden. Diese modernen Konstruktionen ermöglichen

eine größere Flexibilität und Anpassungsfähigkeit bei der Verarbeitung komplexer Datenmuster und beim Lernen komplizierter Darstellungen.

Zu den gebräuchlichen Arten von verborgenen Schichten gehören vollständig verbundene Schichten, Faltungsschichten, Pooling-Schichten und Normalisierungsschichten. Vollständig verknüpfte Schichten bestehen aus Neuronen, die mit allen Neuronen in benachbarten Schichten verbunden sind, was eine komplexe Mustererkennung durch lineare Kombinationen und nichtlineare Aktivierungen ermöglicht. Faltungsschichten, die bei der Entwicklung von Faltungsneuronalen Netzen (CNN) eine zentrale Rolle spielen, nutzen räumlich lokale Verbindungen und gemeinsame Gewichte, um lokale Muster auszunutzen und die Anzahl der Parameter zu reduzieren, wodurch sie sich besonders für Bildklassifizierungsaufgaben eignen.

Pooling-Schichten, die oft in Verbindung mit Faltungsschichten verwendet werden, verkleinern die räumlichen Dimensionen des Inputs, reduzieren die Rechenkomplexität und bieten eine Form der Translationsinvarianz. Normalisierungsschichten, wie die Stapelnormalisierung, werden zur Stabilisierung und Beschleunigung des Trainings verwendet, indem die Aktivierungen innerhalb einer Schicht normalisiert werden.

CNNs werden in der Regel durch das Stapeln mehrerer Faltungsschichten und Pooling aufgebaut, um einen hierarchischen Merkmalsextraktor zu bilden, gefolgt von einer oder mehreren vollständig verbundenen Schichten, um die endgültige Klassifizierung durchzuführen. Dank dieser Architektur können CNNs effektiv hierarchische Merkmale lernen, von einfachen Mustern wie Kanten und Texturen bis hin zu komplexen Abstraktionen wie Objektteilen und ganzen Objekten, was letztlich zu einer robusten und genauen Klassifizierung führt.



2. Methoden

Es gibt zwei allgemeine Methoden, die zur automatischen Erstellung von Formeln für ein bestimmtes neuronales Netz verwendet werden können: Statische Analyse des Berechnungsgraphen oder dynamische Verfolgung.

Dynamische Verfolgung hat den großen Vorteil, dass ein Großteil der Infrastruktur bestehender maschineller Lernsysteme wiederverwendet werden kann, weshalb wir uns für diesen Ansatz entschieden haben.

2.1 Dynamische Verfolgung mit der Programmiersprache Julia

Das Kernkonzept der dynamischen Verfolgung ist relativ einfach: Obwohl es sich um tensorbasierte Operationen handelt, wird jede mathematische Operation letztlich an den Elementen (d. h. Zahlen) innerhalb des Tensors durchgeführt. Indem wir die Zahlen durch einen Datentyp ersetzen, der nicht nur das Ergebnis einer Berechnung enthält, sondern auch alle Berechnungen aufzeichnet, die zu diesem Ergebnis geführt haben, können wir Eingaben, die in einen solchen Datensatztyp verpackt sind, über ein Netzwerk senden. Dieser Prozess würde nicht nur das Ergebnis für eine einzelne Eingabe liefern, sondern auch die vollständige Formel aller Berechnungen, die stattgefunden haben, um das Ergebnis an den Ausgangsneuronen zu erzeugen.

Während die Grundidee recht einfach ist, erweist sich ihre Umsetzung in den meisten Programmiersprachen und Frameworks für maschinelles Lernen als schwierig, weil sie entweder die Definition benutzerdefinierter Datentypen nicht unterstützen, die transparent genug sind, um problemlos durch ein neuronales Netz gesendet zu werden, oder aus Leistungsgründen sehr spezifische Datentypen erfordern. Python unterstützt beispielsweise nicht die Definition benutzerdefinierter Datentypen in einer Weise, die sie für das System vollständig transparent macht (d. h. sie verhalten sich genauso wie integrierte Datentypen), und C++-basierte Systeme schränken die zulässigen Datentypen aus Leistungsgründen häufig ein.

Die Programmiersprache Julia stellt eine bemerkenswerte Ausnahme dar. Julia wurde speziell für wissenschaftliches Rechnen, Hochleistungsrechnen und maschinelles Lernen entwickelt und ermöglicht die Definition völlig transparenter benutzerdefinierter Datentypen. Außerdem sind die meisten Pakete für maschinelles Lernen in Julia so geschrieben, dass sie solche Datentypen nahtlos verarbeiten können.

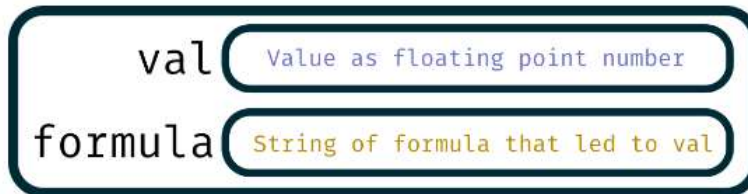
Daher haben wir beschlossen, die Exportmethode in der Programmiersprache Julia zu implementieren und Flux.jl als Deep-Learning-Framework zu verwenden.

2.2 Einzelheiten zur Implementierung des Programmcodes

Der Datentyp `RecordFormula`, auf den im Code Bezug genommen wird, ist recht einfach. Er enthält einen tatsächlichen Wert namens "val" zusammen mit einer Liste von Operationen, die durchgeführt

werden, um das Endergebnis zu erhalten:

```
struct RecordFormula{T<:Real} <: Number
    val::T
    formula::Vector{Any}
end
```



Außerdem definieren wir Standardfunktionen für unseren neuen Datentyp: wie man ihn aus verschiedenen anderen Datentypen konstruiert, wie man die automatische Konvertierung der verpackten Werte vornimmt, was Gleichheit bedeutet, usw:

```
# Define how to construct a RecordFormula in different ways
RecordFormula(v::T, f::S) where{T<:Real, S} = RecordFormula{T}(v, f)
RecordFormula{S}(v::T) where {S, T<:Real} = RecordFormula{S}(S(v), [string(v)])
RecordFormula(v::T) where {T<:Real} = RecordFormula{T}(v, [string(v)])
RecordFormula(v::Bool) = RecordFormula{Int}(Int(v))

# Define functions to access the value and the formula
val(r::RecordFormula) = r.val
function joinrecurse(a)
    if typeof(a) <: AbstractString
        return a
    end
    return join([joinrecurse(i) for i in a])
end
formula(r::RecordFormula) = joinrecurse(r.formula)

# Define how to construct similar Arrays that contain RecordFormulas
Base.similar(a::Array{T}, m::Int) where {T<:RecordFormula} = zeros(T, m)
Base.similar(a::AbstractArray{T}, dims::Base.DimOrInd...) where {T<:RecordFormula} =
    zeros(T, Base.to_shape(dims))
Base.similar(a::Array{T}, dims::Dims{N}) where {T<:RecordFormula, N} =
    zeros(T, dims)
Base.similar(a::AbstractArray, ::Type{T}, dims::Base.DimOrInd...) where {T<:RecordFormula} =
    zeros(T, Base.to_shape(dims))

Base.typemin(t::Type{RecordFormula{T}}) where {T} = typemin(T)
Base.typemin(x::RecordFormula{T}) where {T} = typemin(T)

# Define rules on how to convert and promote when RecordFormula is involved
# These allow for seamless interaction between RecordFormula instances and other numeric types.
Base.convert(::Type{RecordFormula{T}}, v::T) where {T} =
    RecordFormula{T}(v, [string(v)])
Base.convert(::Type{RecordFormula{S}}, v::RecordFormula{T}) where {S, T} =
    RecordFormula{S}(S(val(v)), v.formula)
Base.convert(::Type{RecordFormula{T}}, x::Number) where {T} = RecordFormula{T}(x)
Base.promote_rule(::Type, ::Type{RecordFormula{T}}) where {T} = RecordFormula{T}
Base.promote_rule(::Type{RecordFormula{S}}, ::Type{RecordFormula{T}}) where {S, T} =
    RecordFormula{promote_type(S, T)}
Base.promote_rule(::Type{Bool}, ::Type{RecordFormula{T}}) where {T} =
    RecordFormula{T}

# Define equality
Base.:(==)(a::RecordFormula, b::RecordFormula) = val(a) == val(b)
```

Wir können nun Operatoren und Standardfunktionen setzen, die mit unserem neuen Datentyp verwendet werden können. In beiden Fällen führen wir einfach die Operation oder Funktion aus und speichern das Ergebnis in dem Wert, den der Datentyp verpackt, und zeichnen die Operationen und Konstanten auf, die zu diesem Wert führen:

```
# Define binary operators for RecordFormula
for op in [:+, :-, :/, :*]
  @eval begin
    function Base.$op(a::RecordFormula, b::RecordFormula)
      str = ["(", a.formula, string($op), b.formula, ")"]
      newval = $op(a.val, b.val)

      return RecordFormula(newval, str)
    end
  end
end

# Define unary operators for RecordFormula
for op in [:sin, :cos, :tan, :tanh, :log, :exp, :~, :abs]
  @eval begin
    function Base.$op(a::RecordFormula)
      str = [string($op), "(", a.formula, ")"]
      newval = $op(a.val)

      return RecordFormula(newval, str)
    end
  end
end

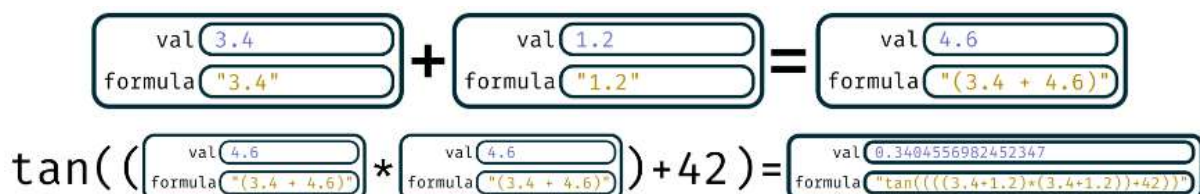
# Define different forms of maximum
function Base.max(a::RecordFormula, b::RecordFormula)
  v = max(val(a), val(b))
  return RecordFormula(v, ["(", formula(a), " max ", formula(b), ")"])
end

Base.max(a::Real, b::RecordFormula) = max(RecordFormula(a), b)
```

Wir können nun Werte in unseren Datentyp einpacken, sie arithmetisch bearbeiten, das Ergebnis der Berechnung mit der Funktion "val" ermitteln und die Funktion "formula" verwenden, um die Berechnungen zu finden, die zu diesem Ergebnis geführt haben. Z.B.:

```
a = RecordFormula(3.4)
b = RecordFormula(1.2)

formula(a) # -> "3.4"
c = a + b
formula(c) # -> "(3.4+1.2)"
val(c) # -> 4.6
d = tan(c*42)
formula(d) # -> "tan((((3.4+1.2)*(3.4+1.2))+42))"
val(d) # -> 0.3404556982452347
```

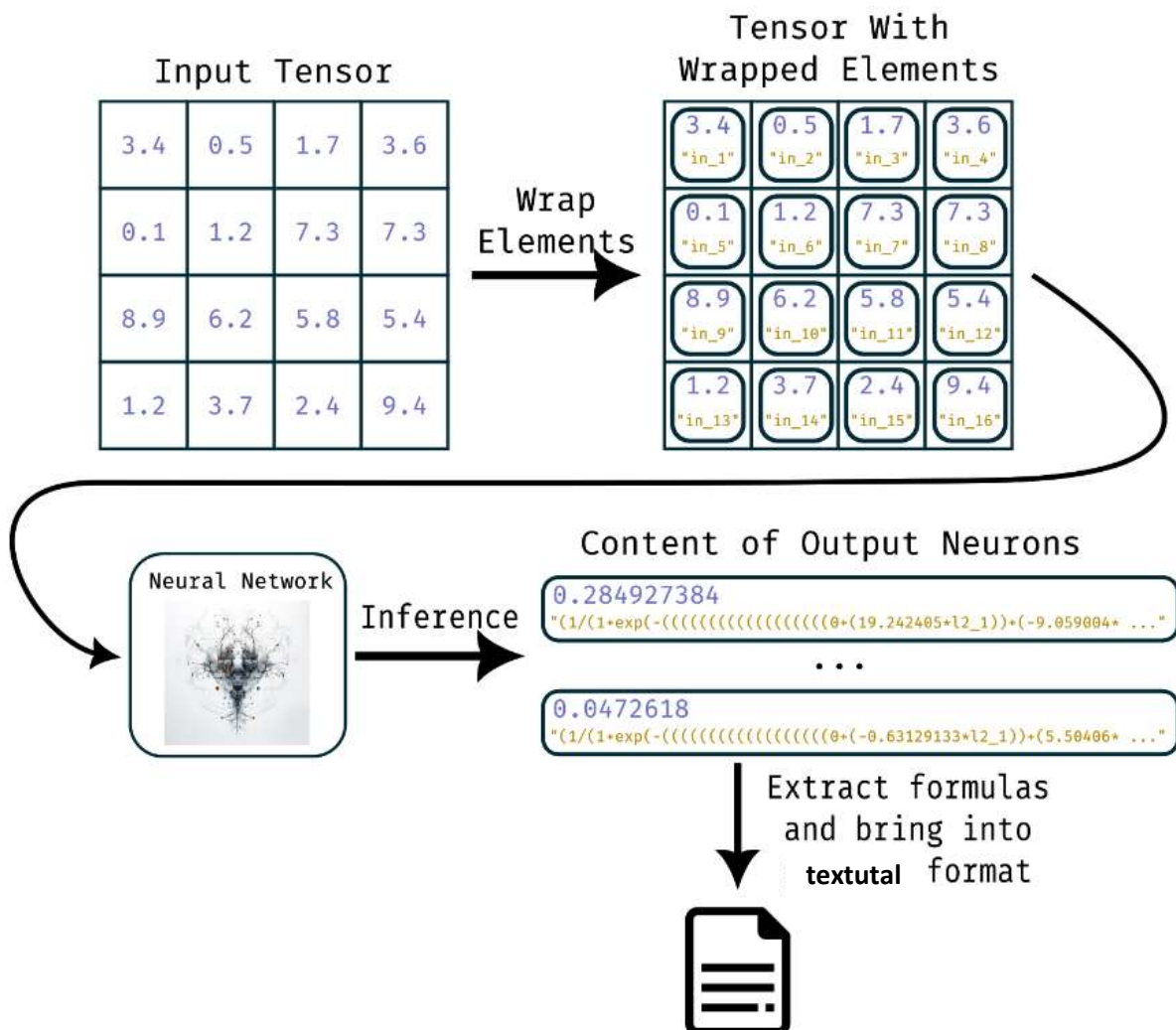


Jetzt können wir mit Flux.jl ein neuronales Netz definieren und trainieren, wie wir es in jedem anderen DL-Framework tun würden. Wir nehmen einen Beispiel-Eingabe-Tensor, verpacken alle seine Werte in eine `RecordFormula`, senden diesen `RecordFormula`-Tensor durch das Netzwerk und

rufen die Formeln von der Ausgabe ab.

2.3 Überblick über den gesamten Arbeitsablauf

1. Bereiten Sie den Datensatz vor, teilen Sie einen Testsatz für die Auswertung ab, usw.
2. Definition der zu verwendenden Netzarchitektur
3. Trainieren Sie das neuronale Netz auf den Daten
4. Die Elemente eines Eingabetensors in unseren Datentyp verpacken
5. Senden Sie diesen Eingabetensor durch das neuronale Netz
6. Formeln aus den in unserem Datentyp aufgezeichneten Vorgängen generieren
7. Bringen Sie alles in das textuelle Format, das **paceval**. erwartet, und speichern Sie es zusammen mit den Eingabebeispielen usw.
8. Speichern Sie Beispieleingaben und erwartete Ausgaben in einem Format, das von **paceval**. weiterverwendet werden kann



3. Verbesserungen und Lösungen

Dies ist eine sehr einfache Beispielimplementierung, die bereits für die meisten neuronalen Netze funktioniert. Es gibt potenzielle Verbesserungen für die aktuelle dynamische Methode zur Generierung von Formeln aus neuronalen Netzen, die wir hier vorgestellt haben. In diesem Abschnitt werden diese potenziellen Verbesserungen und bereits implementierte Lösungen diskutiert. Wir werden in diesem Whitepaper nicht auf spezielle Netze eingehen, die Verzweigungspfade oder nichtlineare Elemente verwenden, aber wir möchten erwähnen, dass es auch für diese Netze Lösungen gibt (falls sie überhaupt benötigt werden).

3.1 Exponentielles Wachstum der exportierten Formeln

Die Erzeugung eines geschlossenen Ausdrucks für die Ausgabe eines neuronalen Netzes wird mit zunehmender Tiefe des Netzes exponentiell komplexer, was auf die Zusammensetzung der Funktionen in jeder Schicht zurückzuführen ist. In einem tiefen neuronalen Netz ist die Ausgabe das Ergebnis einer Reihe von Transformationen, wobei jede Schicht einen Satz von Gewichten und Verzerrungen anwendet, gefolgt von einer Aktivierungsfunktion. Mit zunehmender Tiefe steigt die Anzahl der funktionalen Zusammensetzungen, was zu einem schnellen Anstieg der Komplexität der Formel führt. Dieses exponentielle Wachstum ergibt sich aus den Wechselwirkungen zwischen den Schichten, da jede zusätzliche Schicht die Anzahl der möglichen Verbindungen und Funktionskombinationen vervielfacht. Folglich wird die Konstruktion eines einzigen geschlossenen Ausdrucks für jeden Output für solche Netze schnell unlösbar.

Unsere Lösung zur Abschwächung des exponentiellen Anstiegs der Komplexität bei der Erstellung eines geschlossenen Ausdrucks für die Ausgabe eines tiefen neuronalen Netzes besteht in der Einführung von Zwischenvariablen auf bestimmten Schichten. Bei diesem Ansatz wird die Gesamtformel in mehrere kleinere Ausdrücke aufgeteilt, die jeweils einen bestimmten Teil des Netzes darstellen. Durch diese Aufteilung der Formel können wir unnötige Wiederholungen vermeiden und die Gesamtkomplexität des Ausdrucks reduzieren. Durch die Verwendung von Zwischenvariablen, die den Ausgang jeder Schicht bezeichnen, können wir die Gleichung für den geschlossenen Regelkreis für jede Schicht in Form dieser Variablen ausdrücken, anstatt uns auf eine einzige, unhandliche Formel zu stützen. Diese Methode wurde bereits in unserem Beispielcode angewandt, so dass die Tiefe des neuronalen Netzes kein Problem mehr darstellt. Diese Aufteilung und Bearbeitung des Netzes mit mehreren kleineren Ausdrücken hat natürlich auch Vorteile auf Systemen, die eine parallele Ausführung erlauben, wie z.B. Multiprozessorsysteme oder FPGAs.

3.2 Wachstum der Formeln mit zunehmender Größe der Eingaben

Wenn Sie auf einem kleinen System wie einem IoT- oder IIoT-Gerät mit Hilfe eines neuronalen Netzes Entscheidungen über eine große Anzahl von Eingaben treffen wollen, muss das System mit ausreichend Speicher ausgestattet sein. Zur Veranschaulichung nehmen wir die Pixel eines Bildes als Eingaben für das neuronale Netz. Die Generierung geschlossener Formeln für alle Ausgänge von

Faltungsnetzwerken (CNNs) stellt selbst bei mittleren Bildgrößen eine große Herausforderung dar, da die Anzahl der benötigten Formeln mit zunehmender Bildgröße schnell ansteigt, da ein Bild von z.B. 640×480 Pixel somit bereits 307.200 Eingaben hat. CNNs verwenden Faltungsschichten, um das Eingabebild systematisch zu scannen und wertvolle Merkmale zu extrahieren, was zahlreiche Filter erfordert, die im gesamten Bild arbeiten. Folglich steigt mit zunehmender Bildgröße die Zahl der Berechnungen, Parameter und Interaktionen zwischen Filtern und räumlichen Positionen, was zu einer erheblichen Ausweitung der Zahl der Formeln führt, die zur Beschreibung des Verhaltens des Netzes erforderlich sind. Dies ist eine grundlegende Tatsache, die nicht umgangen werden kann.

Größere Filter können dazu beitragen, die Komplexität der Erzeugung geschlossener Ausdrücke für Faltungsneuronalen Netze (CNN) bis zu einem gewissen Grad zu reduzieren. Die Verwendung größerer Filter ermöglicht es dem Netz, größere Bereiche des Eingabebildes in weniger Schritten abzudecken, wodurch die Anzahl der Berechnungen und Interaktionen zwischen Filtern und räumlichen Positionen verringert wird. Durch die Einbeziehung der Schrittgröße, mit der sich der Filter über das Bild bewegt, kann das Netz außerdem Pixel überspringen, was die Anzahl der erforderlichen Berechnungen weiter reduziert. Wir haben diese Methode für das große Netz zur Erkennung von Gesichtern in Bildern verwendet. Diese Techniken können das Komplexitätsproblem zwar bis zu einem gewissen Grad abmildern, haben aber auch gewisse Nachteile. Größere Filter können zu einem Verlust an feinkörnigen räumlichen Informationen führen, da der Filter einen größeren Teil des Bildes abdeckt und möglicherweise kleinere, aber wichtige Merkmale übersehen werden, und das Netz verliert wichtige Informationen über das Eingabebild.

Eine mögliche Lösung, die allgemein für eine große Anzahl von Eingaben funktioniert (die in unserem Beispielcode nicht enthalten ist), ist die Verwendung einer Notation, die es ermöglicht, wiederkehrende Formeln anzugeben (d. h. Formeln, die wiederholt auf verschiedene Eingaben angewendet werden). Dieser Ansatz wäre ein geeignetes Mittel zur Darstellung von Faltungsschichten, da das Hauptziel dieser Schichten darin besteht, z. B. denselben Filter auf ein ganzes Bild anzuwenden. Die Einführung einer solchen Notation würde die Darstellung von Faltungsschichten vereinfachen und die Kompaktheit der geschlossenen Formeln erheblich verbessern.

Beispiele für drei Grösstentypen von neuronalen Netzen (Fashion – klein, Hautkrebserkennung – mittel und Gesichtskennung – gross) finden sich auf dem GitHub von *paceval*. unter <https://github.com/paceval> unter dem Punkt:

- [paceval and artificial intelligence](#) describes the method to export each neural network into closed expressions with mathematical formulas.

Closed expressions for the outputs of a neural network can also offer several advantages, making them an attractive option for certain applications.

Anhang C: OFFIS-Machbarkeitsstudie für die Umsetzung der paceval-Software in FPGA- Hardware

Abschlussbericht

Dr.-Ing. Jörg Walter, OFFIS e.V.

31.07.2023

Inhalt

1.	Einleitung	78
2.	Untersuchungsziele	78
2.1.	Rechenleistung	79
2.2.	Kosteneffizienz	80
2.3.	Energieeffizienz	80
3.	Evaluationsmethode	81
3.1.	Systemarchitektur	81
3.2.	Implementation des Prototyps	82
3.3.	Hardware-Plattform	83
3.4.	Messverfahren	83
4.	Ergebnisse	86
4.1.	Rechenleistung und Ressourcenbedarf	86
4.2.	Kosteneffizienz	87
4.3.	Energieeffizienz	87
5.	Bewertung	88
5.1.	Rechenleistung	88
5.2.	Kosteneffizienz	89
5.3.	Energieeffizienz	89
6.	Zusammenfassung	90
7.	Quellenangaben	91

Dokumentenhistorie

Datum	Autor	Kommentar
30.07.2023	Jörg Walter	Initiale Version
31.07.2023	Jörg Walter	Zusätzliche Quellenangaben eingefügt

1. Einleitung

Die Firma paceval hat eine Software-Bibliothek entwickelt, die nahezu beliebig komplexe mathematische Ausdrücke berechnen kann. Die Software liest eine textuelle Beschreibung der Formeln ein, die die Grundrechenarten, die üblichen transzendenten Funktionen (Trigonometrie, Exponentialfunktion usw.) und andere gebräuchliche Operationen enthalten darf. Ausdrücke können beliebig viele Platzhalter (Variablen) enthalten. Berechnungen werden in wählbarer Genauigkeit durchgeführt (einfach, doppelt und erweitert mit 80bit). Zusätzlich kann die Bibliothek auch ein Intervall ausgeben, das die Fehlergrenzen aufgrund der begrenzten Genauigkeit der Gleitkommazahlenformate angibt.

Die Bibliothek übersetzt diese Formeln in interne Datenstrukturen und führt dann die dadurch spezifizierten Berechnungen durch. Dabei nutzt die Bibliothek eine interne Darstellung, die eine einfache Parallelisierung ermöglicht. Die bestehende Implementierung kann dadurch alle Rechenkerne eines Hauptprozessors nutzen, um die Berechnung zu beschleunigen.

Die beschriebene Arbeitsweise lässt vermuten, dass sich das Berechnungsverfahren auch effizient in Hardware realisieren lässt. Chip-Design für anwendungsspezifischen integrierten Schaltkreise (ASICs) ist jedoch äußerst aufwändig und hätte in diesem Fall viele Personenjahre erfordert. Field Programmable Gate Arrays (FPGAs) sind hingegen ein einfacherer Weg, eigene Hardware zu entwerfen und zu erproben. Prinzipbedingt sind FPGAs weniger effizient als ASICs (bezogen auf Chipfläche, Energie und Taktgeschwindigkeit). Sie sind aber dennoch regelmäßig in der Lage, mit spezialisierten Schaltkreisen eine Software auf einem normalen Prozessor bei Energiebedarf und/oder Rechengeschwindigkeit zu übertreffen.

Im Rahmen eines Validierungsauftrages der deutschen Sprunginnovationsagentur SPRIN-D hat paceval einen Unterauftrag an OFFIS vergeben, in dem mit Hilfe eines FPGA-Prototypen zu bestimmen ist, ob sich dieses Potential auch für die paceval-Software realisieren lässt. Dieser Bericht präsentiert Vorgehensweise, Ergebnisse Bewertung dieser Untersuchung

Die Struktur des Dokuments ist wie folgt: Die technische Zielsetzung wird im nächsten Abschnitt beschrieben. In Abschnitt 3 wird dann erläutert, wie der Prototyp umgesetzt und vermessen wurde. Abschnitt 4 präsentiert die Messwerte, die dann in Abschnitt 5 für eine Bewertung genutzt werden. Abschließend werden die Ergebnisse noch einmal kurz zusammengefasst.

2. Untersuchungsziele

Eine Implementation auf einem FPGA ist vor allem durch die vorhandenen Hardware-Ressourcen begrenzt. Die primäre Ressource sind dabei sogenannte Look-Up Tables (LUTs), mit der digitale Schaltkreise nachgebildet werden. Weiterhin befindet sich in einem FPGA sogenanntes Block-RAM (BRAM), welches verteilt über den ganzen FPGA einen großemäßig eingeschränkten, aber schnellen Speicher darstellt. Für Gleitkommaoperationen stellen moderne FPGAs zusätzlich noch sogenannte DSP-Blöcke zur Verfügung. Diese sind spezialisierte Recheneinheiten, die bestimmte Gleitkomma-Operationen wesentlich effizienter umsetzen als wenn man die gleiche Funktionalität mit LUTs nachgebildet hätte.

Die entscheidende Herausforderung ist bei Hardware-Design für FPGAs, mit diesen begrenzten Ressourcen optimal umzugehen. Für die Bewertung einer Hardware-Implementation der paceval-Software sind dabei mehrere Kennzahlen von Bedeutung.

2.1 Rechenleistung

Die grundsätzliche Beschleunigungsstrategie der paceval-Software ist eine parallele Abarbeitung der Berechnungen. Die konkrete Umsetzung in der Software begünstigt dabei auch die Parallelisierung in Hardware.

Der populärste Ansatz der Hardware-Parallelisierung ist Datenparallelität (Single Instruction Multiple Data – SIMD), z.B. In Vektoreinheiten oder Grafikprozessoren. Beim paceval-Ansatz handelt es sich allerdings um komplexe Rechenfolgen auf verhältnismäßig wenig Daten (Multiple Instruction Multiple Data – MIMD).

Der Prototyp soll daher so aufgebaut werden, dass möglichst viele unabhängige Berechnungseinheiten gleichzeitig an einer Berechnung arbeiten können. Für die Hardware-Implementierung ist die Rechenleistung somit davon abhängig, welche Rechenleistung eine einzelne Berechnungseinheit hat, und wie viele Berechnungseinheiten mit den oben genannten Ressourcen eines einzelnen FPGAs realisiert werden können.

Die reine Rechenleistung ist an sich nicht aussagekräftig, da bei einer parallelen Abarbeitung auch einfach mehrere FPGAs zusammengeschaltet werden könnten. Die Rechenleistung kann also eigentlich nur in Relation mit einer zweiten Größe bewertet werden.

Im einfachsten Fall ist dies einfach der Platzbedarf, also die Fläche auf einer Leiterplatte (und deren Design-Aufwand). Je mehr Chips kooperieren müssen, desto eher negiert die Komplexität des Gesamtaufbaus die ansonsten denkbaren Vorteile.

Es wäre also wünschenswert, dass die Rechenleistung in einem einzelnen FPGA zumindest die Größenordnung erreicht, die ein einzelner generischer Prozessor bzw. System-on-Chip (SoC) liefert. Hierbei sollte ein Vergleichsprozessor verwendet werden, der aus der gleichen Technologie-Generation wie der FPGA stammt (16nm Strukturgröße). Es bietet sich der ARM Cortex-A72 an, der ein Performance-optimierter Prozessor ist, der in einer 4-Kern-Konfiguration mit nominell 1,4 GHz läuft im Raspberry Pi 3B+ und vielen anderen eingebetteten Systemen zum Einsatz kommt. Als Embedded-Prozessor ist er auch energiesparender als eine Desktop- oder Server-CPU, was im FPGA-Vergleich sinnvoll ist.

Es ist allgemein üblich, die Rechenleistung in Millionen Fließkommaoperationen pro Sekunde anzugeben (MFLOPS, aber auch GFLOPS, TFLOPS usw.). Dabei ist zu beachten, dass der Gegenstand der Untersuchung Gleitkommaberechnungen in doppelter Genauigkeit nach den Regeln der Norm IEEE 754 sind. Es wird also eine Bit-genaue Umsetzung der IEEE-754-Regeln angestrebt. Insbesondere bei sogenannten denormalisierten Zahlen ist eine standardgetreue Implementation aufwändiger als eine, bei der dieser Randfall nicht implementiert ist.

Der LINPACK-Benchmark ist der allgemein akzeptierte Standard, um die praktisch erreichbaren MFLOPS einer Hardware zu messen. Er besteht hauptsächlich aus Additionen und Multiplikationen. Er ist für Vergleichszahlen gut geeignet, da für diesen viele hochoptimierte Implementierungen vorliegen. LINPACK hat allerdings einen hohen Datenparallelismus, weswegen er für die paceval-

Bibliothek eine etwas überpessimistische Referenz darstellt und deren Stärken bei heterogenen Berechnungen nicht berücksichtigt. Da es beim Vergleich aber nur um die grobe Größenordnung geht, ist dies eine hinnehmbare Schwäche.

2.2 Kosteneffizienz

Für den in dieser Untersuchung verwendeten FPGA-Baustein lässt sich mit der Rechenleistung auch unmittelbar die Kosteneffizienz berechnen, also Rechenleistung pro Geldeinheit. Da leistungsfähige FPGAs im Vergleich zu generischen Prozessoren unverhältnismäßig teuer sind, ist nicht zu erwarten, dass die reine Kosteneffizienz mit in Großserie hergestellten Prozessoren mithalten kann. Denkbar wäre aber, dass in Rechenzentren die Einsparungen bei Energiekosten die Anfangsinvestition ausgleichen.

Ansonsten sind zwei Fälle zu unterscheiden: Wenn das Ziel einer Implementation die FPGA-Technik ist, muss eine andere Kennzahl einen Vorteil haben, der die geringere Kosteneffizienz rechtfertigt. Dies ist insbesondere im Bereich des Internet of Things (IoT) und allgemein eingebetteter Systeme denkbar, da dort regelmäßig kleine Formfaktoren und begrenzte Energiebudgets notwendig sind.

Der andere Fall ist eine zukünftige ASIC-Umsetzung. Hier sind die FPGA-Kosten irrelevant. An deren Stelle treten die enormen Fixkosten eines ASIC-Entwurfsprozesses. Da die Gesamtkosten von vielen unbekannten Faktoren abhängen (Größe, Technologie, Stückzahl etc.), wird sich ohne zusätzliche Informationen keine glaubwürdige Extrapolation der Kosteneffizienz einer ASIC-Implementation vornehmen lassen.

2.3 Energieeffizienz

Die wichtigste Kennzahl ist somit die Energie. In IoT-Szenarien und eingebetteten Systemen gibt es oft eine harte Obergrenze bei der zulässigen Leistungsaufnahme. Diese entsteht durch die Dimensionierung der Stromversorgung und/oder durch die Fähigkeit, die entstehende Verlustwärme abzuführen. Tatsächlich ist beides auch regelmäßig in Server-Anwendungen relevant.

Eingebettete Systeme sind oft besonders limitiert, da viele von ihnen passiv gekühlt werden (müssen). Typische Leistungsgrenzen für passive Kühlung mit ausreichend Luftzirkulation sind ca. 10W. In vollständig geschlossenen Geräten mit kleinem Formfaktor kann diese Grenze leicht bis 1W sinken. Dies ist für traditionelle Rechentechnik nur unter erheblichen Leistungseinschränkungen zu erreichen.

Bei Server-Technik besteht mehr Flexibilität im Entwurf der Systeme. Dort sind die Energiekosten der eigentliche Grund, warum Energieeffizienz notwendig ist. Dazu kommt, dass die Energiekosten proportional zu den Kosten für die Wärmeabfuhr sind, bei vergleichbarer Größenordnung. Man kann hier also überschlägig mit den doppelten Energiekosten rechnen.

Als Kennzahl bietet sich daher an, die Energieeffizienz in MFLOPS pro Watt zu verwenden. Da FPGA-Technik in vielen Anwendungen eine hohe Energieeffizienz bewiesen hat, sollte zu erwarten sein, dass hier das wahre Potential des paceval-Beschleunigers gezeigt werden kann. Zudem lässt sich der

Wert zumindest grob auf ASICs extrapolieren, da es hierzu schon wissenschaftliche Untersuchungen gibt.

3. Evaluationsmethode

3.1 Systemarchitektur

Um zu bestimmen, welche Kennzahlen ein paceval-Beschleuniger erreichen kann, wird angenommen, dass die finale Systemarchitektur so aussieht wie in Abbildung 40. Die Berechnungen werden in eine geeignete Datenstruktur (Node List) umgeformt, die dann in ein eingebettetes BlockRAM übertragen wird. Dies steuert die Ausführung des Beschleunigers. Fasst man die Recheneinheit als eine ungewöhnliche Art von Spezialprozessor auf, stellt die Node List das Programm dar. Für Konstanten und Variablen wird ein separater eingebetteter Speicher genutzt. Beides zusammen bewirkt, dass Latenzen durch Speicherzugriffe minimiert werden, und dass der Prozessor während der Berechnungen vollständig autark arbeiten kann. Daten werden in einem separaten Schritt über ein reguläres Bussystem mit der Software ausgetauscht.

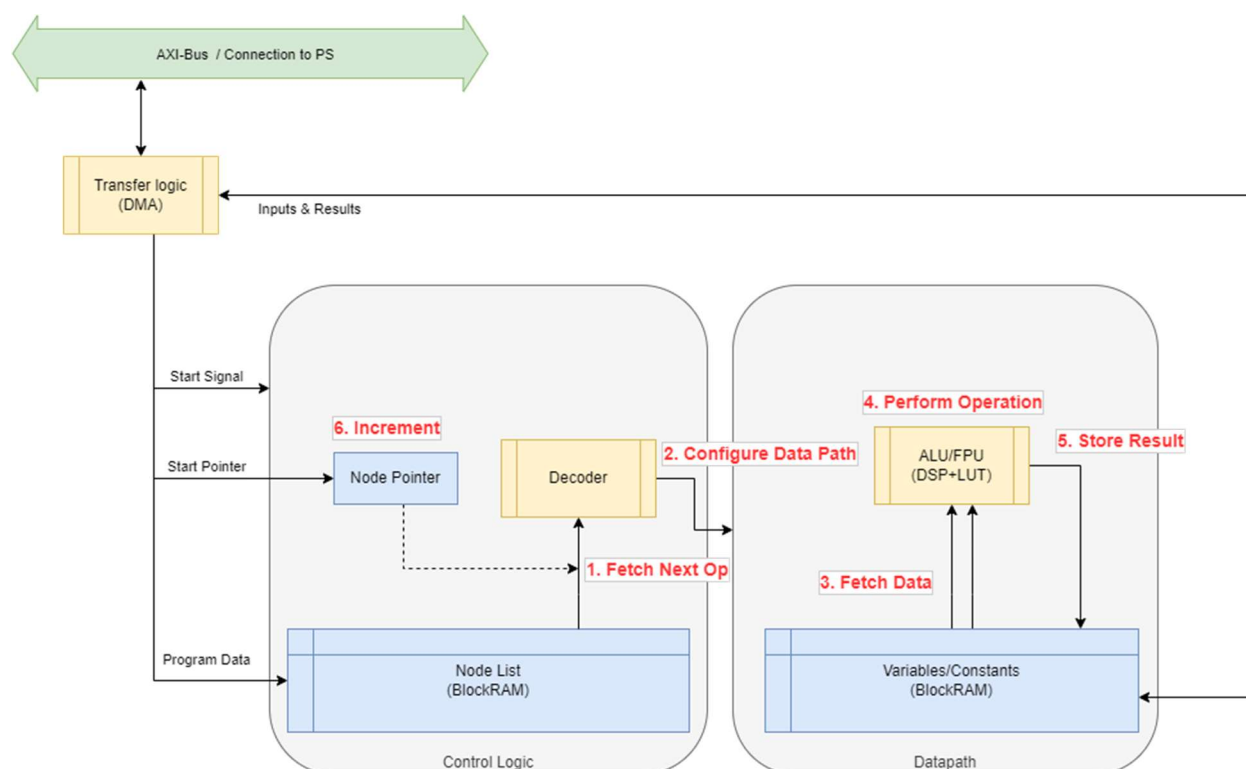


Abbildung 40: Die geplante Zielarchitektur des Beschleunigers

Die Ausführung besteht aus wiederholten Zyklen aus sechs Schritten:

1. Zuerst wird die nächste Operation aus der Node List ausgelesen. Hierzu existiert ein Node-Pointer-Register, das den Index der aktuellen Operation enthält
2. Die Kontrolllogik konfiguriert den Datenpfad der eigentlichen Recheneinheit (FPU)
3. Die Operanden der Operation werden aus dem Datenspeicher gelesen
4. Die FPU führt die Operation durch
5. Das Ergebnis wird in den Datenspeicher geschrieben
6. Der Node Pointer wird um eins erhöht, um auf das nächste Element zu zeigen

Diese Schritte müssen nicht strikt linear abgearbeitet werden, in einer optimierten Implementation können sich zwei Zyklen überlappen, z.B. kann Schritt 6 eher (parallel) stattfinden, und Schritt 1 des Folgezyklus kann während Schritt 5 des vorangegangenen Zyklus durchgeführt werden.

Da die Node List linear abgearbeitet wird und insbesondere keine Sprünge oder Verzweigungen enthält, fällt die Steuerlogik (Decoder) der Recheneinheit besonders einfach aus. Zudem ist das Zeitverhalten sehr gut vorhersagbar. Dies hat entscheidende Vorteile bei der Parallelisierung, da sich dadurch parallele Teilaufgaben statisch verteilen lassen und auf aufwendige Synchronisation zwischen den Recheneinheiten verzichtet werden kann. Allerdings setzt dies voraus, dass in der Software entsprechende Vorarbeiten geleistet werden, welche für diese Studie natürlich nicht verfügbar waren.

3.2 Implementation des Prototyps

Für die Messungen hat OFFIS diesen Prozessor in einer Minimalversion implementiert. Er führt die Zyklen in einer Zustandsmaschine von 5 Schritten durch (Abbildung 41). Die verwendete FPU ist eine Open-Source-Implementation, die die Regeln des IEEE 754 vollständig umsetztⁱ. Die FPU von Xilinx garantiert dieses Verhalten nicht. Die Konsequenzen dieser Wahl und mögliche Alternativen werden in Abschnitt 0 genauer erörtert.

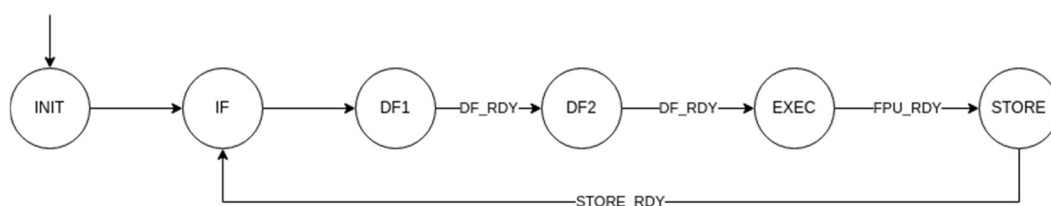


Abbildung 41: Zustandsmaschine der Kontrolllogik in der prototypischen Implementation. IF = Instruction Fetch; DF = Data Fetch

Da die Energieeffizienz eine wesentliche Kennzahl ist, muss die Hardware vermessen werden können, ohne dass Software-Bestandteile in das Messergebnis einfließen. Hierzu wurde die Node List mit einem festen Programm vorbelegt, das einen autarken Betrieb ermöglicht. So konnte der Energieverbrauch der reinen Hardware vermessen werden. Die fest eingebauten Prozessorkerne auf dem Ziel-FPGA lag während der gesamten Untersuchung brach.

Das vorbelegte Programm führt eine gemischte Berechnung aus Additionen und Multiplikationen durch. Dies ermöglicht eine gute Bewertung anhand von LINPACK-basierten Vergleichswerten anderer Plattformen.

Auch für typische neuronale Netze ist dies ein realistischer Testfall. Diese bestehen aus Additionen und Multiplikationen für die gewichteten Aktivierungen und einer komplexeren Funktion (z.B. Sigmoid) pro Neuron als Aktivierungsfunktion. Aufgrund der Struktur neuronaler Netze kommen aber Addition und Multiplikation um viele Größenordnungen häufiger vor. Schon bei einem schlichten MNIST-Benchmark gibt es 1000 mal mehr Additionen/Multiplikationen als Aufrufe der Exponentialfunktion. Letztere kann die Effizienz eines Beschleunigers also nicht maßgeblich beeinflussen.

Die Parallelisierung wurde dadurch hergestellt, dass möglichst viele unabhängige Recheneinheiten instanziiert wurden. Dies ist eine hinreichend gute Approximation der echten Parallelisierungsarchitektur, die wegen der fehlenden Software-Erweiterungen nicht komplett nachgebildet werden konnte.

3.3 Hardware-Plattform

Als Zielplattform wurde während des Projektes in Rücksprache mit *paceval* das Entwicklerboard AMD (Xilinx) EK-U1-ZCU104-G gewählt. Auf diesem befindet sich ein Zynq Ultrascale+ XCZU7EV-2FFVC1156 System-on-Chip, dessen FPGA-Anteil 504.000 Logikzellen, 11 Megabit BlockRAM und 1728 DSP-Blöcke hat.

In der Untersuchung wurde der FPGA mit 125MHz getaktet, wobei auch 300MHz als Takt zur Verfügung steht. Dies konnte jedoch nicht unmittelbar genutzt werden, da die FPU in ihrem aktuellen Zustand diesen Takt nicht unterstützt.

3.4 Messverfahren

Zunächst wurde die korrekte Funktion der Recheneinheiten in einer Simulation geprüft. Abbildung 42 illustriert exemplarisch, wie detailliert eine Simulation den internen Zustand der Hardware anzeigt. Diese Simulation ist normalerweise sehr zuverlässig, aber um zu prüfen, dass tatsächlich die korrekte Funktion ausgeführt wird, wurden mehrere Signale der ersten Recheneinheit auf Pins des Entwicklerboards herausgeführt, wo sie mittels Oszilloskop beobachtet und mit den Simulationsergebnissen verglichen werden konnten. Aus diesen Ergebnissen lässt sich die Rechenleistung berechnen.

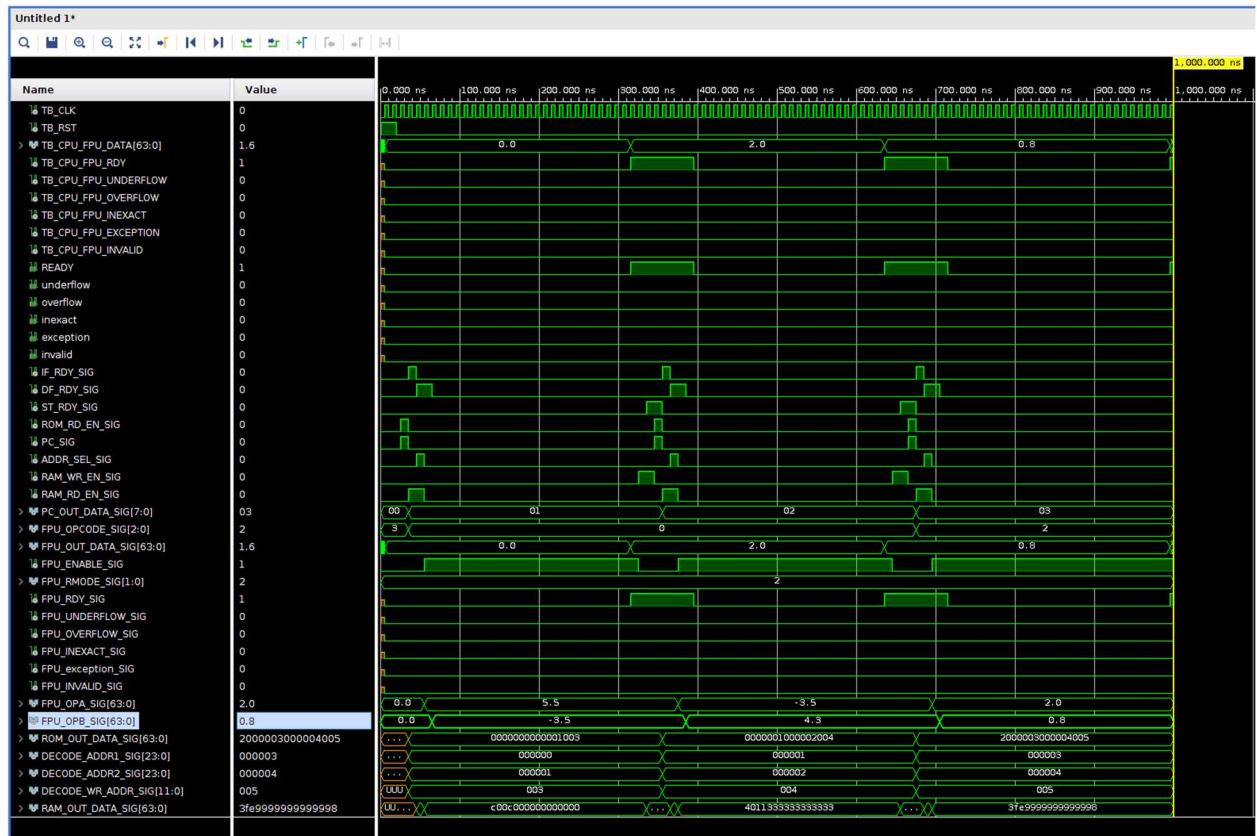


Abbildung 42: Illustration des Detailgrades, in dem die simulierte Funktionalität beobachtet werden kann

Die Energieaufnahme wurde indirekt gemessen. Das Entwicklerboard bietet keinen getrennten Zugang zu der Stromversorgung des FPGAs. Daher wurde die Hauptstromversorgung vermessen. Hierbei werden allerdings auch alle aktiven Komponenten auf dem Entwicklerboard gemessen werden, die in diesem Fall eine signifikante Leistungsaufnahme haben.

Für die Bestimmung der Leistungsaufnahme des reinen Hardware-Designs wird daher zweimal gemessen: einmal bei unprogrammiertem FPGA, und einmal mit dem aktiven Evaluations-Design. Die Differenz beider Werte stellt den Anteil dar, der durch die Hardware-Aktivität verursacht wird (dynamische Verlustleistung). Er enthält in diesem Fall auch die statische Verlustleistung, da laut Xilinx ein unprogrammierter FPGA keine statische Verlustleistung hatⁱⁱ.

Auf dem Entwicklerboard befindet sich in der Hauptstromversorgung ein Messwiderstand von 5 Milliohm. Dieser lieferte jedoch bei den beobachteten Strömen eine zu geringe Spannungsdifferenz um präzise messen zu können. Um in einem zuverlässigeren Messbereich arbeiten zu können, wurde in die Hauptstromversorgung ein 1-Ohm-Widerstand eingefügt, über den ausreichend Spannung abfiel. Strom und Leistung wurden dann regulär über das Ohm'sche Gesetz bestimmt (Abbildung 43).

Zum Messen der Spannungsdifferenz kam ein Oszilloskop vom Typ Agilent MSO6014A zum Einsatz. Dies wurde mit Mittelwertbildung über 8 Samples konfiguriert. Die Spannungsdifferenz wurde über die Mathematik-Funktion bestimmt, und über die Mess-Funktion wurde dann ein Gesamt-Mittelwert für jedes der drei Messsignale (Spannung vor und nach dem Messwiderstand plus Differenz) bestimmt.

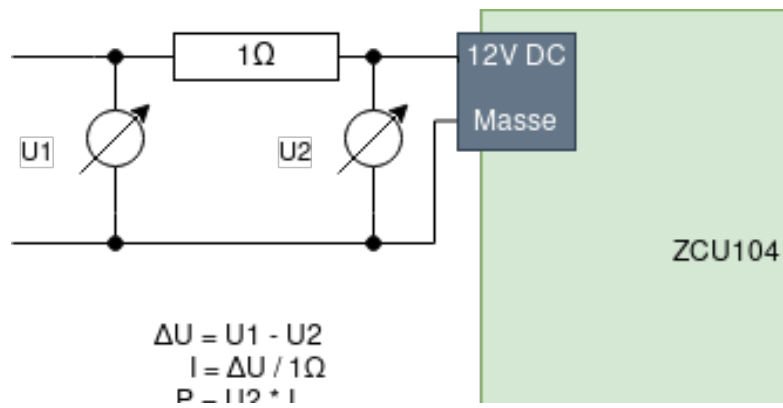


Abbildung 43: Messaufbau für die Energiemessung

4. Ergebnisse

Zunächst konnte mit dem Oszilloskop beobachtet werden, dass der Prozessor exakt wie vorhergesagt funktioniert. Abbildung 44 zeigt ein Beispiel, in dem man das FPU_READY-Signal aus Abbildung 41 sieht, sowie den Systemtakt. Die Messung deckt sich mit dem von der Simulation vorhergesagten Verhalten.

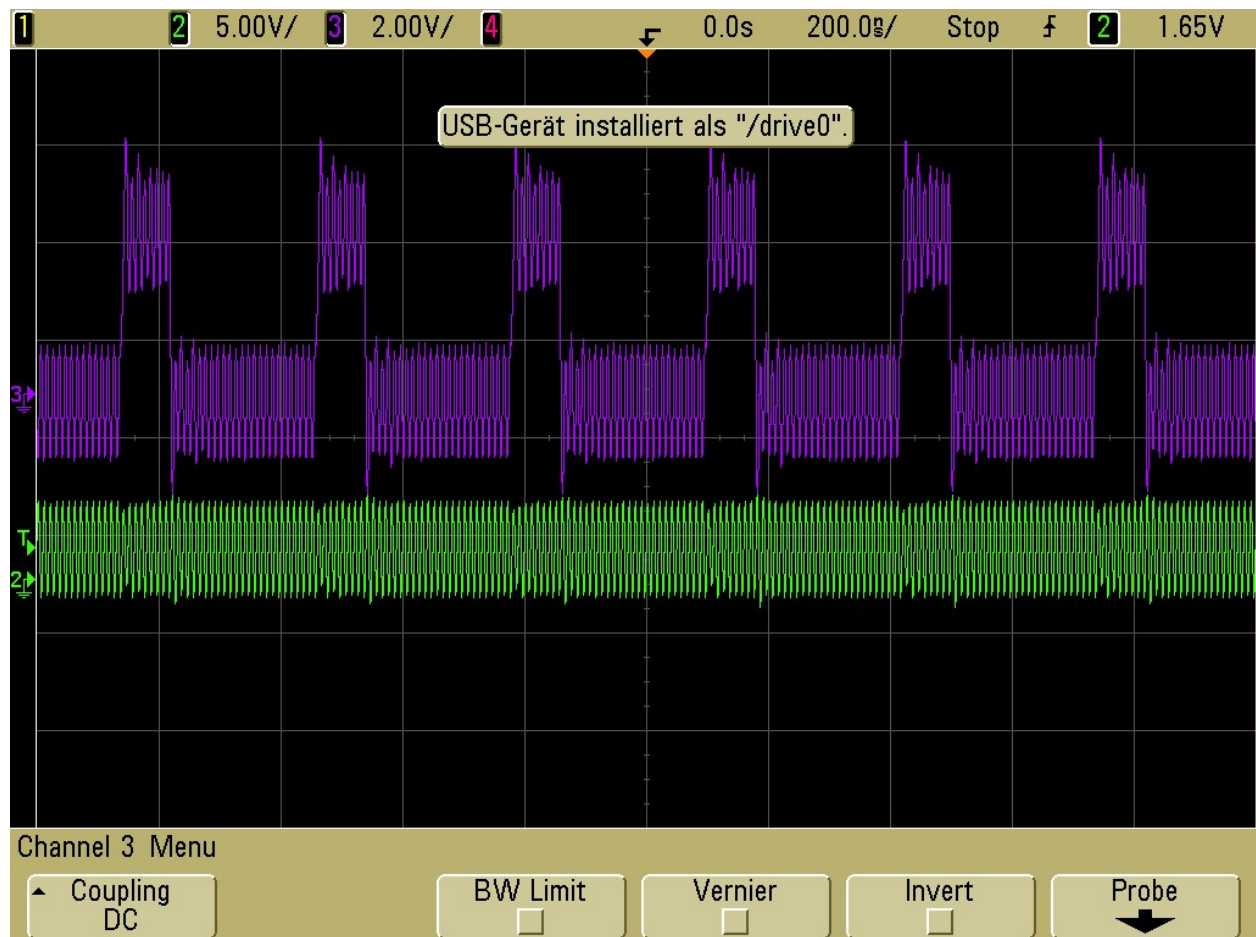


Abbildung 44: Bildschirmfoto von der Oszilloskop-Beobachtung des FPU_READY Signals (lila) und des Systemtakts (grün)

4.1 Rechenleistung und Ressourcenbedarf

Der Prototyp konnte insgesamt 89 mal in der Ziel-Hardware instanziiert werden. Abbildung 45 zeigt den Bericht von Xilinx Vivado über die Ressourcenauslastung. Die LUT-Auslastung ist ungewöhnlich hoch, dafür waren noch über die Hälfte der DSP-Blöcke frei. Die Ursache dürfte sein, dass Xilinx Vivado bei der Synthese der FPU nicht in der Lage war, die Addition mit DSP-Blöcken zu realisieren. Laut Syntheseprotokoll wurde diese komplett in LUT-Logik umgesetzt. Im Vergleich mit dem Xilinx-eigenen Floating-Point IP-Core sticht insbesondere hervor, dass dort denormalisierte Gleitkommazahlen nicht standardkonform behandelt werden können. Dies könnte die Ursache für dieses Verhalten sein.

Resource	Utilization	Available	Utilization %
LUT	228641	230400	99.24
FF	307228	460800	66.67
BRAM	89	312	28.53
DSP	801	1728	46.35
IO	6	360	1.67
BUFG	1	544	0.18
MMCM	1	8	12.50

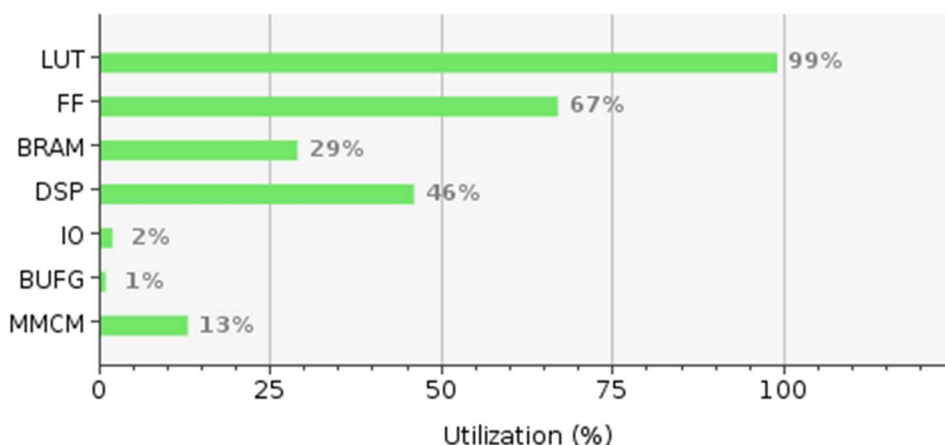


Abbildung 45: Ressourcenauslastung des vermessenen Prototypen

Die Recheneinheit benötigt 24 Taktzyklen für eine Addition und 26 Taktzyklen für eine Multiplikation. Bei 125 MHz Systemtakt und 89 Instanzen ergibt dies 445 MFLOPS.

4.2 Kosteneffizienz

Die UltraScale+-FPGAs sind derzeit äußerst schwer verfügbar. Die einschlägigen Händler (Farnell, Mouser, RS-Online, DigiKey) haben nur vereinzelte Exemplare auf Lager. Entsprechend hoch ist der aktuelle Marktpreis von bis zu 5.000 € pro Stück. Ein größerer Chip mit etwa 50% mehr DSPs ist zum Zeitpunkt des Berichts in größeren Stückzahlen zum Preis von 783€ lieferbarⁱⁱⁱ.

Extrapoliert man die damit erreichbare Rechenleistung (Faktor 1,46) ergibt sich so eine Kosteneffizienz von ca. 0,83 MFLOPS pro Euro.

4.3 Energieeffizienz

Das Entwicklerboard hat eine Leistungsaufnahme von 7,084W im unprogrammierten Zustand. Wie in Abschnitt 0 erläutert umfasst dies aber zahlreiche Komponenten, auf dem Entwicklerboard, die nicht

den Energieverbrauch des eigentlichen FPGAs darstellen. Ein Wert für den reinen FPGA ließ sich nicht messen, aber die Xilinx-Software hat für die implementierte Hardware einen lastunabhängigen Grundverbrauch von 0,615W geschätzt.

Mit den 89 laufenden Recheneinheiten erhöht sich die Gesamt-Leistungsaufnahme des Entwicklerboards auf 7,205W. Die Differenz beider Messwerte stellt die Leistungsaufnahme des reinen Hardware-Schaltkreises dar. Dieser benötigt also lediglich 0,121W.

Es ergibt sich somit eine Energieeffizienz von 3678 MFLOPS pro Watt.

5. Bewertung

5.1 Rechenleistung

In einer wissenschaftlichen Untersuchung der Höchstleistung unter Idealbedingungen eines verwandten FPGAs^{iv} haben die Autoren eine Rechenleistung von 63,5 GFLOPS erreicht. Korrigiert man diesen Wert um den abweichenden Systemtakt (200 MHz, Faktor 1,6) und die größeren Ressourcen (2.520 DSPs, Faktor 1,46) ergeben sich 27,2 GFLOPS. Die prototypische Implementation war also nicht in der Lage, die belegten Ressourcen effizient zu nutzen.

In Abschnitt 0 wurde eine deutliche Anomalie erkannt, nämlich die übermäßige LUT-Nutzung, während DSP-Blöcke ungenutzt bleiben. Zusammen mit der Beobachtung, dass 80% der Zykluszeit von der FPU verbraucht wird, ist eindeutig zu sehen, dass die Effizienz des Beschleunigers maßgeblich von einer optimierten FPU beeinflusst wird. Eine derartige Optimierung war im Rahmen der Studie aus Zeitgründen nicht durchführbar.

Doch trotz der sehr ineffizienten Implementation lässt sich bereits ein Effizienzvorteil beobachten. Um das Potential nach oben abzugrenzen, wird im folgenden zusätzlich zu den tatsächlich beobachteten Werten eine rechnerisch erreichbare Rechenleistung angesetzt:

Die jetzige FPU ist im Vergleich mit FPU von normalen Prozessoren um einen Faktor 40 zu langsam. Performance-optimierte ARM-Prozessoren (z.B. der Cortex-A72, der aus einer vergleichbaren Technologie-Generation wie der Xilinx UltraScale+ stammt) können Multiplikation und Addition mit 4 Takten Latenz durchführen, dank Pipelining dabei aber 2 Operationen pro Takt erreichen.

Auch wenn eine solche FPU einen enormen Ressourcenbedarf haben dürfte, sollte es möglich sein, eine für den Beschleuniger geeignete FPU mit deutlich kürzerer Latenz zu entwerfen. Im RISC-V-Ökosystem gibt es ressourcensparende und schnelle FPU^v. Basierend auf verfügbarer Hardware sollte sich die Zykluszeit um einen Faktor 4 bei vergleichbarem Ressourcenbedarf reduzieren lassen.

Damit erreicht der Beschleuniger theoretisch 1,78 GFLOPS, was immer noch ein Faktor 15 unter der oben berichteten Höchstleistung ist. Dieser Wert besteht jedoch aus fest bestimmten homogenen Operationen, während die FPU beliebige, ständig wechselnde Operationen durchführt. Dieser verbleibende Unterschied lässt sich daher vermutlich nicht eliminieren.

Im Vergleich mit dem ARM Cortex-A72 (siehe Abschnitt 0) ist diese Rechenleistung geringer, aber wie erwartet in der gleichen Größenordnung: In einer 4-Kern-Konfiguration mit 1,4 GHz erreicht der

ARM bei LINPACK ca. 5,5 GFLOPS^{vi}. Damit ist die FPGA-Implementation auf vergleichbarem Niveau in der absoluten Leistung.

Eine Extrapolation auf ASIC-Technologie ist schwer, jedoch eignet sich die Parallelisierung für eine breite Spanne an Konfigurationen. Ein ASIC kann auf einem Chip wesentlich mehr Recheneinheiten haben.

5.2 Kosteneffizienz

Die Kosteneffizienz ist unzweifelhaft schlecht. Selbst unter idealen Bedingungen erreicht der FPGA-Beschleuniger nur 2,27 MFLOPS/€. Für den Vergleichsprozessor konnte kein Stückpreis ermittelt werden, aber Entwicklerboards kosten um 30 Euro, was 183 MFLOPS/€ ergibt. Wie erwartet ist ein FPGA-Beschleuniger entweder ein Zwischenschritt zu einer ASIC-Implementierung oder wird in Szenarien mit streng begrenzten Energie-Budgets genutzt, die höhere Kosten rechtfertigen.

5.3 Energieeffizienz

Das wahre Potential ergibt sich bei der Energieeffizienz. Selbst wenn man nur die gemessenen Werte ansetzt, kommt man auf eine Energieeffizienz von 3,7 GFLOPS/W. Der Vergleichsprozessor verbraucht dagegen grob 7W bei LINPACK, was 0,79 GFLOPS/W ergibt.

Rechnet man mit der theoretisch erreichbaren Rechenleistung, dann kommt der FPGA-Beschleuniger sogar auf 14,7 GFLOPS/W. Hier kommt man in Regionen, wo ein Vergleich mit der vermutlich energieeffizientesten Rechenhardware der 16nm-Generation möglich ist, einer Nvidia GP100-Grafikkarte. Diese ist natürlich hochoptimiert und damit energieeffizienter als ein FPGA-Beschleuniger, aber der Unterschied fällt gering aus. Belastbare Zahlen sind leider nicht zu finden, aber aus den Spezifikationen ergibt sich eine Energieeffizienz von bestenfalls 22 GFLOPS/W.

Untersuchungen haben ergeben, dass der Unterschied zwischen FPGA- und ASIC-Designs selbst im ungünstigsten Fall ein Faktor 5 und oftmals ein Faktor 10 ist^{vii}. Rechnet man dies noch ein, würde ein ASIC-Beschleuniger eine hochoptimierte, in gleicher Technologie gefertigte GPU in der Energieeffizienz deutlich übertreffen.

6. Zusammenfassung

In dieser Untersuchung wurde geprüft, wie groß das Potential für eine FPGA-Implementierung der paceval-Software ist. Dabei wurde ein minimaler Prototyp entwickelt, der genug Funktionalität enthält, um belastbare Messungen zu ermöglichen.

Anders als die aktuell gebräuchlichsten Beschleuniger (SIMD/Vektor-Einheiten, Grafikprozessoren) ist der paceval-Beschleuniger in der untersuchten Architektur darauf optimiert, komplexe Berechnungen auf verhältnismäßig wenig Eingabedaten auszuführen. Die Funktionsweise des paceval-Algorithmus ermöglicht es zudem, die Berechnung in der Hardware automatisiert extrem stark zu parallelisieren. Das gibt ihm ein funktionales Alleinstellungsmerkmal.

Der erstellte Prototyp liefert bereits überzeugende Ergebnisse im Vergleich zu existierenden energieeffizienten Prozessoren. Die detaillierten Messungen erlauben es darüber hinaus, das volle Potential dieser Technik abzuschätzen, sollten im weiteren Verlauf eine breite Palette von Optimierungsmöglichkeiten genutzt werden. Diese konnten im Rahmen dieser ersten Machbarkeitsstudie nicht umgesetzt werden. Insbesondere das eigentliche Gleitkomma-Rechenwerk bietet erhebliches Optimierungspotential (ca. Faktor 4). Im RISC-V-Ökosystem wurden bereits geeignete freie Hardwarealternativen sondiert, die diese Lücke schließen könnten. Weiteres Optimierungspotential innerhalb der bestehenden Lösung ergibt sich aus der aktuell geringen Taktfrequenz der Hardwarelösung (ca. Faktor 2). Insgesamt ergibt sich so aus technologieunabhängigen Maßnahmen ein geschätztes Verbesserungspotential in der Performance von Faktor 8.

Bei der Energieeffizienz, die in IoT und Edge-Anwendungen äußerst wichtig ist, zeigen die Messwerte und Analysen bereits jetzt einen signifikanten Vorteil gegenüber herkömmlichen Prozessoren. Diese Kennwerte könnten durch die Optimierung der Synthese-Parameter (z.B. mehr dedizierte DSP Blöcke) deutlich steigen.

Die Ergebnisse (Performance und Energieeffizienz) ließen sich darüber hinaus nochmal erheblich steigern, wenn der paceval-Beschleuniger in einem ASIC-Prozess umgesetzt werden würde. Der Technologiewechsel lässt eine Verbesserung der Energieeffizienz um den Faktor 5-10 erwarten. Pro Chipfläche kann die erreichbare Rechenleistung sicherlich um einen Faktor 10 wegen höherer Taktfrequenz und wegen mehr Parallelität des Rechenpfads um einen weiteren Faktor 10 steigen.

Damit ergibt sich für eine ASIC-Realisierung ein klares Potential, für komplexe mathematische Berechnungen konkurrenzfähig zu existierenden Beschleuniger-Produkten (beispielsweise Grafikkarten der namhaften Hersteller) zu sein.

Es ist dabei wichtig festzuhalten, dass selbst mit der aktuellen Architektur und Implementierung bereits konkurrenzfähige und bzw. überlegene Kennwerte erreicht werden konnten.

7. Quellenangaben

ⁱ https://github.com/freecores/fpu_double

ⁱⁱ <https://docs.xilinx.com/r/en-US/ug949-vivado-design-methodology/Static-Power>

ⁱⁱⁱ <http://web.archive.org/web/20230731101332/https://shop.trenz-electronic.de/en/TE0808-05-9BE21-LZ-UltraSOM-MPSoC-Module-with-AMD-Zynq-UltraScale-ZU9EG-1E-4-GB-DDR4-LP?c=242>

^{iv} Calore, Enrico, and Sebastiano Fabio Schifano. "Energy-efficiency evaluation of FPGAs for floating-point intensive workloads." *Parallel Computing: Technology Trends*. IOS Press, 2020. 555-564.

^v Bertaccini, Luca, et al. "Tiny-FPU: low-cost floating-point support for small RISC-V MCU cores." *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2021.

^{vi}

<http://www.roylongbottom.org.uk/Raspberry%20Pi%203B%20and%203B+%20High%20Performance%20Linpack%20and%20Error%20Tests.htm>

^{vii} Kuon, Ian, and Jonathan Rose. "Measuring the gap between FPGAs and ASICs." *Proceedings of the 2006 ACM/SIGDA 14th international symposium on Field programmable gate arrays*. 2006.

Anhang D: SAP-Empfehlungsschreiben



SAP SE
Dietmar-Hopp-Allee 16
69190 Walldorf
Deutschland
T +49 6227 7-47474
F +49 6227 7-57575
E info@sap.com

SAP SE, Dietmar-Hopp-Allee 16, 69190 Walldorf

Bundesministerium für Bildung und
Forschung
53170 Bonn

12.07.23
Suat Tabanlı
Head of Discrete Industry Business Networks
T +49 6227 7-769078
E s.tabanli@sap.com

Betreff: Empfehlungsschreiben für das Projekt „Design-Instrumente für souveräne Chipentwicklung mit Open-Source (DE:Sign)“ der Zukunftsstrategie Forschung und Innovation der Deutschen Bundesregierung

Sehr geehrte Damen und Herren des Bundesministeriums für Bildung und Forschung,

Die SAP Industry Cloud erweitert die Intelligent Suite mit innovativen Industry Cloud-Lösungen, die von SAP und unseren Partnern basierend auf der SAP® Cloud Platform in der Cloud entwickelt wurden und auf der Cloud-Infrastruktur von SAP und unseren Hyperscaler-Partnern laufen. Das Ergebnis ist eine offene Geschäfts- und Technologieplattform für das Kerngeschäft unserer Kunden in ihren jeweiligen Branchen.

Industry Cloud Lösungen sind cloud-nativ und basieren auf der SAP Business Technology Platform unter Verwendung intelligenter Technologien wie Analytics und künstlicher Intelligenz. Diese Aktivitäten sind aus unserer Sicht besonders relevant für automatisierte Prozesse und Entscheidungen in den vielfältigen Branchen, die SAP mit Ihren Systemen unterstützt.

Die begrenzte Effizienz der Software-Berechnungen verhindert derzeit jedoch noch einen breiteren Einsatz dieser Techniken. Hier sind die Fähigkeiten heutiger Prozessoren noch unzureichend da, insbesondere die Infrastrukturkosten, wie aktuell auch die gestiegenen Stromkosten einen erheblichen Kostenfaktor bei unseren Kunden bedeuten. Hardwarebeschleuniger sind aus unserer Sicht ein geeignetes Mittel, diesem Mangel entgegenzutreten. Als potenzielle Nutzer derartiger Hardware würde uns der Open-Hardware-Ansatz zudem Planungssicherheit und Resilienz in der Lieferkette geben.

Die Entwicklung eines neuartigen Hardware-Beschleunigers für Analytics und künstlicher Intelligenz bewerte ich positiv und unterstütze unseren Mitarbeiter Herrn Jörg Könning bezüglich der Möglichkeit, die von ihm geschaffene Softwarelösung paceval mit dem Institut für Informatik OFFIS in Oldenburg in Hardware umzusetzen. Für Rückfragen stehe ich Ihnen gerne zur Verfügung.

Mit freundlichen Grüßen,

SAP SE
Sitz der Gesellschaft: Walldorf, Deutschland
vertreten durch ihren Vorstand: Christian Klein (CEO), Dominik Asam, Sabine Bendiek, Dr.-Ing. Jürgen Müller, Scott Russell, Thomas Saueressig, Julia White
Vorsitzender des Aufsichtsrats: Hasso Plattner / Registergericht Mannheim HRB 719915
Deutsche Bank AG, Heidelberg: IBAN DE78 6727 0003 0091 2030 00, SWIFT-BIC DEUT DE SM 672 / Steuernummer 32497/82215, USt-ID DE143454214

Abbildungsverzeichnis - Validierungsauftrag Abschlussbericht

Abbildung 1: MPSoC Modul mit AMD Zynq UltraScale+ ZU4EG-1E (5.2 x 7.6 cm)	7
Abbildung 2: Energieeffizienz der Programmiersprachen	10
Abbildung 3: Funktionsweise des FPGA.....	11
Abbildung 4: Das ZC706 Evaluation Board	11
Abbildung 5: High-level Zync-7000 Blockdiagramm	12
Abbildung 6: Zynq-7000 XC7Z045 SoC Blockdiagramm.....	13
Abbildung 7: Ausgabe von phpinfo über das ZC706 Entwicklungsboard mittels Lighthttpd und PHP	14
Abbildung 8: Öffnen der Fritz!Box Benutzeroberfläche einfach im Internetbrowser über die Adresse http://fritz.box.....	14
Abbildung 9: Verarbeitung der Berechnungen mit der mathematischen Engine	16
Abbildung 10: Ausgabe des Beispiels im Browser	17
Abbildung 11: Eine neue Berechnung über die Referenz des vorherigen Beispiels	18
Abbildung 12: Blockdiagramm der hardware-basierten mathematischen Engine as a Service auf dem ZC706 SoC	22
Abbildung 13: Applikationsentwicklung mit der Bibliothek von paceval. auf einem Compiler aus dem Jahr 2000 und einem heutigen Compiler	23
Abbildung 14: Programmierschnittstelle über Foreign function interface (FFI).....	23
Abbildung 15: Laufzeiten von anderen Lösungen und paceval	24
Abbildung 16: Problemstellung bei mathematischen Ungenauigkeiten	24
Abbildung 17: Entscheidungsbaum	25
Abbildung 18: Neuronales Netz	25
Abbildung 19: Funktionsweise Docker Container.....	27
Abbildung 20: Skalierung des paceval -service bei einem Hyperscaler	28
Abbildung 21: Schema der Skalierung für die Hyperscaler mit dem Referenzdesign und Kubernetes	30
Abbildung 22: Python Programmierung mit dem paceval -service	30
Abbildung 23: Schema wie das Python Programm per Python Modul, mit dem Referenzdesign bei einem Hyperscaler interagiert	31
Abbildung 24: Das batteriebetriebene Gerät hier eine Drohne interagiert lokal mit dem Referenzdesign (links) oder über einen Hyperscaler (rechts).....	33
Abbildung 25: Ausführung des MNIST Benchmarks über den HTTP-Service auf dem ZC706 SoC.....	35
Abbildung 26: Die Oberfläche des APPLE Mac Studio Testservers mit dem paceval -service Dienst	38
Abbildung 27: Messung paceval. auf APPLE Mac Studio	44
Abbildung 28: Weltweiter Energieverbrauch der Rechenzentren und der Netzwerkinfrastruktur	46
Abbildung 29: Quantitativer Vergleich der Funktionalitäten von GPUs und FPGAs.....	47
Abbildung 30: Quantitativer Vergleich der Funktionalitäten von GPUs, FPGAs und dem gesteckten Ziel dieses Validierungsauftrags.	48
Abbildung 31: Vergleich bfloat16 mit 7bit Genauigkeit der Mantisse gegenüber 23bit bei IEEE 754 float	49
Abbildung 32: MNIST Erkennung durchgeführt mit paceval . und eingeschalteter Intervallarithmetik für float (23bit Mantisse), double (52bit Mantisse) und long double (≥63bit Mantisse) Genauigkeit.....	50
Abbildung 33: Illusion von Muffins und Hundegesichtern	51
Abbildung 34: Standardansatz beim maschinellen Lernen	51
Abbildung 35: Umfang des erneuten Lernens	52
Abbildung 36: Vorgehensweise der US-FDA bei Änderungen an zuvor genehmigten medizinischen Geräten mit neuronalen Netzen	53
Abbildung 37: Kontinuierliche Optimierung des neuronalen Netzes mit Patches	54
Abbildung 38: Schnellere Produktion und Veröffentlichungen durch Patches	54
Abbildung 39: Erwartetes Marktpotential für KI-Hardware (Quelle: Precedence Research).....	57

Abbildung 40: Die geplante Zielarchitektur des Beschleunigers	81
Abbildung 41: Zustandsmaschine der Kontrolllogik in der prototypischen Implementation. IF = Instruction Fetch; DF = Data Fetch	82
Abbildung 42: Illustration des Detailgrades, in dem die simulierte Funktionalität beobachtet werden kann.....	84
Abbildung 43: Messaufbau für die Energiemessung	85
Abbildung 44: Bildschirmfoto von der Oszilloskop-Beobachtung des FPU_READY Signals (lila) und des Systemtakts (grün).....	86
Abbildung 45: Ressourcenauslastung des vermessenen Prototypen.....	87